

软件工程基础

第3章：需求工程

杨溢龙

北航软件学院

目 录

Contents

软件工程基础

第5章：需求工程

Part 1 需求基本概念

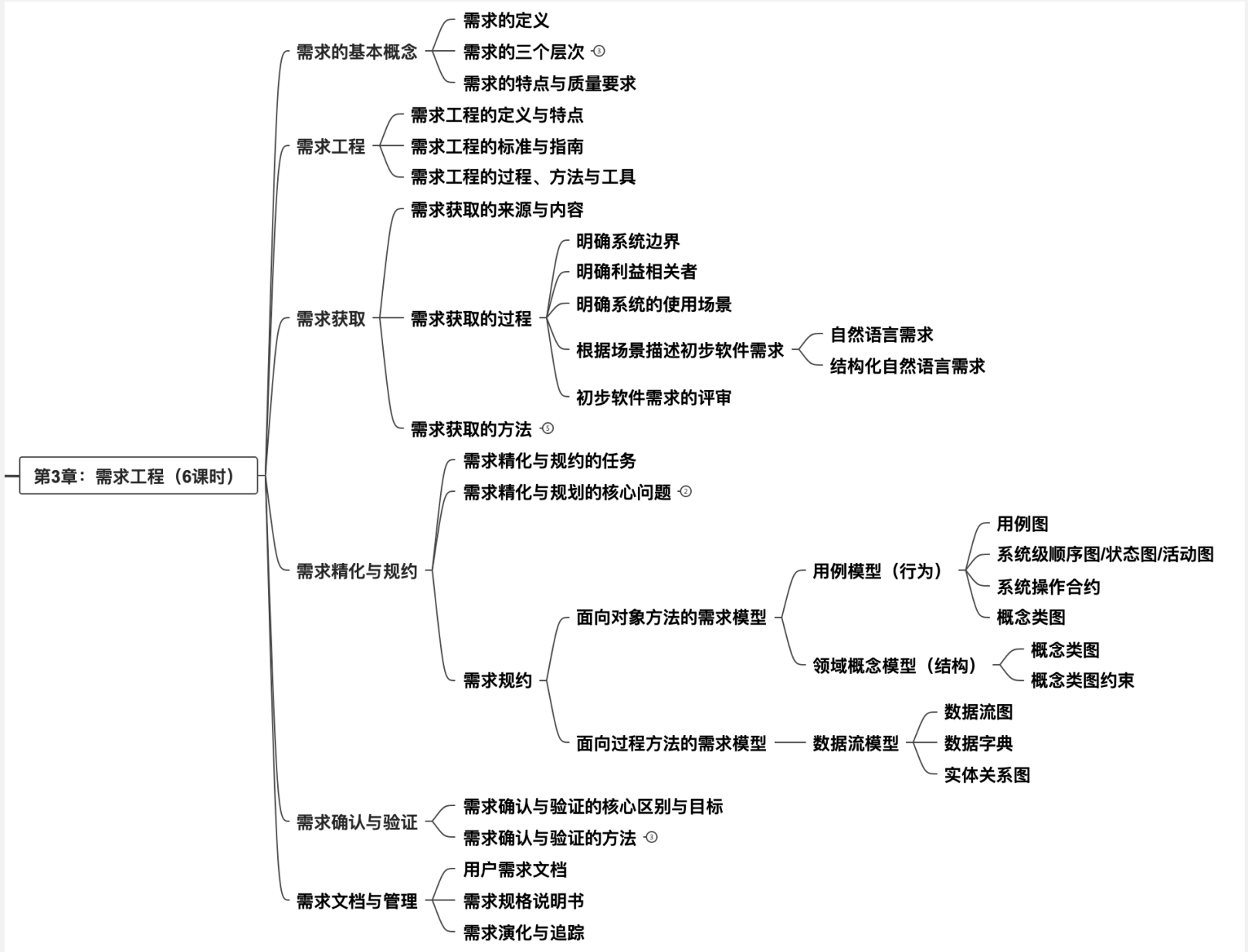
Part 2 需求工程

Part 3 需求获取

Part 4 需求精化与规约

Part 5 需求确认与验证

课时安排：需求工程


第3章：需求工程（6课时）
需求的基本概念
需求的定义
需求的三个层次 ③
需求的特点与质量要求
需求工程
需求工程的定义与特点
需求工程的标准与指南
需求工程的过程、方法与工具
需求获取
需求获取的来源与内容
需求获取的过程
明确系统边界
明确利益相关者
明确系统的使用场景
根据场景描述初步软件需求
自然语言需求
结构化自然语言需求
初步软件需求的评审
需求获取的方法 ⑤
需求精化与规约
需求精化与规约的任务
需求精化与规划的核心问题 ②
需求规约
面向对象方法的需求模型
用例模型（行为）
用例图
系统级顺序图/状态图/活动图
系统操作合约
概念类图
领域概念模型（结构）
概念类图
概念类图约束
面向过程方法的需求模型
数据流模型
数据流图
数据字典
实体关系图
需求确认与验证
需求确认与验证的核心区别与目标
需求确认与验证的方法 ③
需求文档与管理
用户需求文档
需求规格说明书
需求演化与追踪

北京航空航天大學
BEIHANG UNIVERSITY



Part 0

上节课回顾

主观题 10分

设置

软件过程模型主要有哪些？开发时怎么进行选择？

作答

主观题 10分

设置

软件管理的对象有哪些？你认为最重要的方面是什么？

作答

北京航空航天大學
BEIHANG UNIVERSITY



Part 1

需求
基本概念

Part 1 需求的基本概念

- 需求的定义
- 需求的特点与质量要求
- 需求的类型
- 可行性分析

主观题 10分

设置

软件开发的本质是什么?

作答

软件开发的本质

软件工程过程、方法和工具

用户要求

软件需求



软件工程师

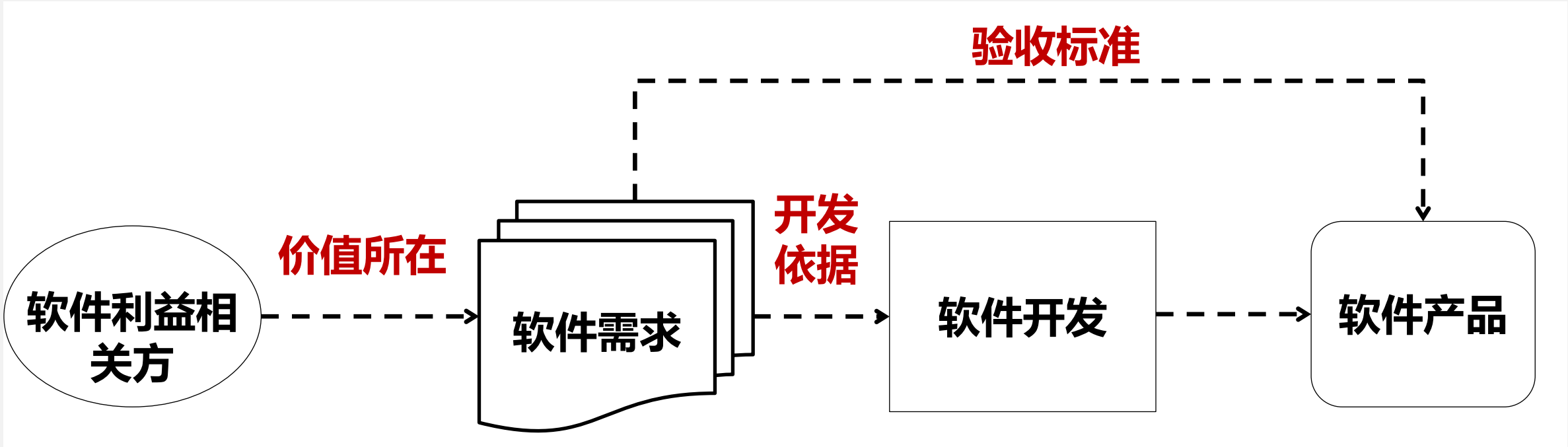
文档、模型、程序、数据

软件系统

开发软件系统的前提是要明确用户的期望和要求，即需求

需求重要性

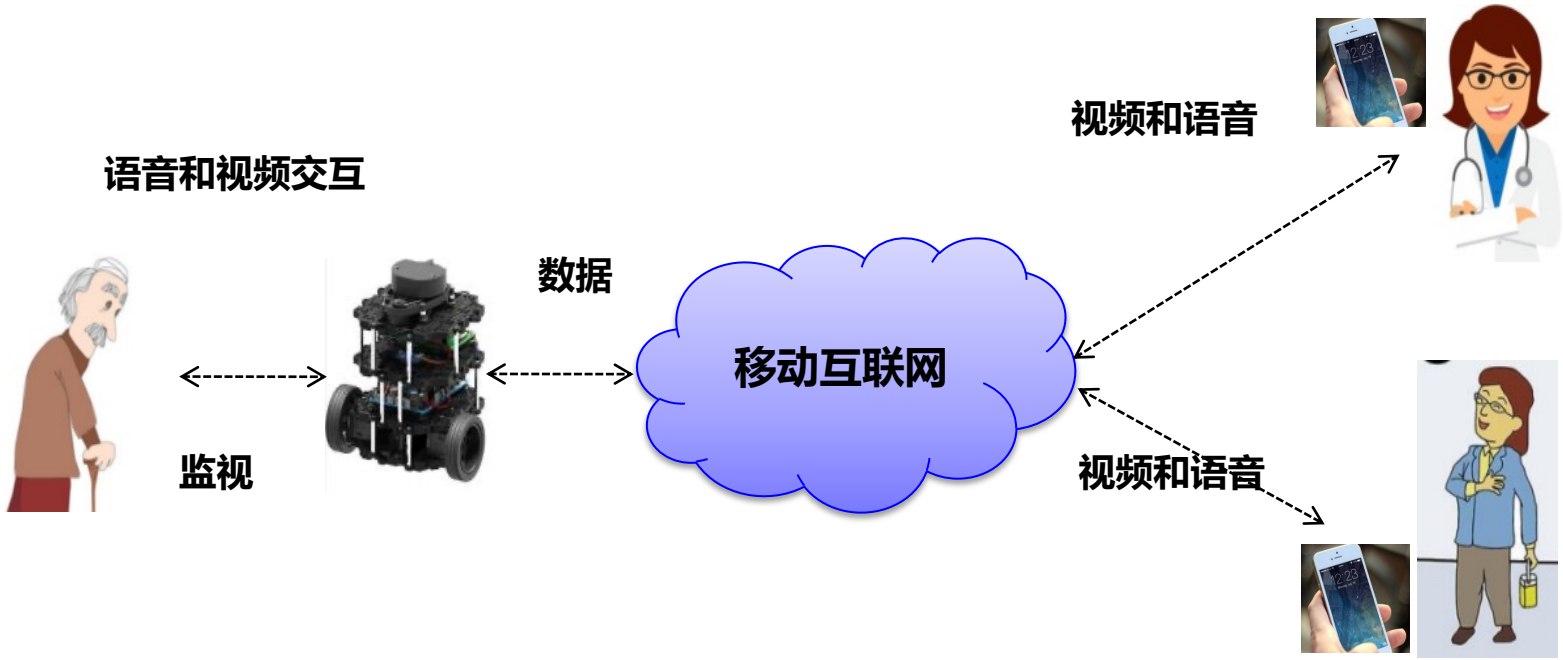
- 软件**价值**所在
- 软件开发的**基础和前提**
- 软件验收的**标准和依据**



主观题 10分

设置

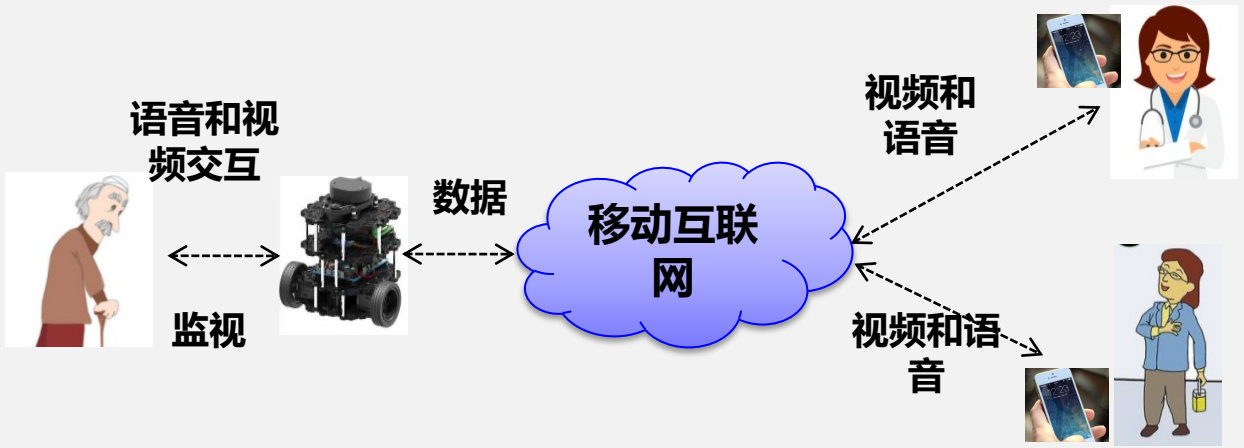
场景：开发空巢老人看护系统，
请站在不同人的角度， 思考其核心需求是什么?



作答

空巢老人看护系统（站在使用者角度）

- **通过软件将自主机器人、智能手机等设备相结合，对家中独居老人进行监护**
  - ✓跟踪老人在家情况
  - ✓老人与远端的家属进行语音和视频交互
  - ✓发现和通告异常情况（如摔倒、突发疾病）
  - ✓将老人在家状况（如图像和视频）和异常信息传送到远端家属或医生的智能手机上
  - ✓通过语音进行呼叫和报警
  - ✓提醒老人按时服药和保健
  - ✓……



需求定义 (Requirements)

1997年，IEEE软件工程标准词汇表中定义需求如下：

- 用户解决问题或达到目标所需的条件或能力（Capability）
- 系统或系统部件要满足合同、标准、规范或其他正式规定文档所需具有的条件或能力
- 反映上述条件或能力的文档说明

解读标准：何为需求?

- 从软件**利益相关者**的角度，需求是指软件系统的**利益相关方**对软件系统的**功能和质量**，以及软件运行环境、交付进度等方面提出的**期望和要求**

- 从**软件本身**的角度，**需求**是指软件用于解决现实世界问题时所表现出的**功能和性能等方面的要求**

- 软件需求刻画了软件系统能做什么（**What to do**），应表现出怎样的**行为**，需满足哪些方面的**条件和约束**等要求

软件系统的利益相关者

❑何为利益相关者(stakeholder)

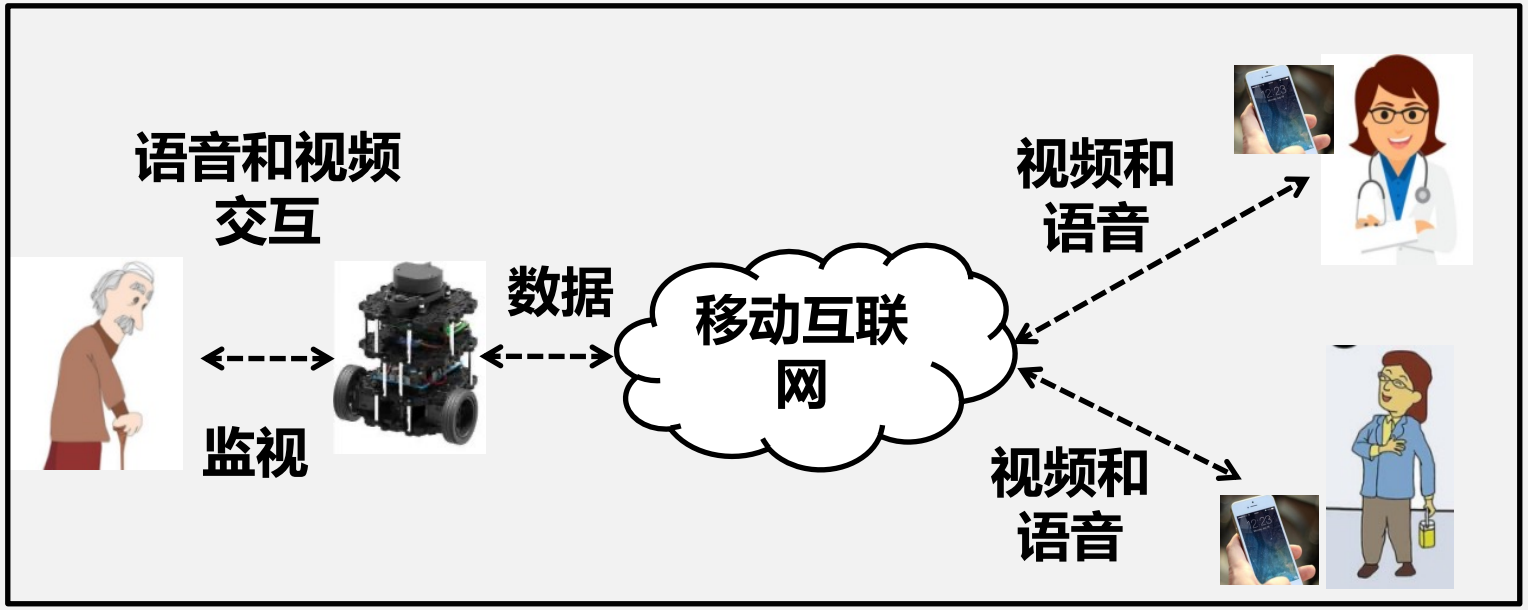
- ✓从软件系统中**受益**或与软件系统**相关**的**人、组织或者系统**
- ✓**受益**：使用、获益、盈利
- ✓**相关**：发生操作和交互、存在关联性

❑软件利益相关者的表现形式

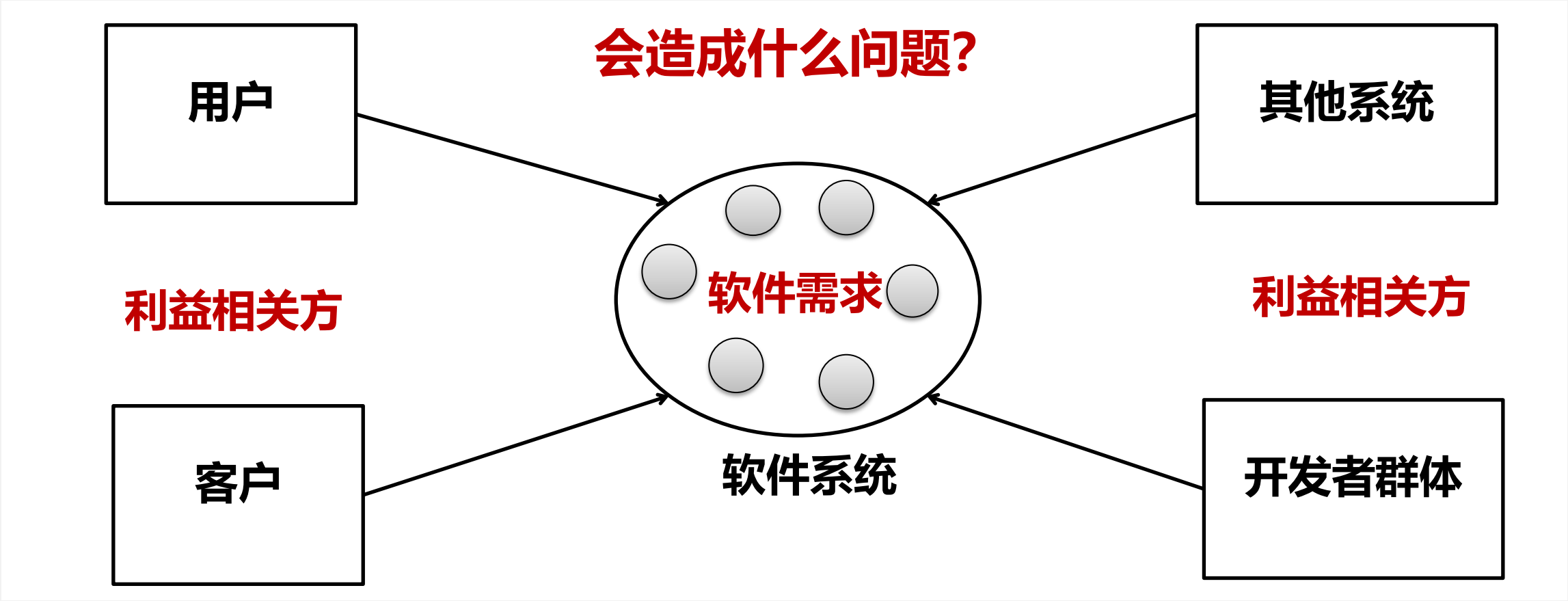
- ✓**用户**：最终使用软件的人
- ✓**客户**：从中获取利益的组织
- ✓**系统**：与待开发系统进行交互的系统
- ✓**开发者**：负责开发软件系统的人

示例：空巢老人看护软件的利益相关方

- **用户**
  - ✓老人
  - ✓家属
  - ✓医生
- **客户**
  - ✓投资方
- **系统**
  - ✓机器人



软件需求及其利益相关方



软件利益相关方会站在自身的角度对软件提出要求

主观题 10分

设置



携程订票系统作为一个综合旅行服务平台，提供机票、酒店、火车票、旅游套餐等多种服务







- ❑ **用户的期望有哪些?**
- ❑ **其他利益相关者期望有哪些?**
- ❑ **可能存在哪些冲突?**

作答

利益相关者对软件需求差异和冲突

用户期望

- 全面的服务选项：提供从机票、酒店到旅游套餐等一站式旅行服务
- 价格优惠：寻求最优惠的价格，包括特价机票和酒店房间
- 易用性：一个直观易用的界面，便于快速找到和预订所需服务
- 灵活的搜索和过滤功能：能够根据不同需求（如时间、价格、地点）灵活筛选
- 可靠的客户服务：问题和紧急情况下的快速响应与解决方案
- 安全的支付方式：多种安全的支付选项和保障个人信息安全
- 个性化推荐：根据历史行为和偏好提供个性化的旅行建议

其他利益相关者期望

- 供应商（如航空公司、酒店）：希望通过携程提高自己的销售额，同时希望能有合理的合作费用
- 投资者：期望公司盈利增长，股价上升
- 合作伙伴（如旅游局、地方政府）：希望通过携程推广当地旅游，增加旅游收入

利益相关者对软件需求差异和冲突 （续）

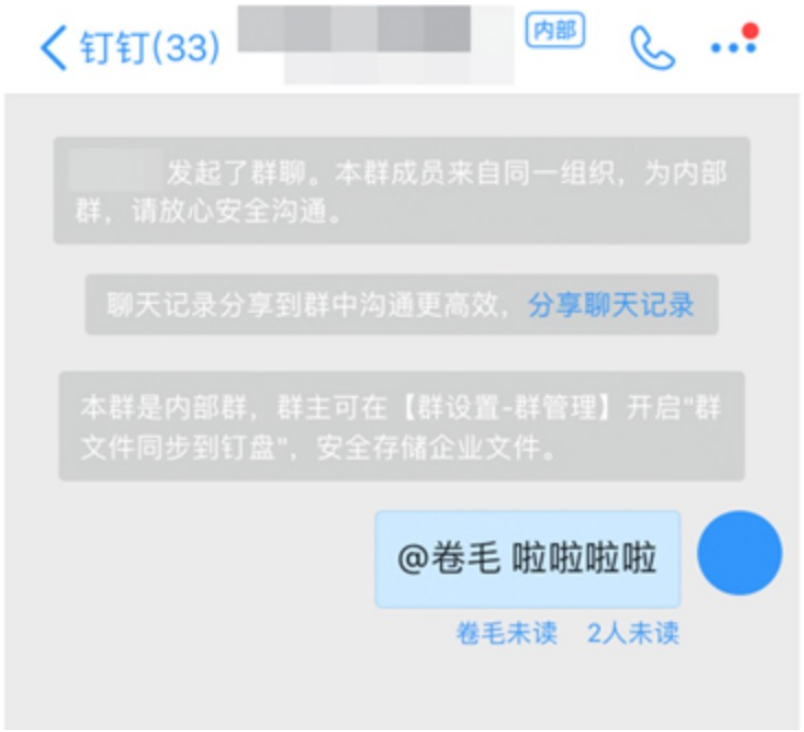
可能存在的冲突

- **价格与利润的冲突**：
  - ✓ 用户希望获得最低的价格，而供应商和携程需要保持一定的利润率。这可能导致携程在提供折扣和特价服务时与供应商之间存在矛盾
- **质量与成本的冲突**：
  - ✓ 用户期望高质量的服务（如即时客服），这可能会增加携程的运营成本，而携程和投资者则可能更关注成本控制
- **个性化与隐私保护的冲突**：
  - ✓ 用户期望获得个性化的旅行推荐，这需要携程收集和分析用户的个人信息。然而，用户同时也非常关心自己的隐私保护，这之间的平衡是一大挑战
- **推广活动与供应商利益的冲突**：
  - ✓ 携程可能会举办促销活动来吸引用户，但过度的促销可能会损害供应商的正常定价策略和品牌形象

主观题 10分

设置

❑ 微信是否需要消息已读功能?



❑ 微信的红包上限为何设置为200元?



作答

产品经理面试



1. 需不需要显示消息已读?



2. 微信红包上限问题

Part 1 需求的基本概念

- 需求的定义
- 需求的特点与质量要求
- 需求的类型
- 可行性分析

主观题 10分

设置

根据前面的介绍，思考一下软件需求的特点?

作答

软件需求的特点（1/2）

- **隐式性**
  - ✓来自于利益相关方，它隐式存在
  - ✓很难辨别，甚至会遗漏掉
- **隐晦性**
  - ✓在利益相关方的潜意识之中，**不易于表达**出来，**难以获取**
  - ✓所表达的软件需求存在**模糊性、歧义性、二义性**
- **多源性**
  - ✓存在多个的利益相关方
  - ✓存在相冲突和不一致的软件需求

软件需求的特点（2/2）

- **易变性**
  - ✓用户对软件的期望和要求也会经常性地发生变化
  - ✓在整个生命周期都会发生变化
- **领域知识的相关性**
  - ✓软件需求的内涵与软件所在领域的知识息息相关
  - ✓“12306”与铁路旅客服务领域相关
- **价值不均性**
  - ✓不同的软件需求对于客户或用户而言所体现的价值是不一样的
  - ✓主要和次要、核心和外围需求

如何从利益相关者获取完整、清晰和一致软件需求是一项挑战！

主观题 10分

设置

- **如果软件需求存在以下问题会给软件开发带来什么样的挑战，产生怎样的后果?**
  - 提不出有价值的软件需求
  - 不清晰的软件需求：没有说清楚
  - 不完整的软件需求：漏掉了重要的软件需求
  - 不一致的软件需求：对同一个需求项有不同的表述

作答

软件需求质量要求（1/2）

- **有价值（Valuable）**
  - ✓基于计算机软件的解决方案，有效提高问题解决的效率和质量，促进相关领域的业务创新
- **正确（Right）**
  - ✓反映利益相关方的期望，不能曲解或误解他们的要求
- **完整（Complete）**
  - ✓不能有遗漏或丢失
- **无二义（Unambiguous）**
  - ✓软件需求的描述应该是清晰和准确的

软件需求的质量要求（2/2）

- **可行（Feasible）**
  - ✓在技术、经济等方面应该是可行的
- **一致（Consistent）**
  - ✓不应存在冲突
- **可追踪（Traceable）**
  - ✓可追踪到其源头
- **可验证（Verifiable）**
  - ✓可找到某种方式来检验软件需求是否在软件系统中得到实现

Part 1 需求的基本概念

- 需求的定义
- 需求的三个层次
- 需求的质量要求
- 可行性分析

主观题 10分

设置

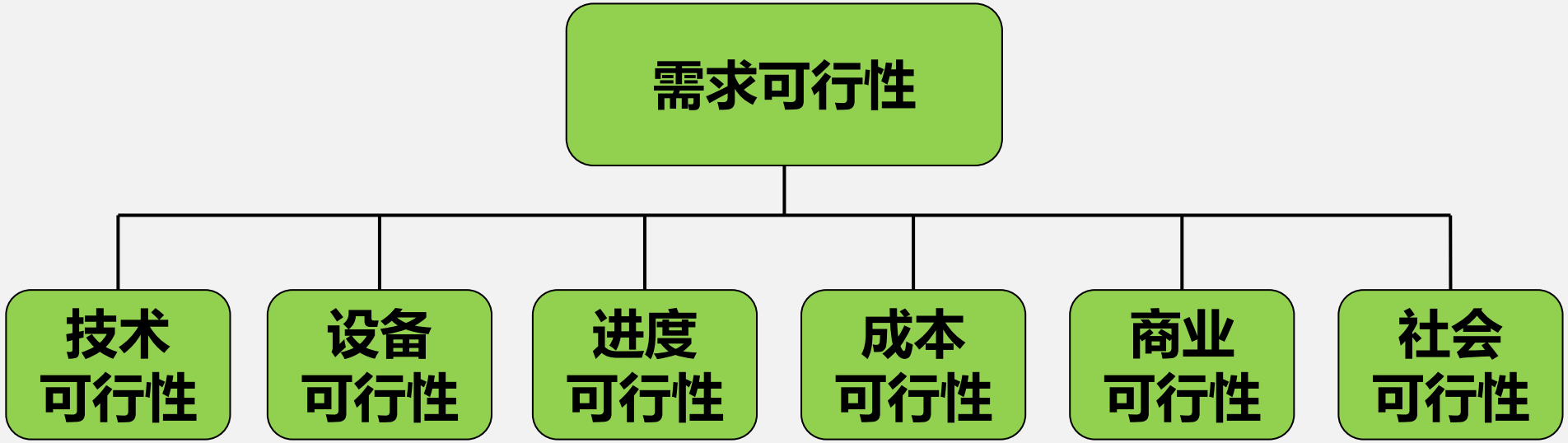
- 项目开发前评估“是否能解决 + 是否值得解决“
- 不关注如何去解决问题

思考一下可行性分析，应该从哪些侧面去分析?

作答

软件需求可行性分析

- 项目开发前评估“是否能解决 + 是否值得解决“
- 不关注如何去解决问题



软件需求可行性分析

❑技术可行性

- ✓研究就是要回答使用现有的技术能否实现待开发的系统的问题。

❑设备可行性

- ✓基于软件的解决方案需要哪些设备或系统，软件项目团队是否已经具备这些设备和系统

❑进度可行性

- ✓软件用户或客户对软件提出什么样的进度要求，针对开发团队的人力资源及技术水平，能否遵循进度约束开发出满足这些需求的软件产品

软件需求可行性分析

❑成本可行性

✓待开发系统的经济效益能否超过它的开发成本的问题。通过成本效益分析，经济可行性研究从经济角度去判断是否值得为该系统投资。

❑商业可行性

✓软件需求是否有商业价值，能否获得用户的青睐以及市场的认可，针对这些软件需求的投入能否获得预期的收益

❑社会可行性

✓当前软件系统已经成为国家和社会的重要组成成分，需要从社会的角度，评估软件的各项需求是否违背社会道德、文化伦理、法律法规、行业标准等，或者与它们是否存在冲突

可行性研究报告

✓可行性研究报告

- 项目背景
- 管理概要
- 候选方案
- 系统描述
- 经济可行性
- 技术可行性
- 社会可行性
- 其他可行性
- 其他与项目有关的问题

北京航空航天大學
BEIHANG UNIVERSITY



Part 2

Requirements Engineering

需求工程

Part 2 需求工程

- 需求工程的定义与特点
- 需求工程的标准与指南
- 需求工程的方法与工具
- 需求工程的过程模型

主观题 10分

设置

根据需求的特点和软件工程的概念，考虑一下为什么需求阶段也需要用工程的方法进行？有哪些主要的环节？

作答

需求工程的重要性

- 在需求中会存在**大量的错误**，这些错误若未即使发现和更正，会引起**软件开发费用增加**、**软件质量降低**；严重时会造成**软件开发的失败**（Failure）
- **失败的软件工程项目进行失败原因分析**
  - ✓ **需求不完整 15.1%**
  - ✓ **缺少用户参与 12.4%，**
  - ✓ **需求/需求规格说明书更改 8.7%**

约1/3的项目失败都与需求有关，需求获取中的错误属软件开发中的早期错误，将给项目成功带来极大风险

何为需求工程?

- 用工程的理念和方法来指导软件需求实践，它提供了一系列的过程、策略、方法学和工具，帮助需求工程师加强对业务或领域问题及其环境的理解，更精准获取和分析软件需求，指导软件需求的文档化和评审，以尽可能获得准确、一致和完整的软件需求，产生软件需求的相关软件制品

需求工程的目标是获取高质量的软件需求



Requirements Engineering

- Requirements engineering is interdisciplinary function that mediates between the domains of the acquirer and supplier to establish and maintain the requirements to be met by the system, software or service of interest. (**ISO/IEC/IEEE 29148**)

- Requirements engineering is concerned with discovering, eliciting, developing, analyzing, verifying, validating, communicating, documenting and managing requirements (**ISO/IEC/IEEE 29148**)

需求工程的特点

- **知识密集型工作，需要交叉多学科的知识**
  - ✓“12306”软件系统就需要充分掌握铁路旅行服务的相关知识，包括火车调度、车次安排、票务服务等等
- **多方共同参与**
  - ✓软件需求的获得需要多方人员的共同参与，包括不同类别的用户、客户、领域和业务专家、各类开发者、质量保证人员等等
- **需求获取的多种形式和源头**
  - ✓获取、构思、创作等，要采用多种形式和手段
- **持续迭代和逐步推进**
  - ✓贯穿于软件整个生命周期，如“12306”APP软件

Part 2 需求工程

- 需求工程的定义与特点
- 需求工程的标准与指南
- 需求工程的方法与工具
- 需求工程的过程模型

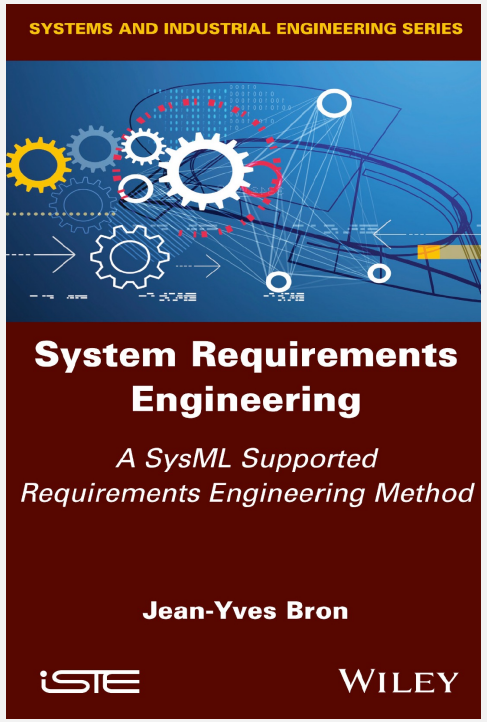
需求工程标准

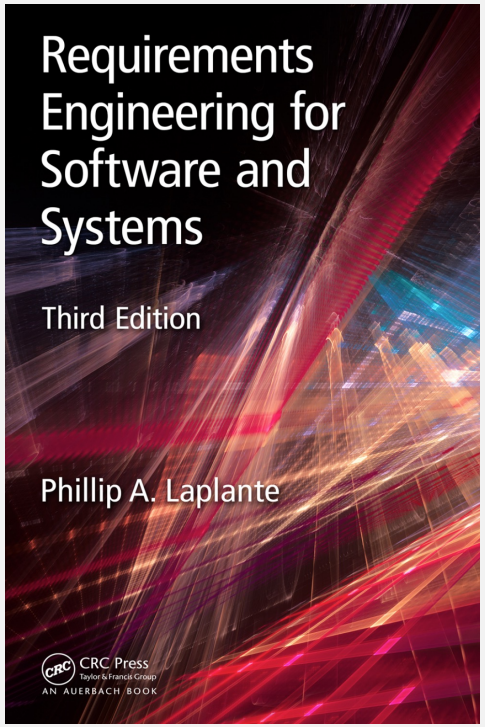
- ISO/IEC/IEEE 15288：Systems and software engineering - System life cycle processes
- ISO/IEC/IEEE 12207：Systems and software engineering - Software life cycle processes
- ISO/IEC/IEEE 29148：Systems and software engineering - Life cycle processes - Requirements engineering

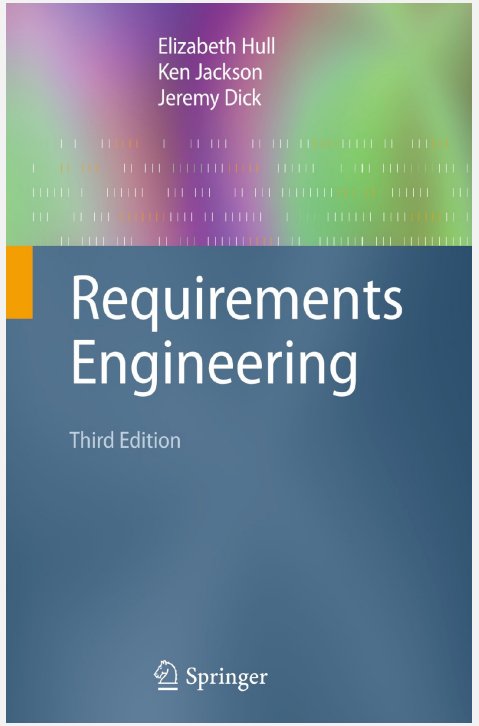
需求工程指南及参考书目

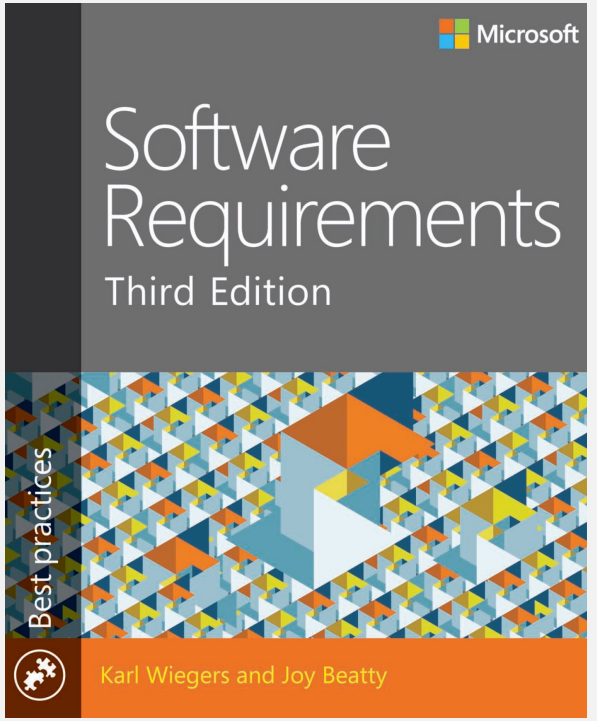
- IREB - A Glossary of Requirements Engineering Terminology
- INCOSE - Guide for Writing Requirements
- IEEE 830 -IEEE Recommended Practice for Software Requirements Specifications







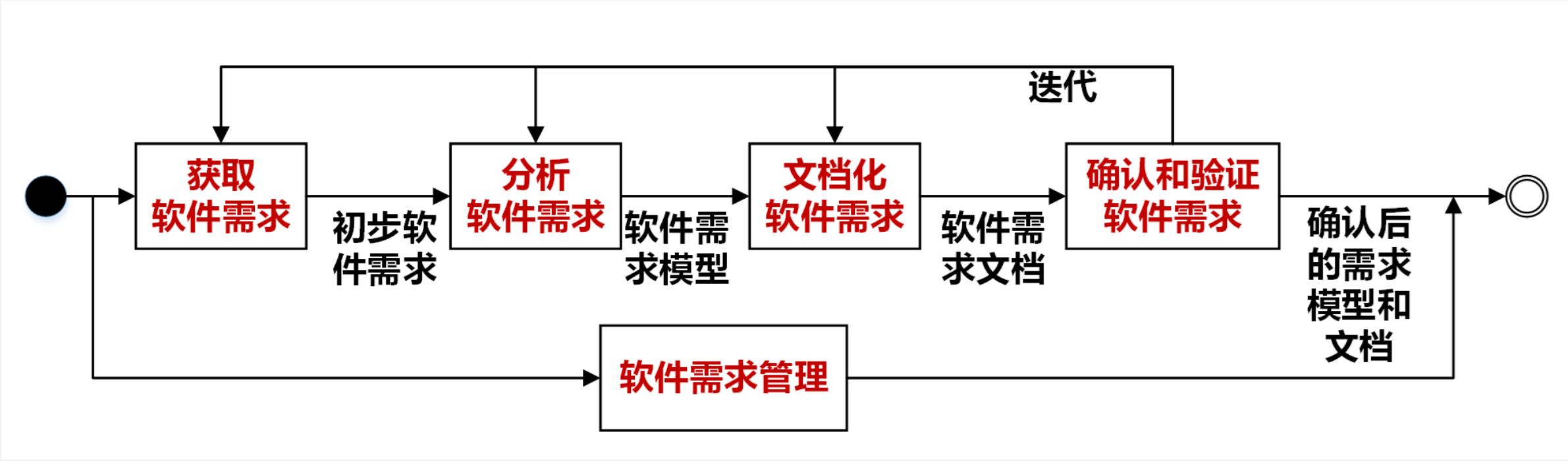




Part 2 需求工程

- 需求工程的定义与特点
- 需求工程的标准与指南
- 需求工程的过程、方法与工具

需求工程：一般性过程



需求过程是一个迭代过程，每次迭代提供更高质量更详细的软件需求

需求工程过程模型

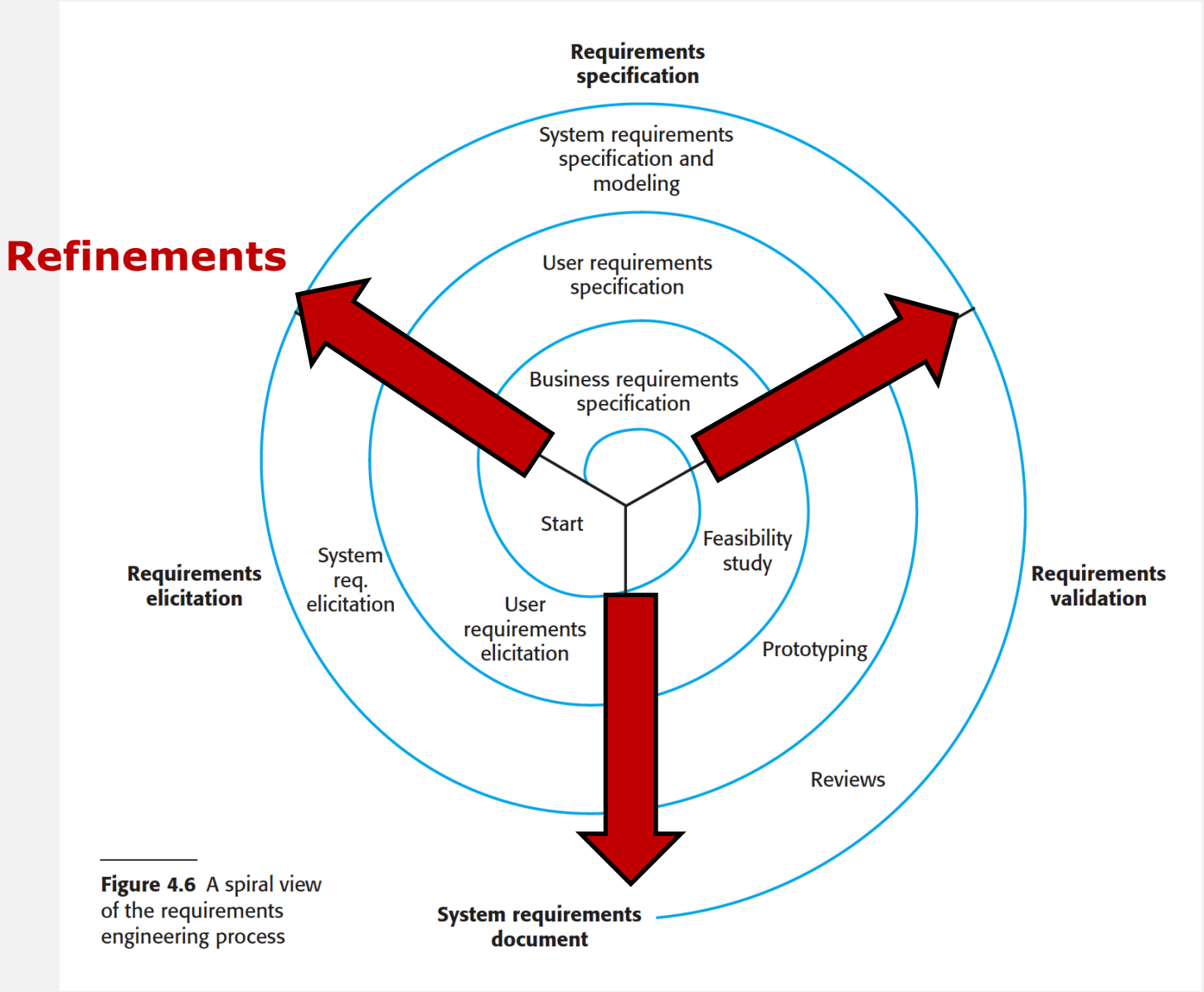


Figure 4.6 A spiral view of the requirements engineering process

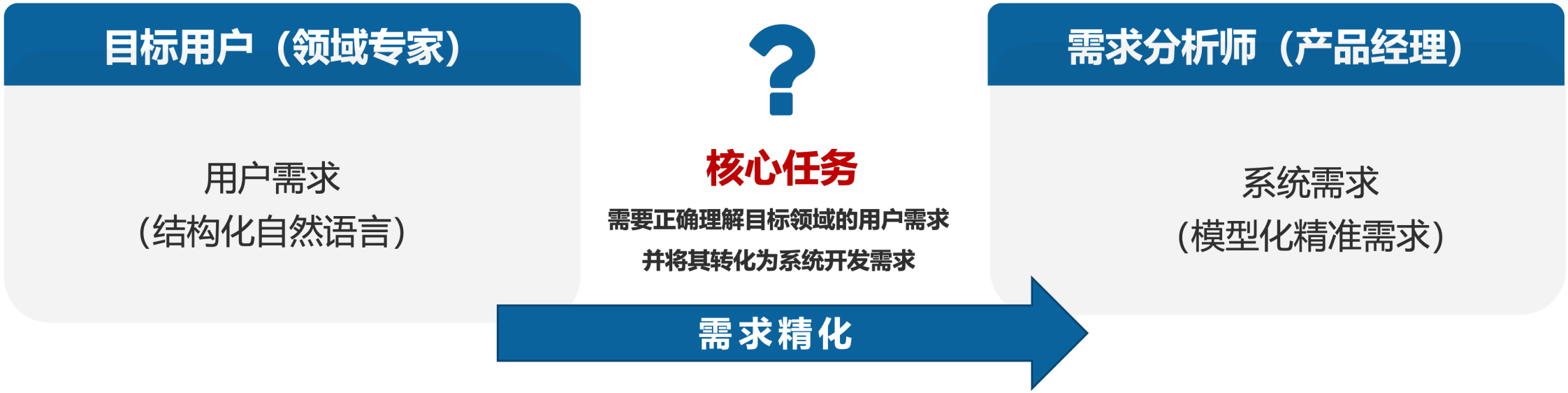
需求过程：螺旋模型

- 三层次：
  - 业务、用户、系统需求
- 三阶段：
  - 需求获取 (Elicitation)
  - 需求精化与规约 (Refinement&Specification)
  - 需求确认与验证 (Validation&Verfication)

核心问题

两层次跨越：

- **内容：用户的初始需求->系统的开发需求**
- **形态：结构化自然语言需求->精准的模型化需求**



需求的三个层次

- **需求是指软件系统必须满足的所有功能、性能和限制。将用户对软件的一系列要求、想法转变为软件开发人员所需要的有关软件的技术规格说明，它涉及以下三个方面：**
  - 面向企业的需求（业务需求）
  - 面向用户的需求（用户需求）
  - 面向开发者的系统需求（系统/软件开发需求）

业务需求

- 业务需求 (Business Requirement)表示组织或客户高层次的目标。
- 业务需求通常来自项目投资人、购买产品的客户、实际用户的管理者、市场营销部门或产品策划部门。
- 业务需求描述了组织为什么要开发一个系统，即组织希望达到的目标。
- 使用前景和范围(vision and scope)文档来记录业务需求,这份文档有时也被称作项目轮廓图或市场需求(project charter 或 market requirement)文档。

用户需求

❑ 定义

- ✓ 关于软件的一系列想法的集中体现、涉及软件的**功能**、操作方式**界面风格**、报表**格式**、用户机构的**业务范围**、工作**流程**以及用户对软件应用的期望等

❑ 获取方式

- ✓ 用户主动提出 or 通过与用户的沟通、交流、问卷调查
- ✓ 由于用户对计算机系统认识的不足，分析人员有义务帮助用户挖掘需求

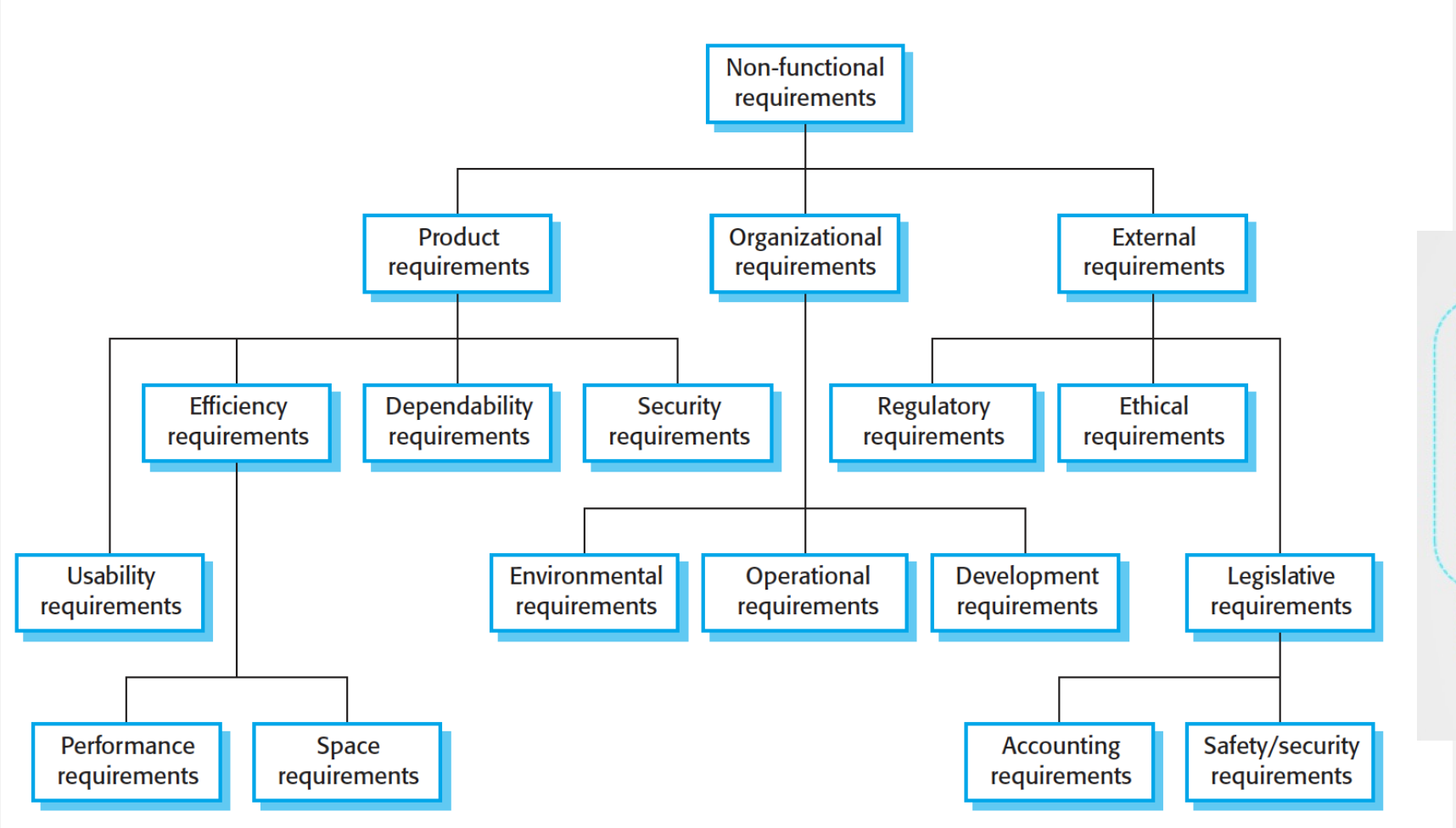
❑ 提供形式

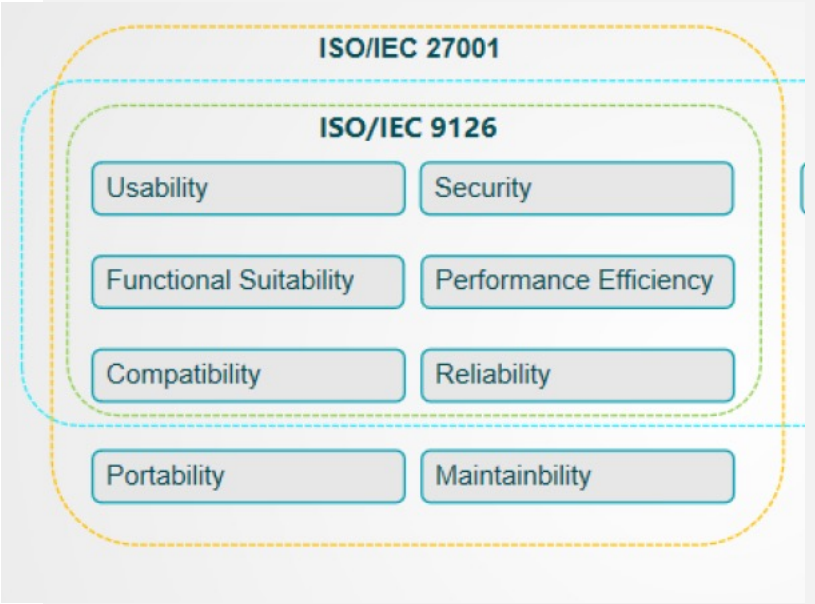
- ✓ 以**文档**的形式提供给用户**审查**，需要使用流畅（精准）的**自然语言**和简洁清晰的**图表**进行表述，方便用户的理解与确认

系统需求（软件需求）

- **系统需求**是比用户需求更具有**技术特性**的需求陈述，提供给**开发者**或用户方**技术人员**。软件开发人员设计系统的**基本依据**，对系统的功能、性能、数据等方面进行规格定义
- **分类**
  - ✓ **功能性需求** **（系统做什么）**
    - 能够完成的**功能**及在某些场景下可展现的**外部可见行为（规定系统不应该做什么）**
  - ✓ **非功能性需求** **（系统特性或约束）**
    - **外部质量**属性：**软件外部可展现的，用户关心的，**如运行性能、可靠性、易用性等
    - **内部质量**属性：**隐藏在软件内部的，软件开发工程师关心的，**如可扩展性、可维护性、可理解性等
    - **开发约束**：开发成本、交付进度、技术选型、遵循标准等方面提出的要求

系统需求：非功能性需求ISO标准





ISO/IEC标准

示例：空巢老人看护软件的需求

❑功能性需求

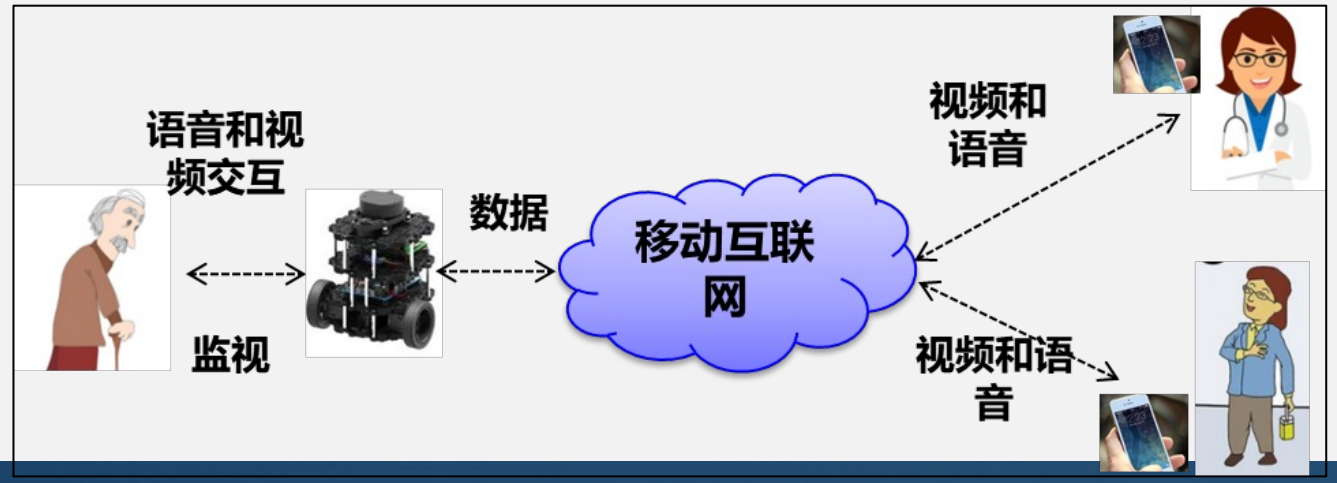
✓自主跟随老人、获取老人图像和视频信息、检测老人是否摔倒等

❑质量方面的需求

✓始终保持在2米的安全距离，对机器人的控制在2秒内响应等

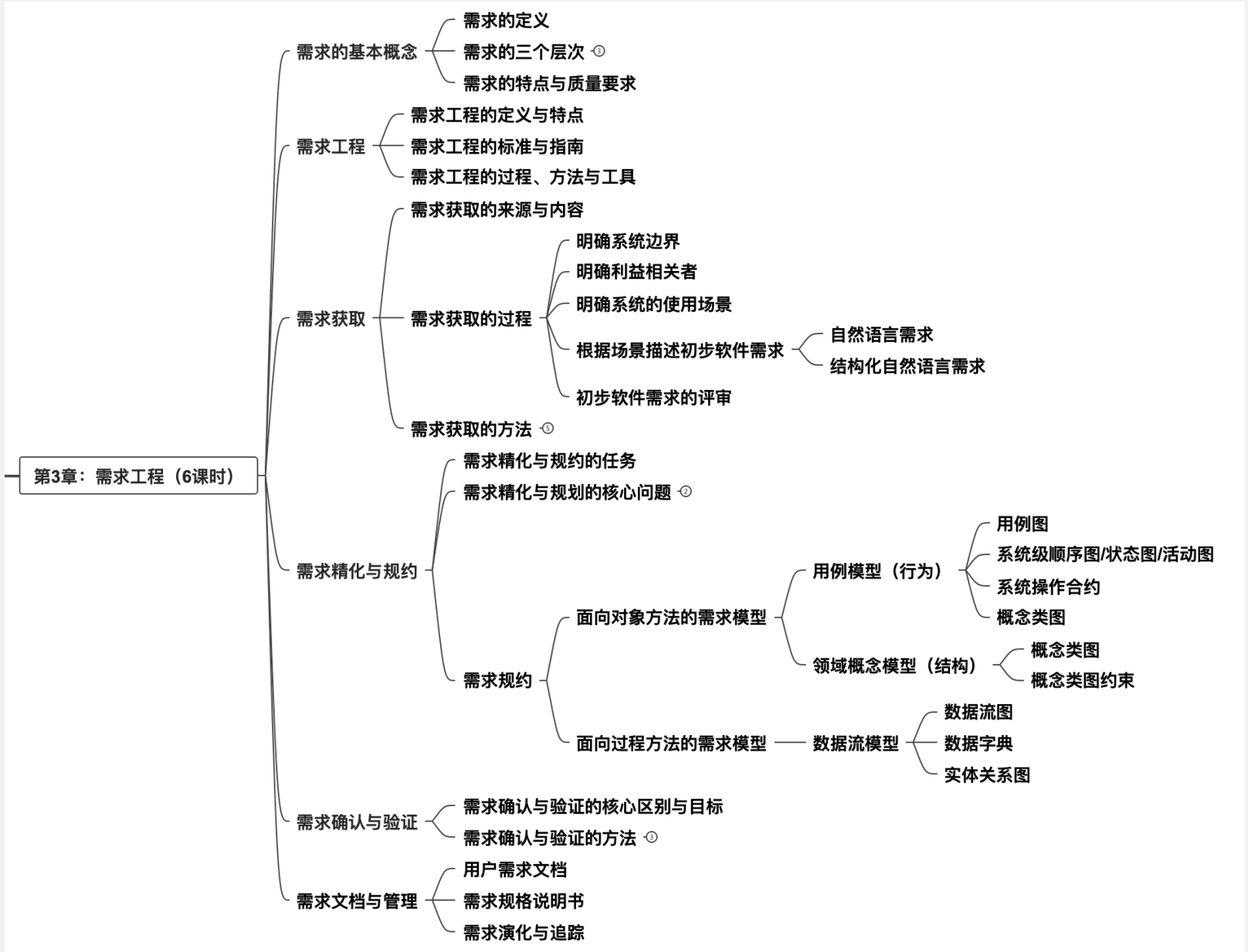
❑约束性需求

✓成本不能超出50万元，要求半年内交付使用等等



这些需求都是谁提出来的?

课时安排：需求工程


第3章：需求工程（6课时）
需求的基本概念
需求的定义
需求的三个层次 ③
需求的特点与质量要求
需求工程
需求工程的定义与特点
需求工程的标准与指南
需求工程的过程、方法与工具
需求获取
需求获取的来源与内容
需求获取的过程
明确系统边界
明确利益相关者
明确系统的使用场景
根据场景描述初步软件需求
自然语言需求
结构化自然语言需求
初步软件需求的评审
需求获取的方法 ⑤
需求精化与规约
需求精化与规约的任务
需求精化与规划的核心问题 ②
需求规约
面向对象方法的需求模型
用例模型（行为）
用例图
系统级顺序图/状态图/活动图
系统操作合约
概念类图
领域概念模型（结构）
概念类图
概念类图约束
面向过程方法的需求模型
数据流模型
数据流图
数据字典
实体关系图
需求确认与验证
需求确认与验证的核心区别与目标
需求确认与验证的方法 ③
需求文档与管理
用户需求文档
需求规格说明书
需求演化与追踪

需求工程过程模型（复习）

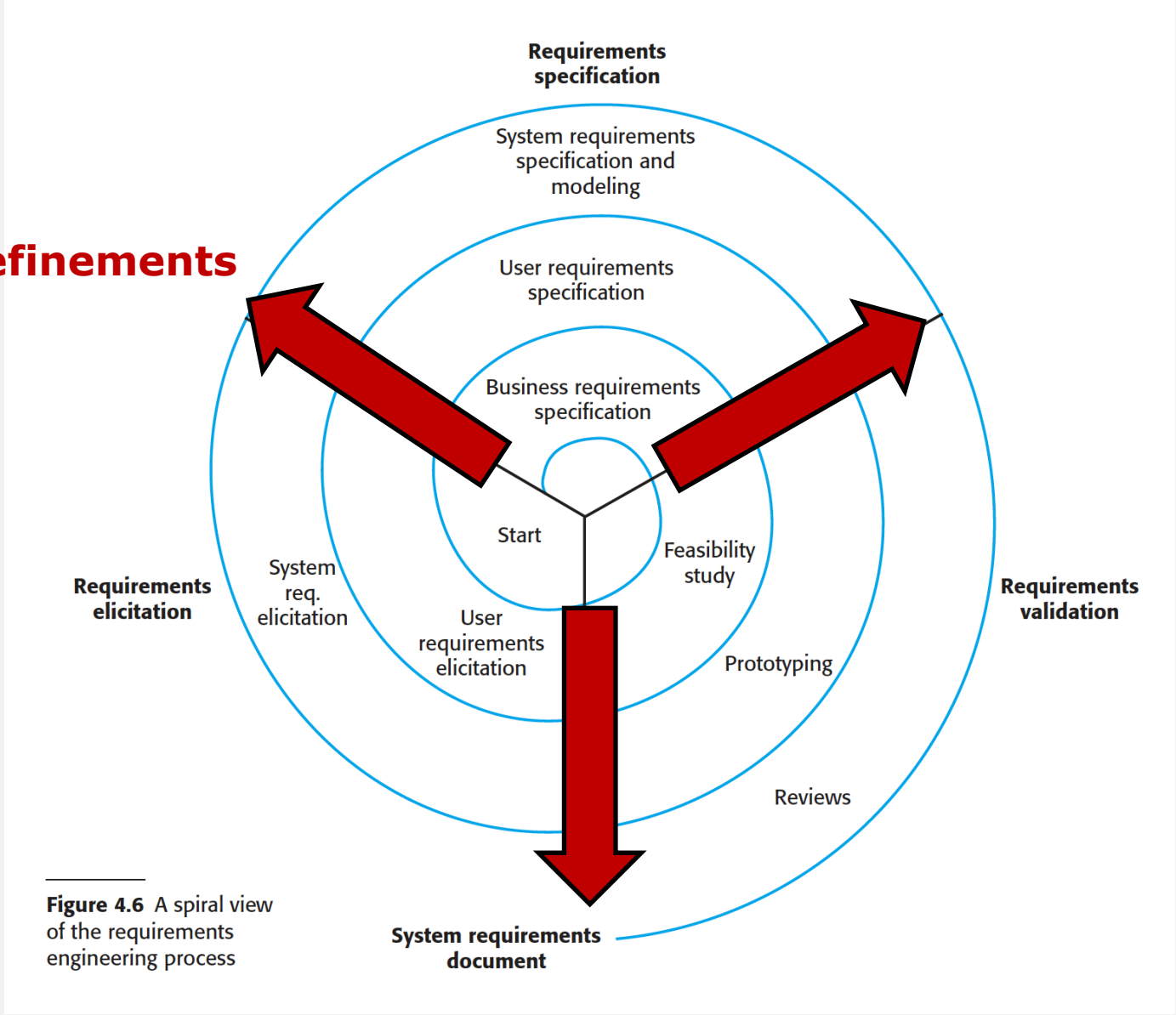


Figure 4.6 A spiral view of the requirements engineering process

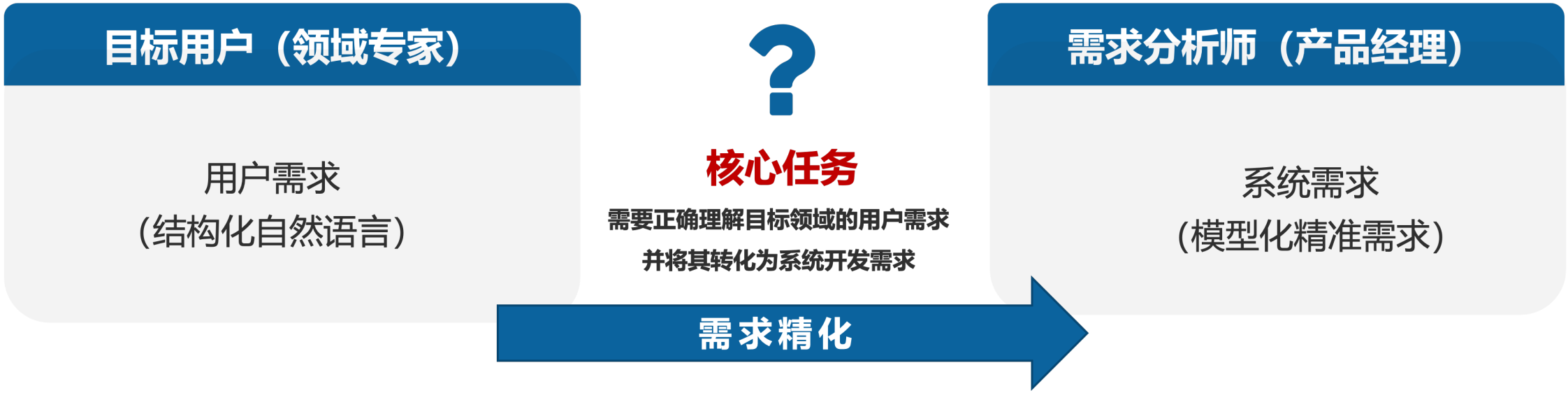
需求过程：螺旋模型

- 三层次：
  - 业务、用户、系统需求
- 三阶段：
  - 需求获取 (Elicitation)
  - 需求精化与规约 (Refinement&Specification)
  - 需求确认与验证 (Validation&Verfication)

核心问题（复习）

两层次跨越：

- **内容：用户的初始需求->系统的开发需求**
- **形态：结构化自然语言需求->精准的模型化需求**



需求工程三阶段（1/3）- 需求获取

核心问题：如何抽取和理解软件需求？软件需求本质是什么？应采用什么样的抽象来刻画软件需求？

- **需求获取+理解：开发者与用户共同抽取的过程**
  - ✓ 划分：捕获问题空间的**“整体/部分”**关系
  - ✓ 抽象：捕获问题空间的**“一般/特例”**关系
  - ✓ 投影：捕获问题空间的**多维“视图”**
- **需求分类+优先级排序+协商**
- **需求文档化**
  - ✓ 对需求进行文档化并提供给后续流程，形成早期草稿即可

需求工程三阶段（2/3）- 需求表示（规约）

核心问题：如何刻画和建立软件需求模型？如何清晰、直观地表达软件需求，便于理解和交流

- **自然语言：采用自然语言或结构化自然语言**
  - ✓ 存在描述不直观、二义性和模糊性等问题
- **可视化模型：图形化的需求建模语言**
  - ✓ 数据流图、UML图
- **文档：需求规格说明书**
  - ✓ 解决需求中存在的二义性和不一致性
  - ✓ 最后以一种系统化的文档形式，准确地表达用户的需求，即需求分析规格说明书

需求工程三阶段（2/3）- 需求表示（规约）

图形化的需求建模语言

- **20世纪70年代结构化数据和数据流抽象**
  - ✓功能表现为**数据处理**，要理解软件功能需求，就要知道软件具有哪些数据以及要对这些数据进行什么样处理
  - ✓提出了一组抽象来表征软件的功能性需求，包括**数据流、数据字典、数据加工或处理、数据源**等，设计了数据流图建模语言
- **20世纪90年代面向对象抽象**
  - ✓功能表现为系统中**对象所展示的行为**，要理解软件功能需求，就要知道软件有哪些对象、有什么样**行为、交互和协作**
  - ✓设计面向对象建模语言UML，产生了面向对象需求分析方法学

软件需求的缺陷

- **需求缺失**，即漏掉了一些重要的软件需求
- 需求描述**不正确**，对软件需求的理解存在偏差
- 需求描述**不准确**，软件需求的表述与用户的要求不一致
- 软件需求**有冲突**、不一致
- 软件需求**不可行**，存在可行性问题
- 软件需求**不详尽**，没有提供足够详细的信息

需求工程三阶段（2/3）- 需求精化

❑如何精化和分析软件需求?

- ✓循序渐进地获得软件需求细节，进而逐步得到详细的软件需求

❑提供策略和手段

- ✓指导一步步地精化和分析软件需求
- ✓建立准确和一致的软件需求模型
- ✓防止漏掉重要的软件需求
- ✓发现并解决其中的问题和存在的缺陷，以保证软件需求的质量

需求工程三阶段（3/3）- 需求确认和验证

❑软件需求确认 （是不是用户要的）

- ✓**站在用户和客户的角度，确保软件需求的正确性**，通常采用需求评审（Review）、原型确认等方式。
- ✓例如，12306 App软件的开发者可邀请一些旅客，作为软件的用户代表，评审12306 App软件的需求文档以及所开发的软件原型，逐条确认各项软件需求的合法性和正确性
- ✓基于原型的确认是一种常用且有效的方式

❑软件需求验证 （是不是没有错的）

- ✓**判断软件需求文档和模型是否准确地刻画了用户和客户的要求**，后续的软件设计制品、程序代码等是否正确地实现了软件需求

软件需求变更管理

❑软件需求的变更管理

- ✓**多变性和易变性**引起的，明确哪些方面的需求发生了变化、反应在软件需求模型和文档的哪些部分、导致软件需求模型和文档的版本发生了什么样变化等

❑软件需求的追溯管理

- ✓开展**溯源追踪**，掌握清楚是谁提出需求变更、为什么要进行变更等内容，以判别需求变更的合法性；评估需求变更的影响域，基于对需求变更的理解，分析需求变更会对哪些软件制品会产生什么样的影响；评估需求变更对软件项目开发带来的影响

❑软件需求的配置管理

- ✓形成软件需求基线

需求工程相关的CASE工具

- 需求文档撰写工具，如借助于Microsoft Office、WPS、RM2PT（文档自动生成）
- 需求建模工具，如利用Microsoft Visio、Rational Rose、StarUML、RM2PT（需求建模）
- 软件原型开发工具，如Mockplus、Axure RP Pro、UIDesigner， RM2PT（原型自动生成）
- 需求分析和管理专用工具，如IBM Rational RequisitePro，RM2PT（错误检测与修复）
- 配置管理工具和平台，如Git、Github、Gitlab、PVCS、Microsoft SourceSafe等

北京航空航天大學
BEIHANG UNIVERSITY



Part 3

Requirements Elicitation

需求获取

回顾：需求工程过程模型

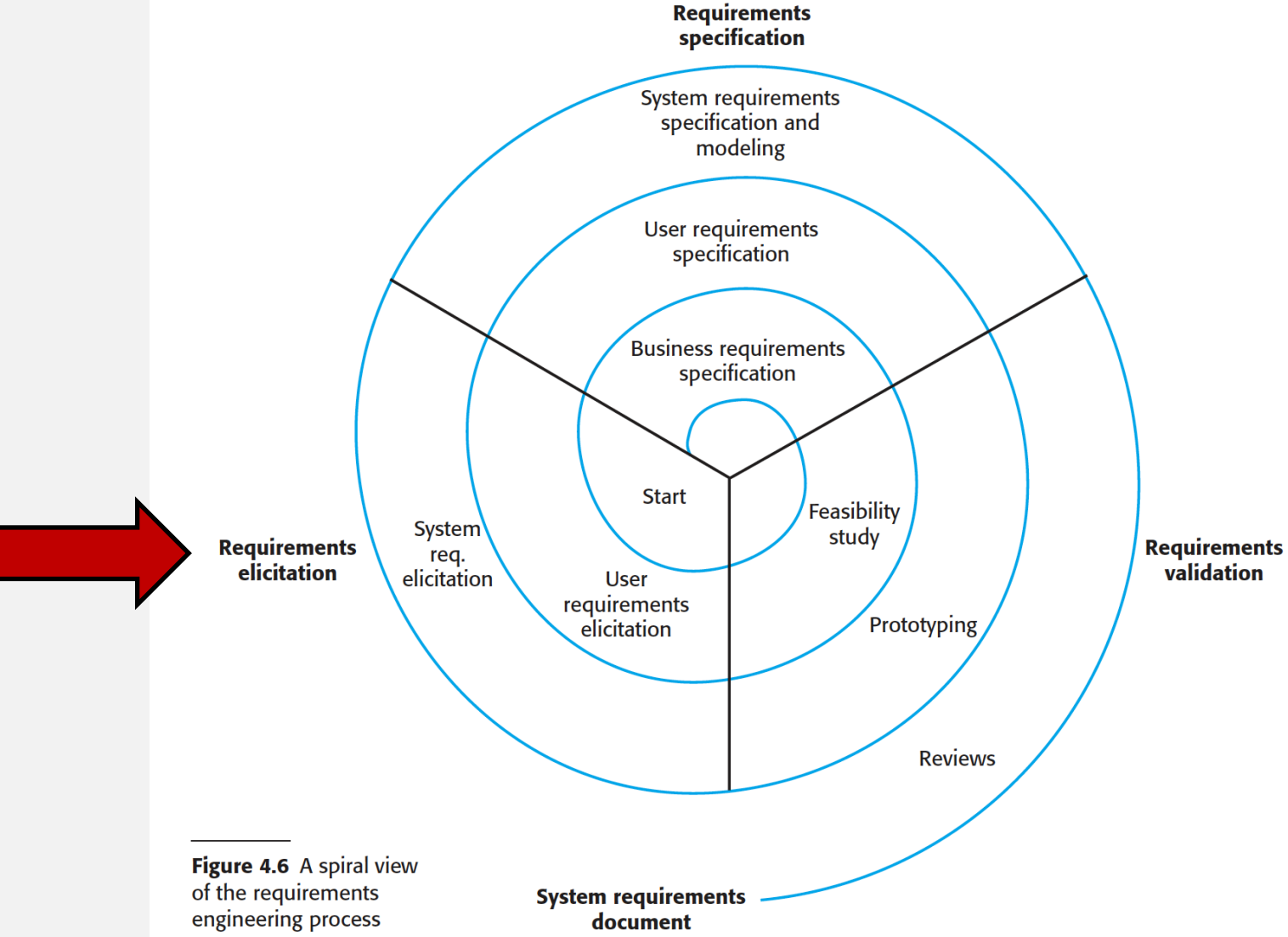


Figure 4.6 A spiral view of the requirements engineering process

需求过程：螺旋模型

- 三层：
  - 业务、用户、系统需求
- 三阶段：
  - 需求获取 (Elicitation)
  - 需求精化与规约 (Refinement&Specification)
  - 需求确认与验证 (Validation&Verfication)

Part 3 需求获取

- 需求获取的来源 – 从哪来？(Where)
- 需求获取的方法 – 怎么来？(How)
- 需求获取的过程 – 来的步骤？(How -> What)

Part 3 需求获取

- 需求获取的来源 – 从哪来?
- 需求获取的方法 – 怎么来?
- 需求获取的过程 – 哪些步骤?

软件需求从何而来?

❑软件开发的动机 （业务需求）- 解决什么问题

- ✓**解决特定行业和领域的问题，体现意义和价值**
- ✓ “MiNote”是为了帮助用户编写和管理便签； “12306”是为了帮助旅客解决购买火车票费时费力的问题； “ElderCarer”是为了解决空巢老人在家无人看护的问题。

❑软件利益相关方的诉求（用户需求）

- ✓软件系统的受益者，软件需求的提出者
- ✓利益相关方有哪些，他们会对软件提出什么样的期望和要求
- ✓ “12306”的利益相关者是旅客，服务于旅客的期望和要求

需求获取为什么难？（回顾）

- **隐式性**
  - ✓来自于利益相关方，它隐式存在
  - ✓很难辨别，甚至会遗漏掉
- **隐晦性**
  - ✓在利益相关方的潜意识之中，**不易于表达**出来，**难以获取**
  - ✓所表达的软件需求存在**模糊性、歧义性、二义性**
- **易变性**
  - ✓用户对软件的期望和要求也会经常性地发生变化
  - ✓在整个生命周期都会发生变化
- **领域知识的相关性**
  - ✓软件需求的内涵与软件所在领域的知识息息相关

软件开发者也可以充当软件的利益相关方

- **许多软件系统的需求来自软件工程师，而非最终的用户**
  - ✓微信、12306等等
- **他们充当软件的用户或客户，构思和提出软件需求**
  - ✓原因是找不到实际的用户来提出需求
- **开源软件的许多需求来自软件开发者，而非最终用户**
  - ✓提出软件的功能性构思Issues

Part 3 需求获取

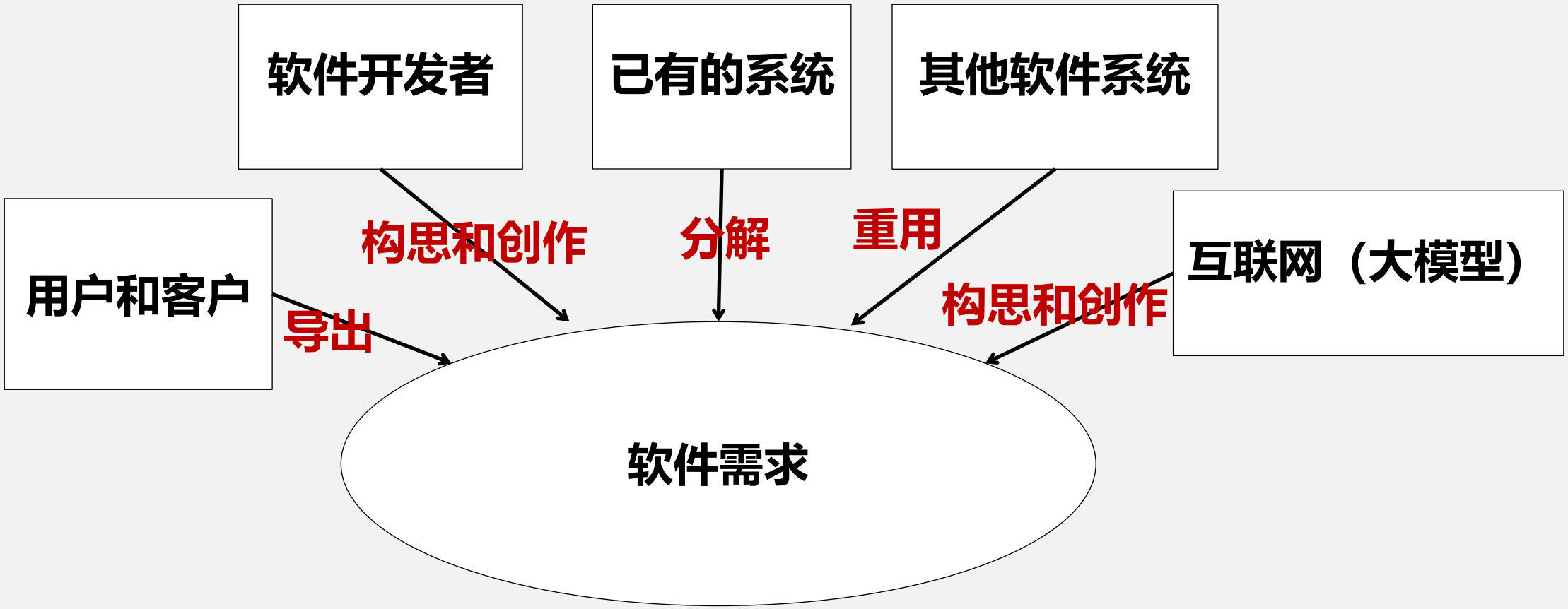
- 需求获取的来源 – 从哪来?
- 需求获取的方法 – 怎么来?
- 需求获取的过程 – 来的步骤?

成立需求分析的联合工作小组

- **小组成员**
  - ✓利益相关者（如用户）
  - ✓软件需求分析人员
  - ✓质量保证人员等
- **目的**
  - ✓共同工作
  - ✓加强联系
  - ✓促进交流
  - ✓增进合作



需求获取的方法



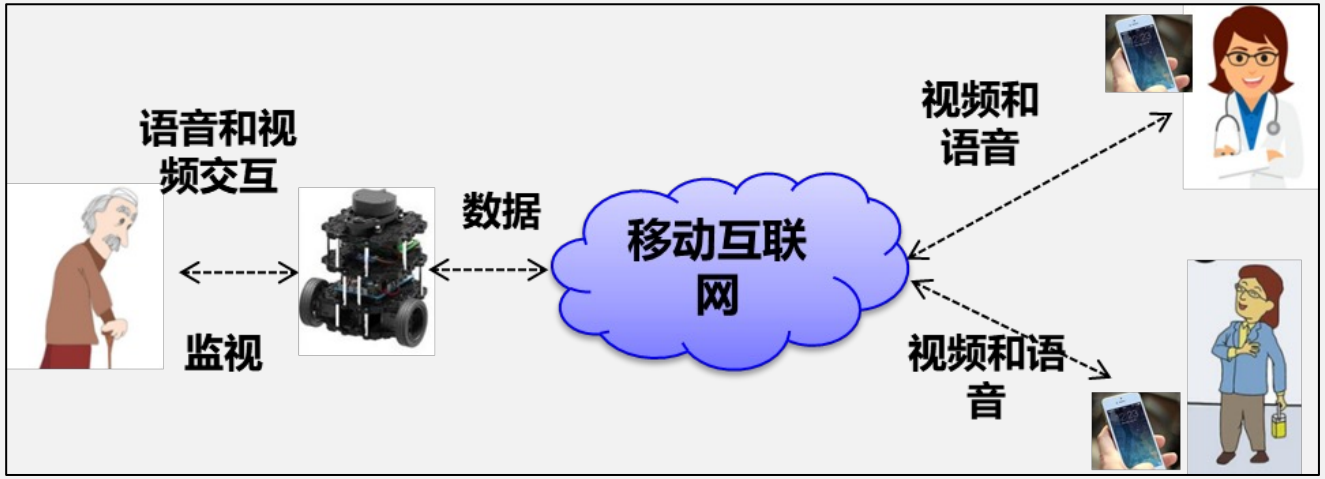
获取软件需求方法

- 访谈和会议
- 调查问卷
- 现场观摩
- 分析业务资料
- 群体化方法
- 大语言模型推荐

访谈和会议

- 召开相关的会议
- 走访相关的人员





参加的人员包括：用户、客户、需求分析人员、质量保证人员等等

调查问卷

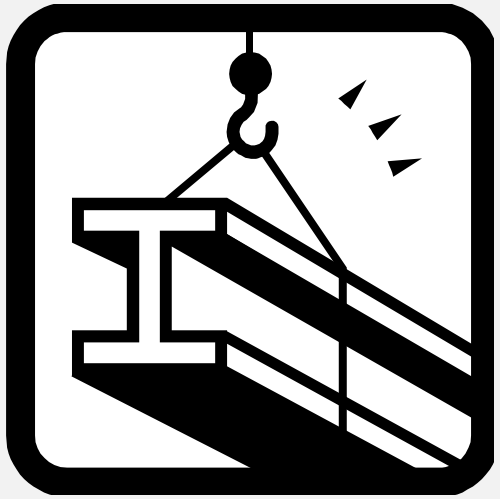
- **设计调查问卷**
  - ✓了解业务流程、实际操作过程
  - ✓用户的期望和要求
- **对谁进行调查**
  - ✓应用领域中扮演不同角色的用户
- **如何利用调查问卷**
  - ✓从调查问卷中了解业务流程
  - ✓从调查问卷中掌握用户需求



现场观摩

- 业务过程、步骤和输出
- 业务工作流程及细节







分析业务资料

- 收集重要的资料
- 阅读和分析与业务相关的文档和资料







群体化方法

- 让互联网上的**海量和开放群体**参与到获取软件需求的工作中来，提出他们对软件需求的想法，并通过组织、汇聚和筛选，从中遴选出有价值的软件需求

- **集思广益**，吸纳更多的人参与到需求创作和构思之中，有助于获得超出开发团队和用户常规想定的软件需求

主观题 10分

设置



- **大模型是否可以帮助需求获取，如何做?**

作答

Part 3 需求获取

- 需求获取的来源 – 从哪来?
- 需求获取的方法 – 怎么来?
- 需求获取的过程 – 哪些步骤?

主观题 10分

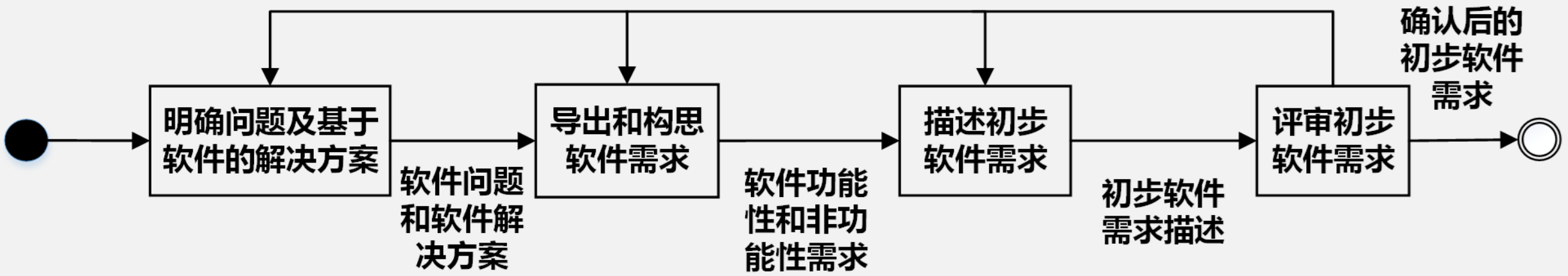
设置



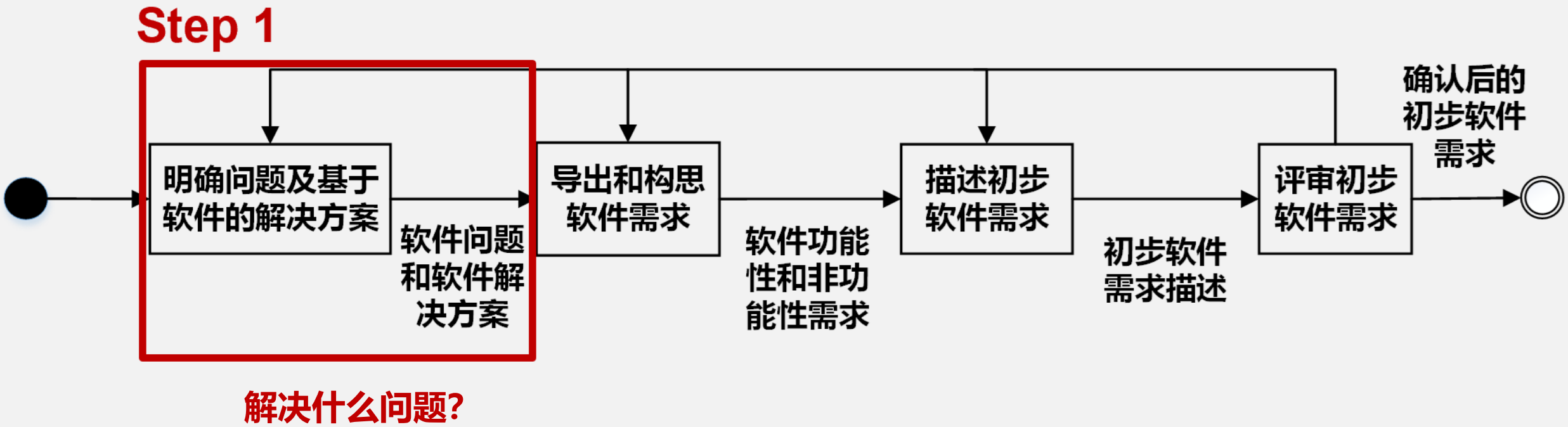
- **回想一下自己之前开发程序的经验，想一想需求获取应该有哪些主要的步骤?**

作答

需求获取的一般过程


明确问题及基于软件的解决方案
软件问题和软件解决方案
导出和构思软件需求
软件功能性和非功能性需求
描述初步软件需求
初步软件需求描述
评审初步软件需求
确认后的初步软件需求

需求获取的一般过程 (1/4)



明确软件要解决的问题（明确系统边界）

❑清晰地界定软件欲解决什么样的问题（系统边界）

- ✓与特定领域及其业务相关联
- ✓或提高业务工作效率，或解决业务瓶颈问题，或提升业务服务水平和质量等等

❑示例

- ✓“12306”软件与铁路旅客服务这一领域及业务相关联，国家铁路集团有限公司投资研制该软件的目的是要改变落后的旅客服务和业务模式，提高旅客服务质量，降低旅客服务成本

课程实践创意示例1：无人值守图书馆

❑问题

- ✓找寻图书困难

❑创意

- ✓线上借阅
- ✓线下机器人获取
- ✓自动找寻



线上基于智能终端的借阅



线下基于机器人的服务





©Copyright Xinjun Mao

课程实践创意示例2：多无人机联合搜寻软件系统

❑问题

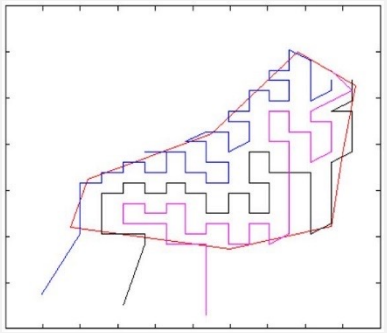
- ✓快速定位被埋人员

❑创意

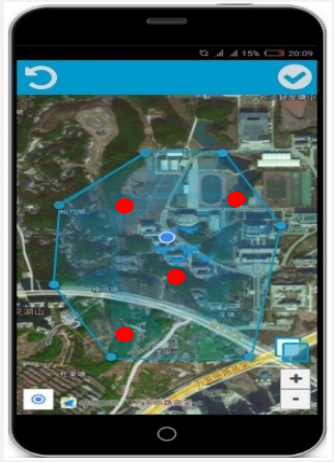
- ✓借助手机定位
- ✓采用无人机编队搜索
- ✓自动生成位置

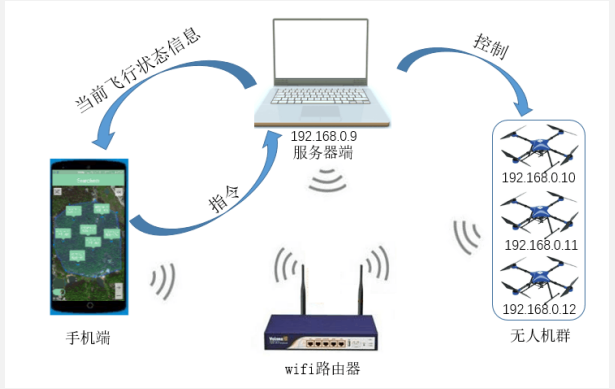




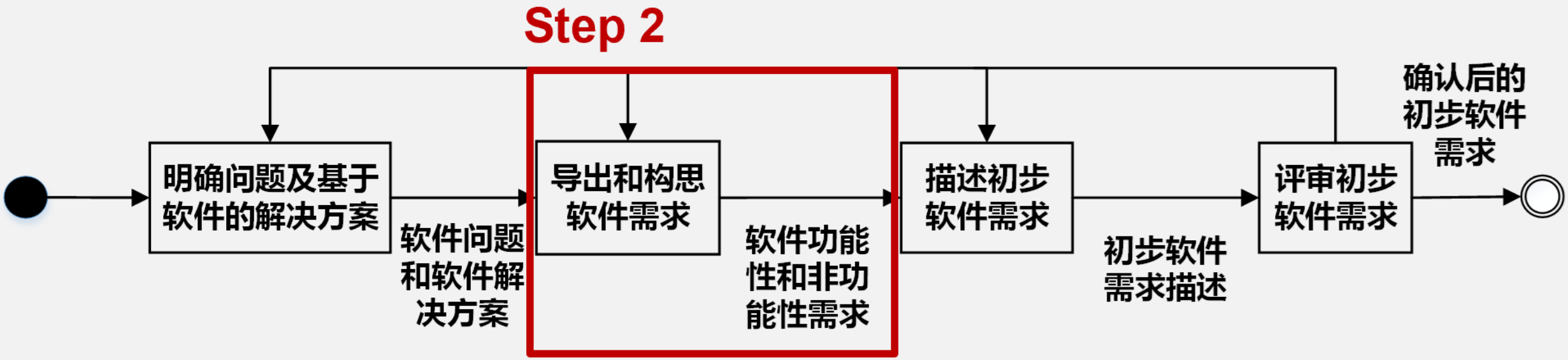








需求获取的一般过程 (2/4)



- 解决谁的问题?
- 他有啥诉求?

识别利益相关方

- 软件需求来自于软件的**利益相关方**，要获取软件需求，首先要搞清楚软件系统有哪些利益相关方
- 软件系统的利益相关方可以表现为特定的**人群和组织**，也可以是一类**系统**
- 不仅**软件用户或客户**可以是软件的利益相关方，软件的**开发者**也可以成为软件的利益相关方。

示例：“空巢老人看护软件”的利益相关方

- **老人（Elder）**
  - ✓与系统通过语音方式进行交互，命令系统为其完成某些事务
- **医生（Doctor）**
  - ✓突发或紧急情景时接受呼叫，与系统进行交互以获取老人状况
- **家属（Family Member）**
  - ✓突发或紧急情景时接受呼叫，与系统进行交互以获取老人的状况
- **管理员（Administrator）**
  - ✓对软件系统进行必要的配置和管理
- **机器人（Robot）**
  - ✓其运动受软件系统的控制

获取软件的功能性需求

- 需求工程师可以通过与利益相关方的交互，听取他们对软件的**期望和要求**，从他们那里导出软件需求

- 采用多种方法来获取软件需求
  - ✓包括与用户或客户的面谈、分析业务资料、观察业务流程、进行问卷调查等。

合理的软件需求

- **并非软件利益相关方提出的每项期望和要求都是软件需求**
  - ✓如果他们提出的**要求与软件及其欲解决的问题无关、没有实际的意义和价值**、或者不存在技术等方面的可行性，那么这些要求不应成为**软件需求**

- **软件需求工程师要有一双“火眼睛”，能够鉴别和发现真正有意义和有价值的重要软件需求**

示例：构思“空巢老人看护软件”功能性需求

❑老人的视角

- ✓**自主跟随老人**，以对老人进行持续跟踪和感知，获取老人信息。
- ✓**提醒服务**，提醒老人按时服药、身体检查等事宜

❑家属和医生

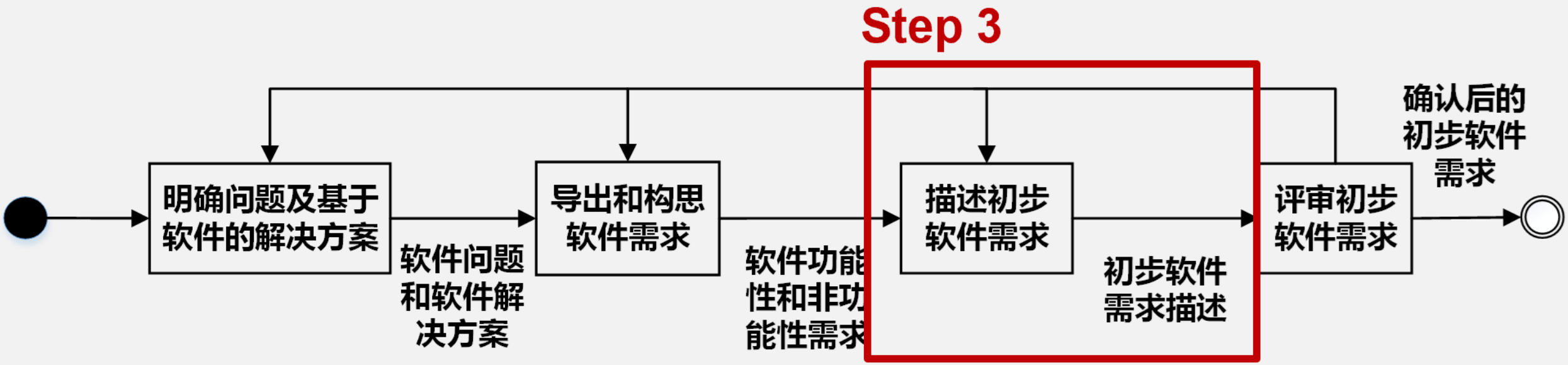
- ✓**监视老人状况**，通过智能手机在远端监视老人在家的状况，获得老人的视频、图像和语音等方面的信息
- ✓**通知异常情况**，将老人的突发异常信息发送给老人家属和医生。
- ✓**远程控制机器人**，通过智能手机在远端来控制机器人移动，以从不同的角度和距离来获取老人的图像、视频和语音信息
- ✓**视频/语音双向交互**，实现老人、医生和家属间的视频和语音交互

获取和构思非功能性需求

- **非功能性需求包括软件质量要求和软件开发的约束性要求**
  - ✓**质量要求**，如软件运行性能、可靠性、易用性、安全性、私密性等属于外部质量要求，软件可扩展性、可维护性、可互操作性、可移植性等内部要求
  - ✓**约束性要求**，包括开发进度要求、成本要求、技术选型等

- **软件的非功能性需求变得越来越重要**
  - ✓在某些情况下它们直接决定了软件是否能用和可用、是否好用和易用、是否高效和可靠运行、是否便于维护和演化等。

需求获取的一般过程 (3/4)



- 表达清楚诉求?

描述初步软件需求 (3/4)

❑初步的软件需求

- ✓之所以称之为是**初步的**，是因为这些软件需求还**很粗略**，只是一个**初步的需求轮廓，不够具体和详尽**，可能有遗漏，会**存在不一致和相冲突**等问题，后续还需要对其开展**进一步的分析和精化**

❑描述需求

- ✓记录下来、描述清楚，形成相关的软件文档，以便于不同人员（如需求工程师、用户、客户等）之间的交流和讨论，及时发现需求理解上存在的偏差，支持后续的需求分析工作

自然语言描述

- **自然语言是最为常用的需求描述手段**
  - ✓描述软件的**功能性需求、质量需求和开发约束需求**等
  - ✓详细刻画需求的具体内容和细节
  - ✓可为各方所理解，便于交流和讨论

自然语言描述的不足

- **不具体**
  - ✓如老人的信息是指什么，上述自然语言描述没有刻画清楚
- **不准确**
  - ✓何为突发异常情况，上述自然语言描述没有定义明确
- **有二义**
  - ✓不同的人看完描述后可能会有不同的理解
- **不直观**
  - ✓很难从中厘清软件系统到底有哪些功能性需求和非功能性需求，这些需求之间存在什么样的关系。

主观题 10分

设置



- **自然语言需求有上述问题？那怎么办？**

作答

结构化自然语言描述

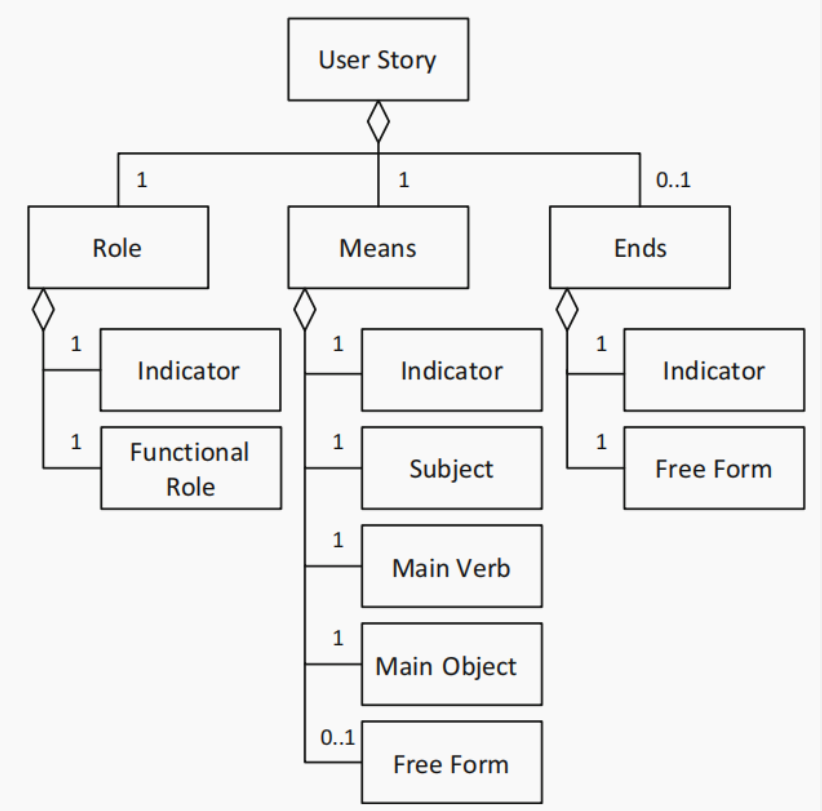
- 用户故事（User Story） - 粗粒度
- 需求描述文法（EARS）- 细粒度
- 交互流程描述（场景描述）- 组织

用户故事 User Story

As a visitor, I want to purchase a ticket so that I can attend the conference

用户故事 = 角色Role, 意图Means, 效果Ends

- **角色:** As a （Indicator 指示词）visitor（角色）
- **意图:** I（Subject 主）want to purchase（Main Verb 谓语） a ticket（Main Object 宾）
- **效果**: so that（Indicator 指示词）I can attend the conference



EARS （简洁需求句法）

- **动机**
  - ✓ 软件需求规范（SRS）定义软件及其功能的文档，SRS通常是用自然语言（NL）编写的
  - ✓ NL需求文档中可能出现的一些问题：**歧义、模糊性、复杂性、重复、需求的不恰当、不可测试性**
- **Easy Approach to Requirements Syntax (EARS，简洁需求句法)**
  - ✓ 引入了结构化的自然语言
  - ✓ EARS于2009年在劳斯莱斯（Rolls-Royce）公司的Alistair Mavin创建
  - ✓ 最初目的是为其产品（包括**航空发动机控制系统**）创建更好的需求文档



Alistair Mavin

EARS特点

❑ 简单易学

- ✓使用五个简单模板编写SRS文档
- ✓英特尔（Intel）的 Requirements Authors Mentoring Program (RAMP, 需求作者指导计划) 用EARS重写需求的实验取得很好的效果

❑ 工业界广泛应用

- ✓Alistair Mavin于2009年创建EARS并应用于航空发动机控制系统
- ✓Alistair Mavin和Eero Uusitalo等人将EARS应用于核能领域的监管需求，即，核电厂安全设计法规指南
- ✓Brendan Hal和Devesh Bhatt等人将EARS用于霍尼韦尔（Honeywell）航天航空，采用约束语言增强的需求方法（CLEAR）方法

EARS 模板

EARS使用五个简单模板编写SRS文档

The lists of EARS templates	
type	syntax
Ubiquitous 普遍存在	The <system/actor name> shall <system/actor response/request>
Event-driven 事件驱动	WHEN <optional preconditions> <trigger> the <system/actor name> shall <system/actor response/request>
Unwanted behaviour 不想要的行为	If<optional preconditions> <trigger>, THEN the <system/actor name> shall <system response/request>
State-driven 状态驱动	WHILE <in a specific state>, the <system/actor name> shall <system response/request>
Optional features 可选的特性	WHERE <feature is included>, the <system/actor name> shall <system response/request>

EARS template

EARS通用需求语法为:

<optional preconditions> <optional trigger> the <system name> shall <system response>

为表示不同类的需求，通用需求语法被五类（普遍存在的、事件驱动的、状态驱动的、不需要的行为和可选的特性），具体描述如下

1.Ubiquitous requirements (普遍存在、无处不在的)

没有任何先决条件或触发器。它不会由在系统检测到的事件或处于定义的系统状态而调用，但它是始终处于活动状态。

syntax : **The <system name> shall <system response>**

例子: “The control system shall prevent engine overspeed”控制系统应防止发动机超速

2.Event-driven requirements (事件驱动)

只有当在系统检测到触发事件发生时，系统才有的响应。关键字“when”用于事件驱动的需求。**syntax : WHEN <optional preconditions> <trigger> the <system name> shall <system response>**

例子: “When continuous ignition is commanded by the aircraft, the control system shall switch on continuous ignition”当飞机发出连续点火指令时，控制系统应接通连续点火开关。

3.State-driven requirements (状态驱动)

当系统处于定义的某个状态时，系统才有的响应。

syntax : WHILE <in a specific state> the <system name> shall <system response>

例子: “While the aircraft is in-flight, the control system shall maintain engine fuel flow above XXlbs/sec”当飞机在飞行中，控制系统应保持发动机燃油流动在XXlbs /sec 以上.

EARS template

4.Unwanted behaviour requirements
(不想要的行为需求)

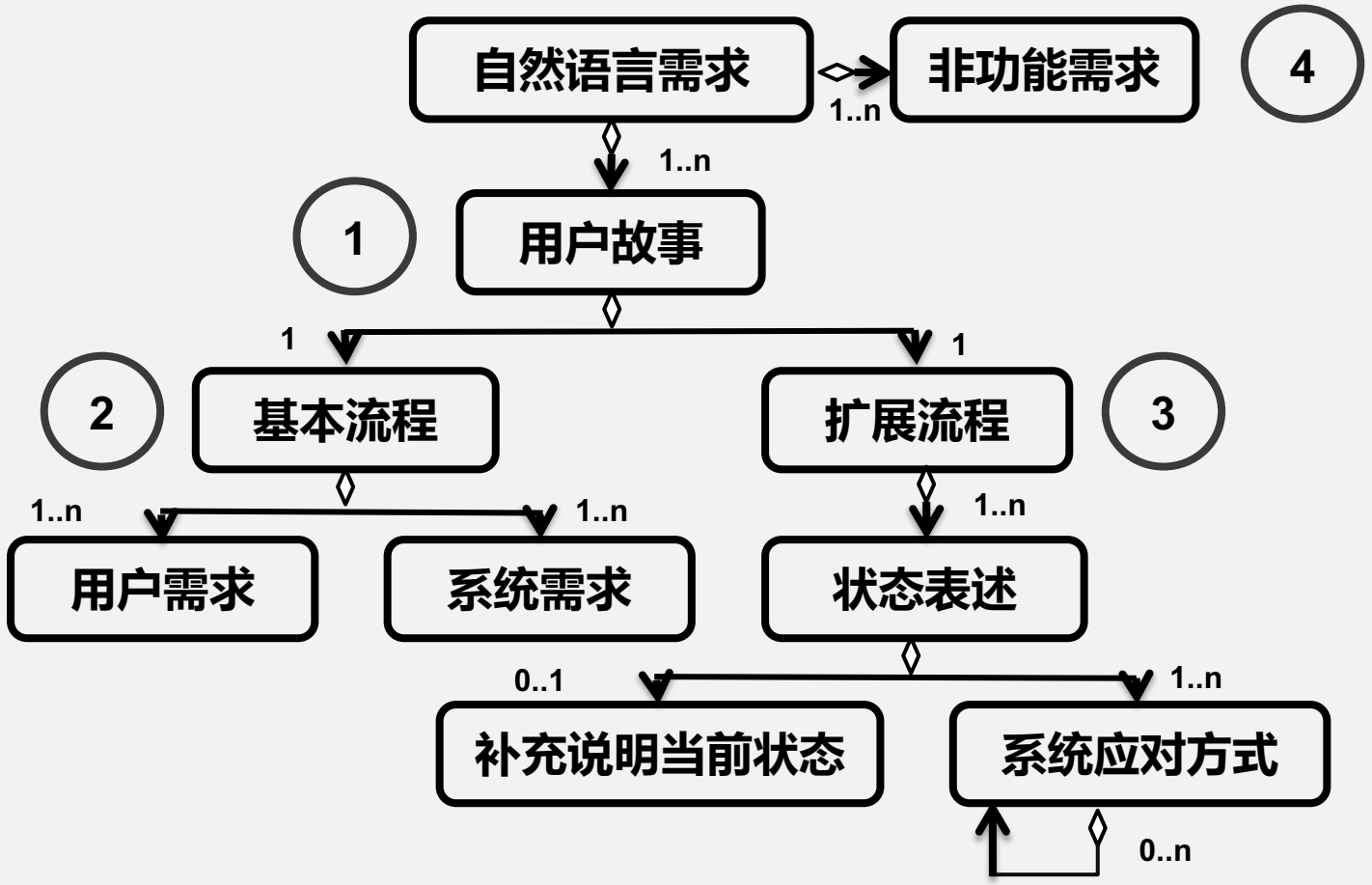
涵盖所有不良情况。这包括故障，干扰，与所需用户行为的偏离以及交互系统的任何意外行为。不想要的行为是早期需求遗漏的主要来源，因此需要进行昂贵的返工。使用关键字If和Then来指定不想要的行为。
syntax：**IF <optional preconditions> <trigger>, THEN the <system name> shall <system response>**
例子：“If an invalid credit card number is entered, then the website shall display “please re-enter credit card details.”
如果输入的信用卡号码无效，则网站将显示“请重新输入信用卡详细信息”。

5.Optional feature requirements
(可选的特性需求)

在系统中，用于指定特性，用关键字Where表示
syntax : WHERE <feature is included> the <system name>shall <system response>
例子：“Where the control system includes an overspeed protection function, the control system shall test the availability of the overspeed protection function prior to aircraft dispatch.”
例子：控制系统包括超速保护功能的，控制系统应在飞机调度前测试超速保护功能的可用性

对于**复杂的需求**，可以将以上五种组合成第6个模式表示复杂的需求。 复杂的需求语法:它由关键字When、While和Where组成。 不想要行为的复杂需求还包括If-Then关键字。
例子: While the aircraft is on ground, when reverse thrust is commanded, the engine control system shall enable reverse thrust.当飞机在地面上，并且当要求执行反推力指令时，发动机控制系统应启用反推力。

交互流程描述（场景描述）


自然语言需求
非功能需求
4
1..n
1..n
1
用户故事
1
1
2
基本流程
扩展流程
3
1..n
1..n
1..n
用户需求
系统需求
状态表述
0..1
1..n
补充说明当前状态
系统应对方式
0..n

用户故事 – 引出用户及主要意图

- **用户故事形式：**

  作为一个<用户角色>，**我想要**<完成活动>，**以便于**<实现价值>

- **用户故事案例：**

  - ✓ ***作为一个**收银员，**我想要**处理销售，**以便于**客户就可以买到商品*
    - ✓ ***前置条件：**收银台以开启。收银员必须经过确认和认证。*
    - ✓ ***后置条件：**存储销售信息。准确计算税金。更新账务和库存信息。记录提成生成票据。记录支付授权的批准。*
  - ✓ ***作为一个**收银员，**我想要**打开收银台，**以便于**客户即可付款*
  - ✓ ***作为一个**收银员，**我想要**关闭收银台*

基本流程

基本流程：内部表述

（用户） N、用户应动作（如果有向系统输入信息需写明）。

（主要用于描述用户的行为，在用户需求的后面可以系统需求（系统响应）也可以没有）

（系统） N、当用户做出行为时，系统应系统的行为（如果有系统向外输出信息需写明）。

（主要用于描述系统的行为，通常情况对应上条用户需求（用户行为）可以多个系统需求对应一个用户需求）

处理销售的基本流程：

（用户） 1、 客户应带着商品到达收银台购买。
（用户） 2、 收银员应开始新的销售。
（系统） 3、当收银员开始新的销售时， CoCoME应将开始新的销售。
（用户） 4、 收银员应输入商品信息。
（系统） 5、当开始新的销售时， CoCoME应记录商品信息输出累加总金额。
（用户） 7、 收银员结束销售。
（系统） 8、当销售结束时， CoCoME应输出合计计算的税款。

基本流程

基本流程：每个用户故事对应0或1个，用于描述主要成功场景。

作为一个收银员，我想要处理销售， 以便于客户就可以买到商品

基本流程 {

需求编号：执行顺序

需求类型

(用户) 1、 客户应带着商品到达收银台购买。

(用户) 2、 收银员应开始新的销售。

(系统) 3、 CoCoME应将开始新的销售。

需要重复执行的需求，需写清楚循环的编号以及结束循环的条件

(用户) 4、 收银员应输入项目标识。

(系统) 5、 当开始新的销售时， CoCoME应记录每个销售线项目，并提供项目描述和运行总额。

(循环 4 ~ 5 直到输入结束)

(系统) 6、 当销售结束时， CoCoME应提交合计计算的税款。

(用户) 7、 收银员应告诉顾客总额并要求付款。

(用户) 8、 客户应支付费用。

(选择 8a, 8b, 8c, 8d)

可选需求

(系统) 9、 当付款结束时， CoCoME应处理。

(系统) 10、当完成销售时， CoCoME应记录完成销售，并将信息发送到外部会计和库存系统，并提交收据。

}

扩展流程 （编号的编写方法）

编号的编写说明

***（小写字母）**：表示**在任意时刻**发生的事件，字母只是序号并没有顺序的意义。

（数字N+小写字母）：**表示在基本流程N发生的事件**

（数字N-数字m+小写字母）：**表示在基本流程N-m发生的事件**

注意：
字母只是序号并没有顺序的意义

（可选）：基本流程中可能出现的其他事件

（可选）*a、 经理在任意时刻，需要进行超控操作
（可选）1a、 客户或经理需要恢复一个中断的销售交易
（可选）2-5a、 顾客告诉收银员其免税状况（e.g., 年长者本国人等）

（异常处理）：基本流程中可能发生的异常状况

（异常处理）*b、 系统在任意时刻失效
（异常处理）4a、 无效商品ID（e.g., 在系统中未发现）
（异常处理）5b、 系统检测到与外部税务计算系统服务的通信故障

（选择）：基本流程中选择的选项

（选择）8a、 现金支付
（选择）8b、 信用卡支付
（选择）8c、 支票支付
（选择）8d、 记账支付

扩展流程

每个用户故事对应0或1个，用于描述主要成功场景

作为一个收银员，我想要处理销售， 以便于客户就可以买到商品

扩展流程 {

第一次层 ⇨ （可选）*a、 经理在任意时刻，需要进行超控操作：

第二次层 ⇨ （系统）1、 系统进入经理授权模式

（系统）2、 经理或收银员执行某一经理模式的操作，取消销售交易等

（用户）3、 系统回复到收银员授权模式

第一次层 ⇨ （可选）1a、 客户或商店经理需要恢复一个中断的销售交易：

（用户）1、 收银员执行恢复操作，并且输入ID以提取对应的销售交易。

（系统）2、 系统显示被恢复的销售交易状态及其小计

第N层 ⇨ （异常处理）2a、 未发现对应的销售交易

与上述架构类似

（系统）1、 系统向收银员提示错误

（用户）2、 收银员可能会开始一个新销售交易，并重新输入所有商品

}

扩展流程 （第二次层）

（可选）1a、 客户或经理需要恢复一个中断的销售交易：

（用户）1、 收银员执行恢复操作，并且输入ID以提取对应的销售交易。

（系统）2、 系统显示被恢复的销售交易状态及其小计。

（选择）8a、 信用卡支付：

（用户）1、 顾客输入信用卡账户信息。

（系统）2、 系统显示其支付信息以备验证。

（用户）3、 收银员确认。

（系统）4、 系统向外部支付授权服务系统发送支付授权请求，并请求批准该支付。

第二层为第一层事件的执行流程，
（用户）为用户向系统的输入操作，
（系统）为系统的输出操作。

后面数字序号代表执行是顺序。



扩展流程 （第二次层）

(可选) 1a、 客户或经理需要恢复一个中断的销售交易：

(用户) 1、 收银员执行恢复操作，并且输入ID以提取对应的销售交易。

(系统) 2、 系统显示被恢复的销售交易状态及其小计。

(异常处理) 4a、 无效商品ID（在系统中未发现）：

(系统) 1、 系统提示错误并拒绝输入该ID。

(用户) 2、 收银员响应该错误。

(选择) 8a、 现金支付：

(用户) 1、 顾客输入信用卡账户信息。

(系统) 2、 系统显示其支付信息以备验证。

(用户) 3、 收银员确认。

(系统) 4、 系统向外部支付授权服务系统发送支付授权请求，并请求批准该支付。

(系统) 5、 系统收到批准支付的应答并提示收银员，同时弹出现金抽展。

(系统) 6、 系统记录信用卡支付信息，其中包括支付批准。

(系统) 7、 系统显示信用卡支付的签名输入机制。

(用户) 8、 收银员请求顾客签署信用卡支付，顾客输入签名。

(系统) 9、 系统记录该信用卡支付。

扩展流程 （第N次层）

(选择) 8b、 *信用卡支付:*

(用户) 1、 *顾客输入信用卡账户信息。*

(系统) 2、 *系统显示其支付信息以备验证。*

(用户) 3、 *收银员确认。*

(可选) 3a、 *收银员取消付款步骤。*

(系统) 1、 *系统回复到（商品输入）模式。*

(系统) 4、 *系统向外部支付授权服务系统发送支付授权请求，并请求批准该支付。*

(异常处理) 4a、 *系统检测到与外部系统协作时的故障。*

(系统) 1、 *系统向收银员提示错误。*

(用户) 2、 *收银员请求顾客更换支付方式。*

(系统) 5、 *系统收到批准支付的应答并提示收银员，同时弹出现金抽展（以便放入签名后的信用卡支付票据）。*

(异常处理) 5a、 *系统收到拒绝支付的应答。*

(系统) 1、 *系统向收银员提示支付被拒绝。*

(用户) 2、 *收银员请求顾客更换支付方式。*

(异常处理) 5b、 *应答超时。*

(系统) 1、 *系统提示收银员应答超时。*

(用户) 2、 *收银员重试，或者请求顾客更换支付方式。*

(系统) 6、 *系统记录信用卡支付信息，其中包括支付批准。*

(系统) 7、 *系统显示信用卡支付的签名输入机制。*

(用户) 8、 *收银员请求顾客签署信用卡支付，顾客输入签名。*

(系统) 9、 *系统记录该信用卡支付。*

*与一层和二层架构一样，需要注意每一层需要有缩进。

非功能需求

非功能需求：

{

1、使用大尺寸平面显示器触摸屏UI，文本信息可见距离为1米·90%的信用卡授权响应时间小于30秒。

2、由于某些原因，我们希望在访问远程服务(如库存系统)失败的情况下具有比较强的恢复功能。

3、支持文本显示的语言国际化。

4、4-8 中能够加入可插拔的业务规则

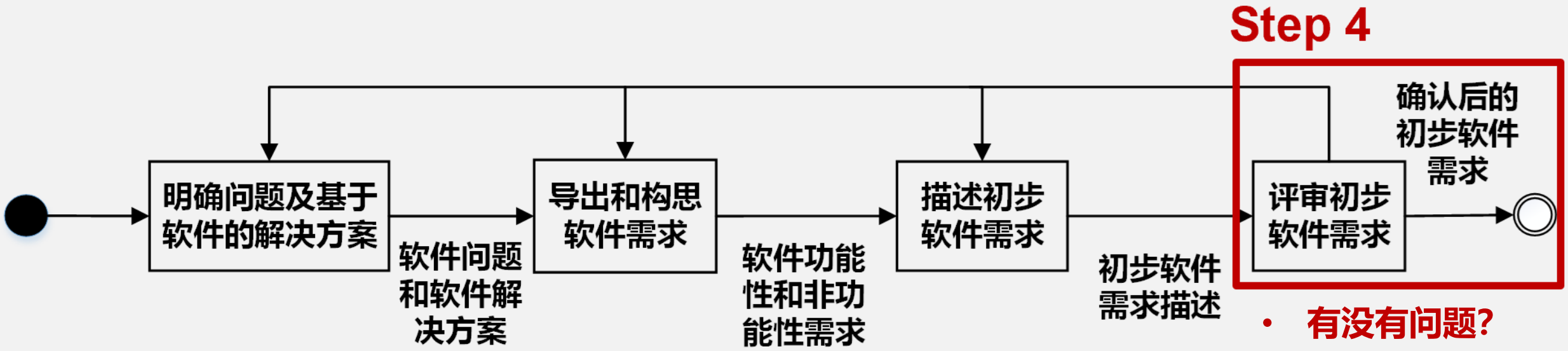
}

没有特殊要求按顺序写即可。

撰写“初步软件需求描述”文档

- 软件背景介绍
- 欲解决的问题
- 软件解决方案
- 利益相关者及主要故事场景
- 软件的功能性需求描述
- 软件的非功能性需求描述
- 可行性及潜在风险

需求获取的一般过程 (4/4)



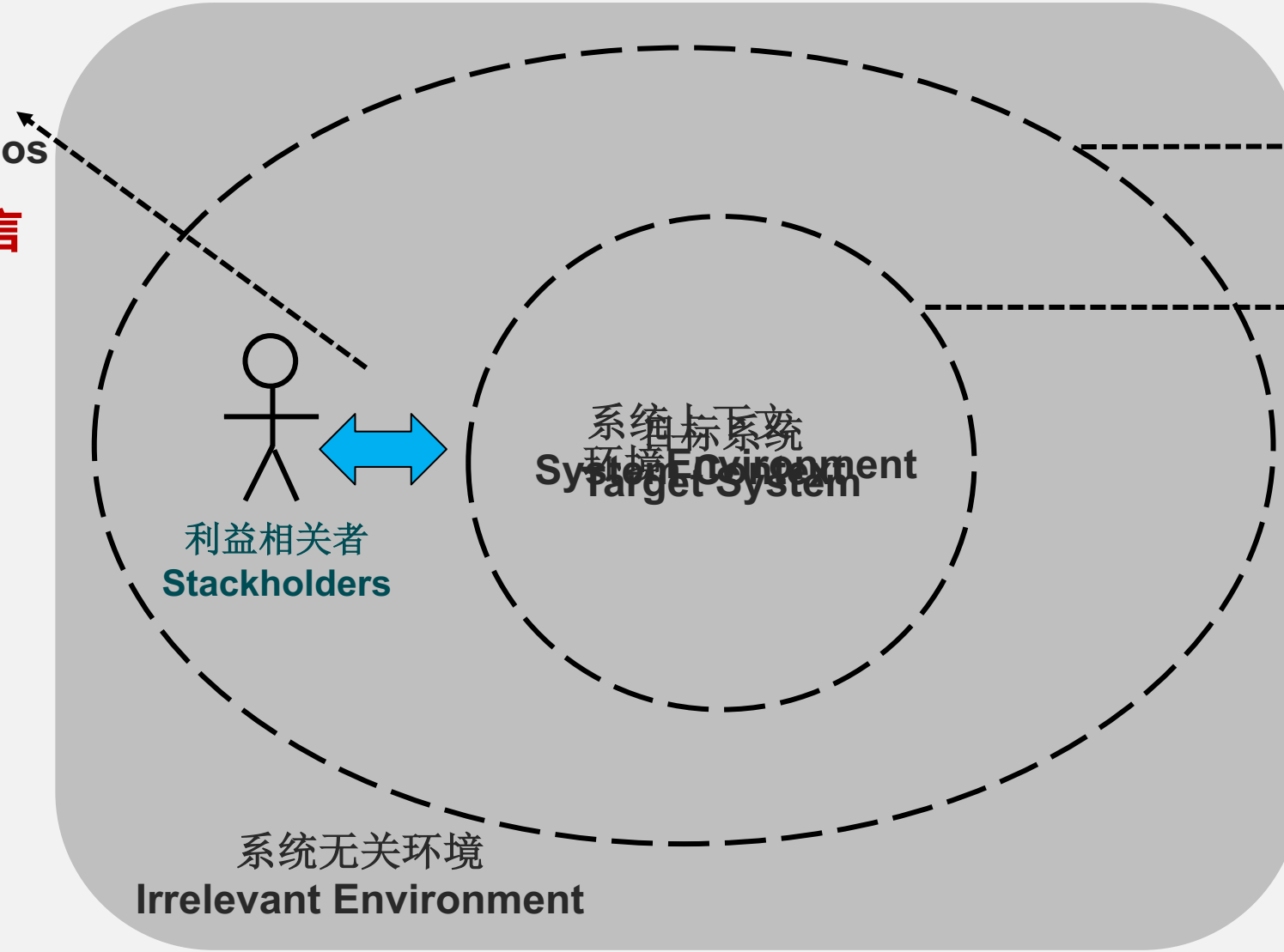
评审初步软件需求 (4/4)

- **中肯性**，软件问题是否反映实际问题，是否有意义和价值
- **合理性**，基于软件的解决方案是否科学和合理。
- **完整性**，软件需求是否覆盖了利益相关方的期望和要求
- **必要性**，每一项软件需求是否有必要
- **溯源性**，每一项软件需求是否都有其来源
- **准确性**，描述是否清晰和准确地反映了软件需求的内涵
- **正确性**，是否正确反映该需求提出者真实想法和关注点
- **一致性**，软件需求文档、用例模型以及软件原型对软件系统需求的表述（包括术语等）是否一致

需求获取（总图）

用户交互场景
Interaction Scenarios

- 结构化自然语言
  - 用户故事
  - EARS
  - 交互流程



- 1.明确边界(问题)
- 2.明确利益相关者
- 3.明确使用场景
  - 用户需求
  - 系统需求
    - 功能
    - 非功能

主观题 10分

设置



- **需求描述到这个程度就够了么?**
- **能不能直接用于开发? 如果不能，还缺少啥**

作答

北京航空航天大學
BEIHANG UNIVERSITY



Part 4

Requirements Specification

需求精化与规约

Part 4 需求精化与规约

- 需求精化与规约的任务
- 需求精化与规划的核心问题
- 面向对象方法的需求模型
- 面向过程方法的需求模型

Part 4 需求精化与规约

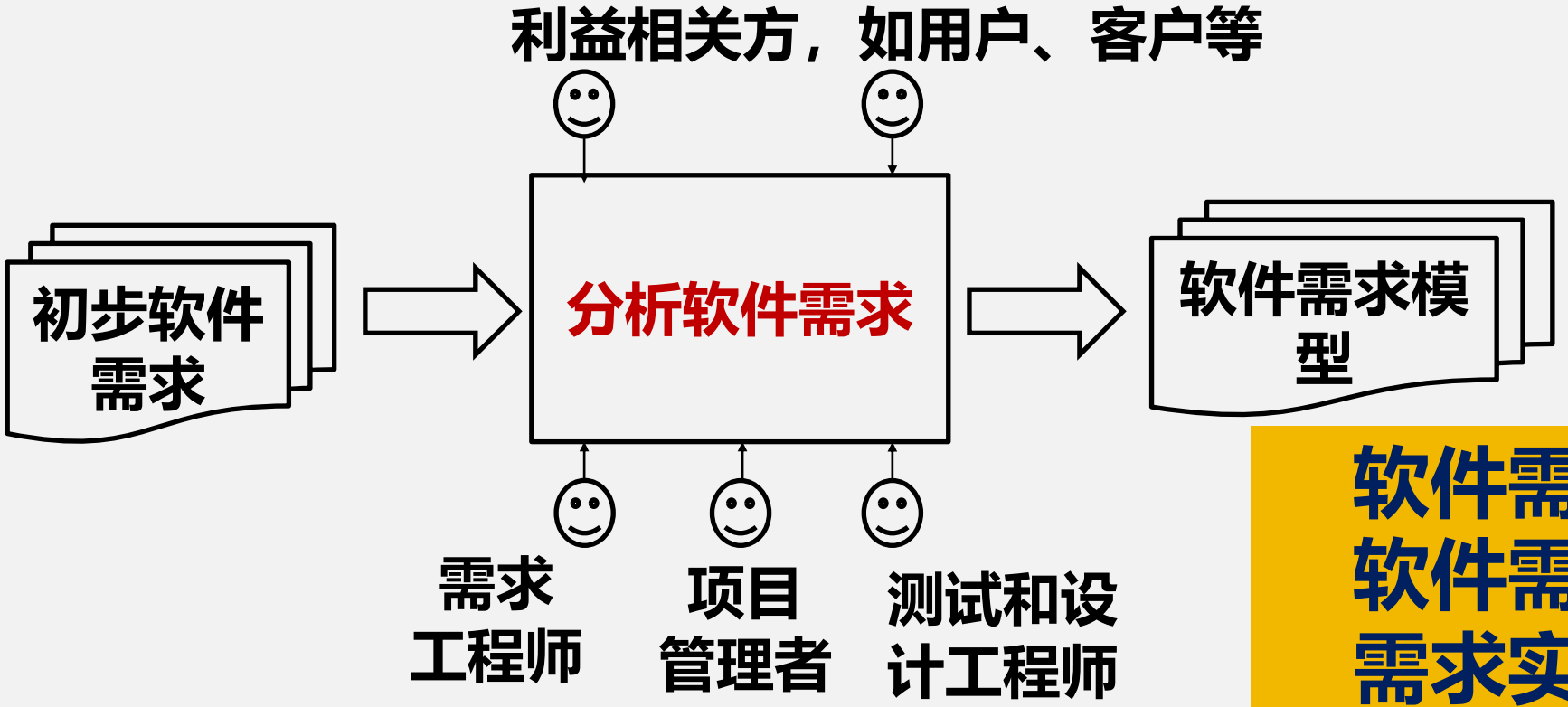
- 需求精化与规约的任务
- 需求精化与规划的核心问题
- 面向对象方法的需求模型
- 面向过程方法的需求模型

为什么要精化初始软件需求

- **初步软件需求存在的问题**
  - ✓**不具体**，没有提供软件需求的细节性内容
  - ✓**不清晰**，对软件需求的描述仍然不够清晰、直观和详实
  - ✓**关系不明朗**，未深入分析不同软件需求项之间的关系
  - ✓**存在潜在缺陷**，潜在的需求缺陷难以被发现
  - ✓**没有区分不同软件需求**，有必要鉴别不同软件需求项的重要性差别，区分不同软件需求的开发优先级

软件需求的精化与规约的任务

- 基于初步软件需求，进一步**分析和细化**软件需求，确定软件需求**优先级**，建立软件需求模型，发现和解决软件需求缺陷，形成**高质量的软件需求模型**



软件需求**重要性**
软件需求**优先级**
需求实现的**次序**

分析和确定软件需求优先级

- 分析软件需求**重要性**
- 分析软件需求**优先级**
- 确定需求实现的**次序**

分析软件需求重要性

- 从用户和客户的视角，软件需求的**重要性**是不一样的

- **核心软件需求**
  - ✓在解决问题方面起到举足轻重的作用，提供了软件系统所特有的功能和服务，体现了软件系统的特色和优势

- **外围软件需求**
  - ✓提供了次要、辅助性的功能和服务

示例：“空巢老人看护软件”的需求重要性

序号	需求名称	需求标识	重要性
1	监视老人	Req-MonitorElder	核心
2	自主跟随老人	Req-FollowElder	核心
3	获取老人信息	Req-GetElderInfo	核心
4	检测异常状况	Req-CheckEmergency	核心
5	通知异常状况	Req-NotifyEmergency	核心
6	控制机器人	Req-ControlRobot	外围
5	视频/语音交互	Req-A&VInteraction	外围
6	提醒服务	Req-AlertService	外围
9	用户登录	Req-UserLogin	外围
10	系统设置	Req-SetSystem	外围

分析软件需求优先级

- **用户对实现这些软件需求的优先级要求也不一样**
  - ✓有些软件需求需要优先实现，尽早交付给用户使用，以发挥其价值；有些软件需求可以滞后实现，晚点交付使用

- **优先级的考虑因素**
  - ✓结合软件项目开发的具体约束，**考虑不同软件需求的重要性**，确定软件需求的实现优先级，确保在整个迭代开发中**有计划、有重点地实现软件需求**
  - ✓按照软件需求的**重要性来确定其优先级**
  - ✓按照用户的**实际需要来确定软件需求的优先级**

示例：确定“空巢老人看护软件”的需求优先级

需求名称	需求标识	重要性	优先级
监视老人	Req-MonitorElder	核心	高
获取老人信息	Req-GetElderInfo	核心	高
检测异常状况	UC-CheckEmergency	核心	高
通知异常状况	UC-NotifyEmergency	核心	高
自主跟随老人	UC-FollowElder	核心	高
视频/语音交互	UC- A&VInteraction	外围	中
控制机器人	UC-ControlRobot	外围	低
提醒服务	UC-AlertService	外围	低
用户登录	UC-UserLogin	外围	低
系统设置	UC-SetSystem	外围	低

确定需求迭代先后次序

- **确保有序地开展需求分析、软件设计和实现工作，使得每次迭代开发有其明确的软件需求集，每次迭代开发结束之后可向用户交付他们所急需的软件功能和服务**

  - ✓结合软件开发的**迭代次数**、每次迭代的**持续时间**、可以投入的**人力资源**等具体情况
  - ✓充分考虑相关软件需求项的**开发工作量**和**技术难度**等因素，确定需求迭代先后次序

示例：“空巢老人看护软件”需求开发安排

需求名称	需求标识	优先级	迭代次序
监视老人	Req-MonitorElder	高	第一次迭代
获取老人信息	Req-GetElderInfo	高	
检测异常状况	Req-CheckEmergency	高	
通知异常状况	Req-NotifyEmergency	高	第二次迭代
自主跟随老人	Req-FollowElder	高	
视频/语音交互	Req-A&VInteraction	中	第三次迭代
控制机器人	Req-ControlRobot	低	
提醒服务	Req-AlertService	低	第四次迭代
用户登录	Req-UserLogin	低	
系统设置	Req-SetSystem	低	

主观题 10分

设置



- **需求描述到这个程度就够了么?**
- **能不能直接用于开发? 如果不能，还缺少啥**

作答

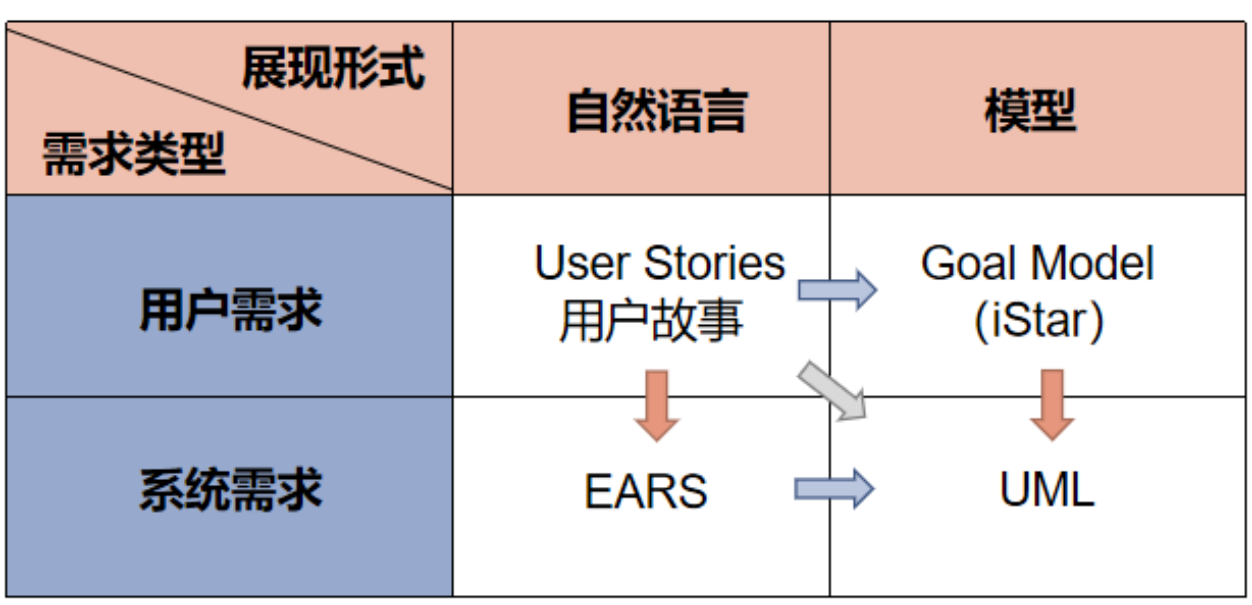
Part 4 需求精化与规约

- 需求精化与规约的任务
- **需求精化与规划的核心问题**
- 面向对象方法的需求模型
- 面向过程方法的需求模型

需求精化与规约的核心问题

两层次跨越：

- **内容：用户初始需求->系统的开发需求**
- **形态：结构化自动语言需求->模型化需求**



初始需求的精化

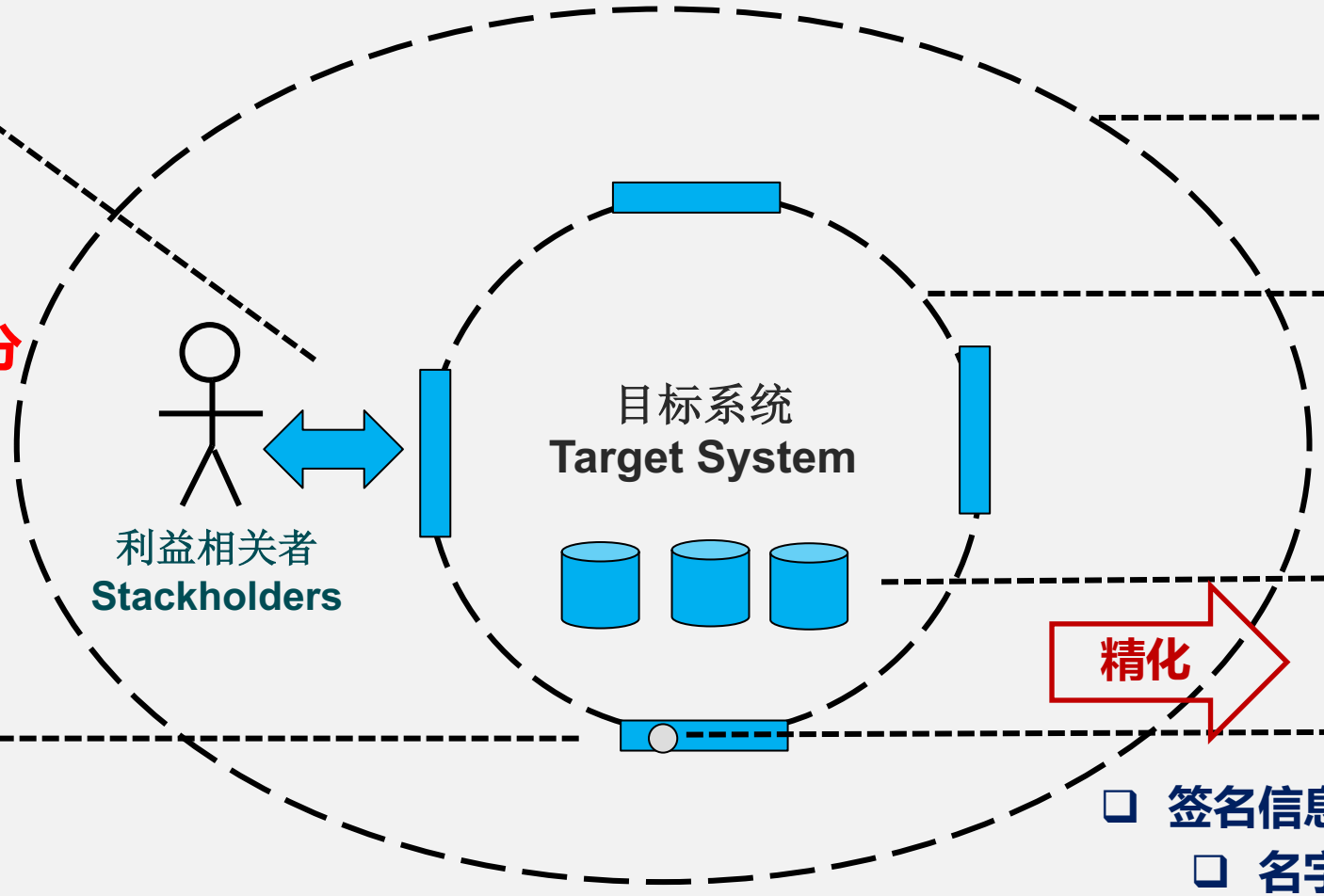
- 明确系统接口
- 明确系统操作
- 明确领域概念

用户交互场景
Interaction Scenarios

- 用户故事
- 交互场景
  - 用户/系统部分

精化

系统接口
System Interfaces



上下文边界
Context Boundary

系统边界
System Boundary

- 概念实体
- 实体属性和关系

领域概念
Domain Concepts

精化

系统操作
System Operations

- 签名信息
  - 名字+输入+输出
- 状态约束 / 非功能约束
  - 前置条件 + 后置条件 + 不变式

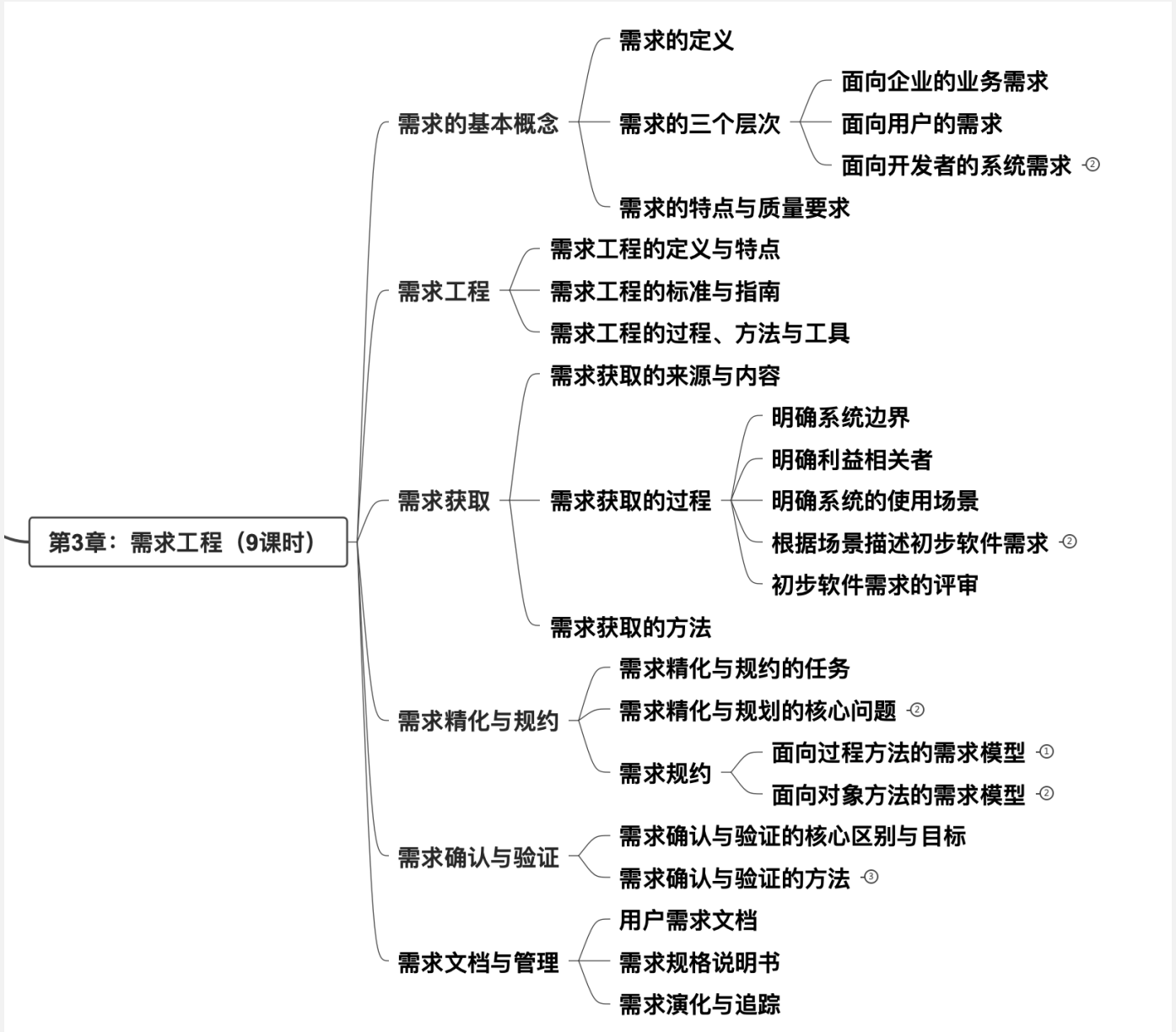
北京航空航天大學
BEIHANG UNIVERSITY

Thank you

北京航空航天大学



（回顾）课时安排：需求工程


第3章：需求工程（9课时）
需求的基本概念
需求的定义
需求的三个层次
面向企业的业务需求
面向用户的需求
面向开发者的系统需求 ②
需求的特点与质量要求
需求工程
需求工程的定义与特点
需求工程的标准与指南
需求工程的过程、方法与工具
需求获取
需求获取的来源与内容
需求获取的过程
明确系统边界
明确利益相关者
明确系统的使用场景
根据场景描述初步软件需求 ②
初步软件需求的评审
需求获取的方法
需求精化与规约
需求精化与规约的任务
需求精化与规划的核心问题 ②
需求规约
面向过程方法的需求模型 ①
面向对象方法的需求模型 ②
需求确认与验证
需求确认与验证的核心区别与目标
需求确认与验证的方法 ③
需求文档与管理
用户需求文档
需求规格说明书
需求演化与追踪

主观题 10分

设置



- **需求过程的三层次与三阶段是什么?**

作答

（回顾）需求工程过程模型

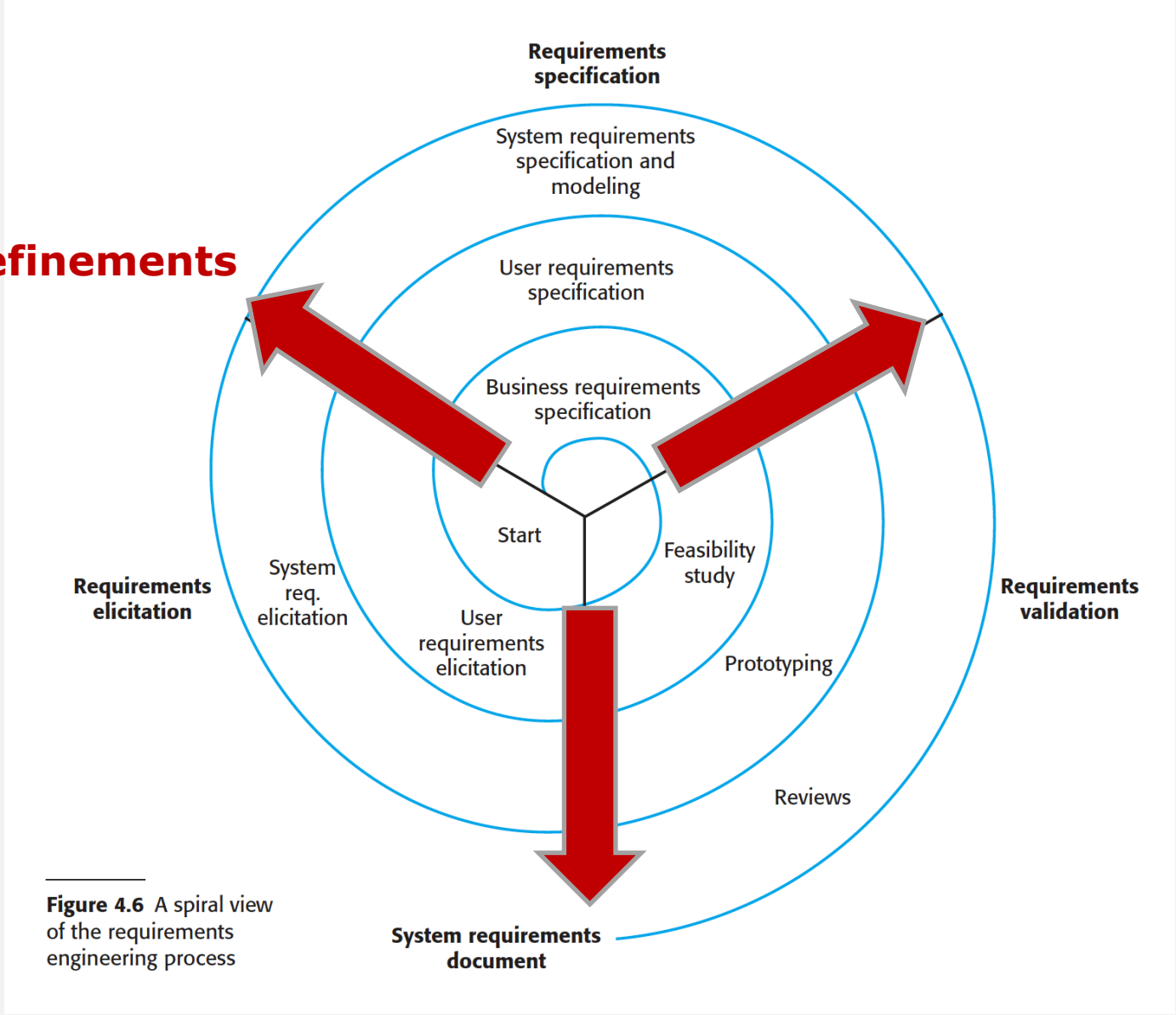


Figure 4.6 A spiral view of the requirements engineering process

需求过程：螺旋模型

- 三层次：
  - 业务、用户、系统需求
- 三阶段：
  - 需求获取 (Elicitation)
  - 需求精化与规约 (Refinement&Specification)
  - 需求确认与验证 (Validation&Verfication)

主观题 10分

设置



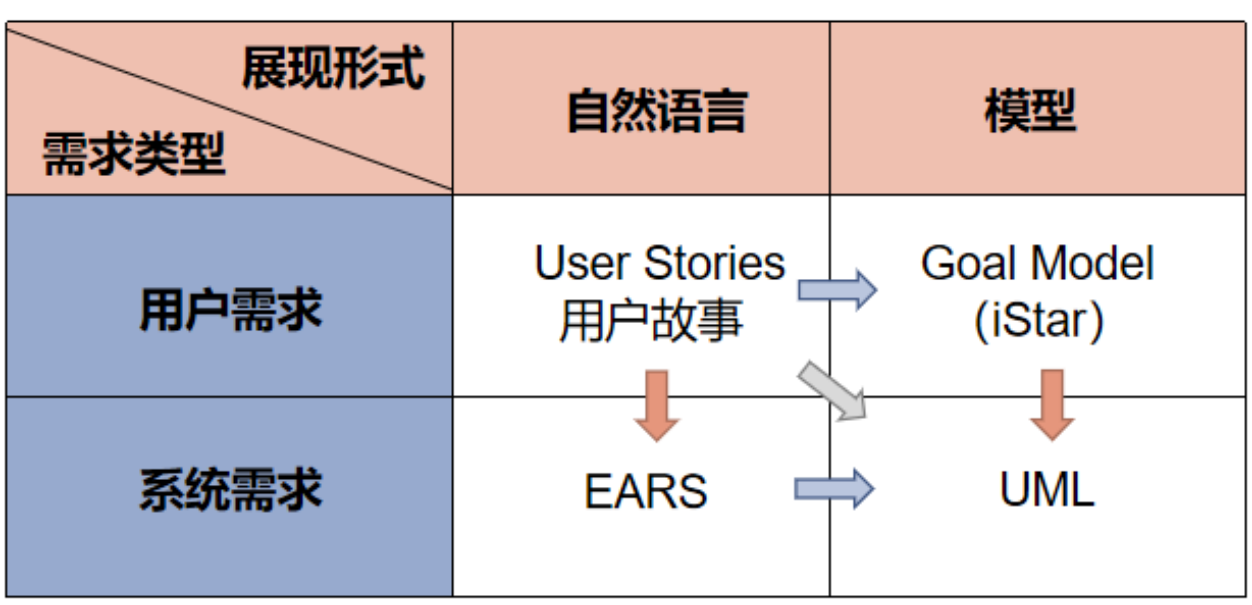
- **需求精化与规约的核心问题是什么?**

作答

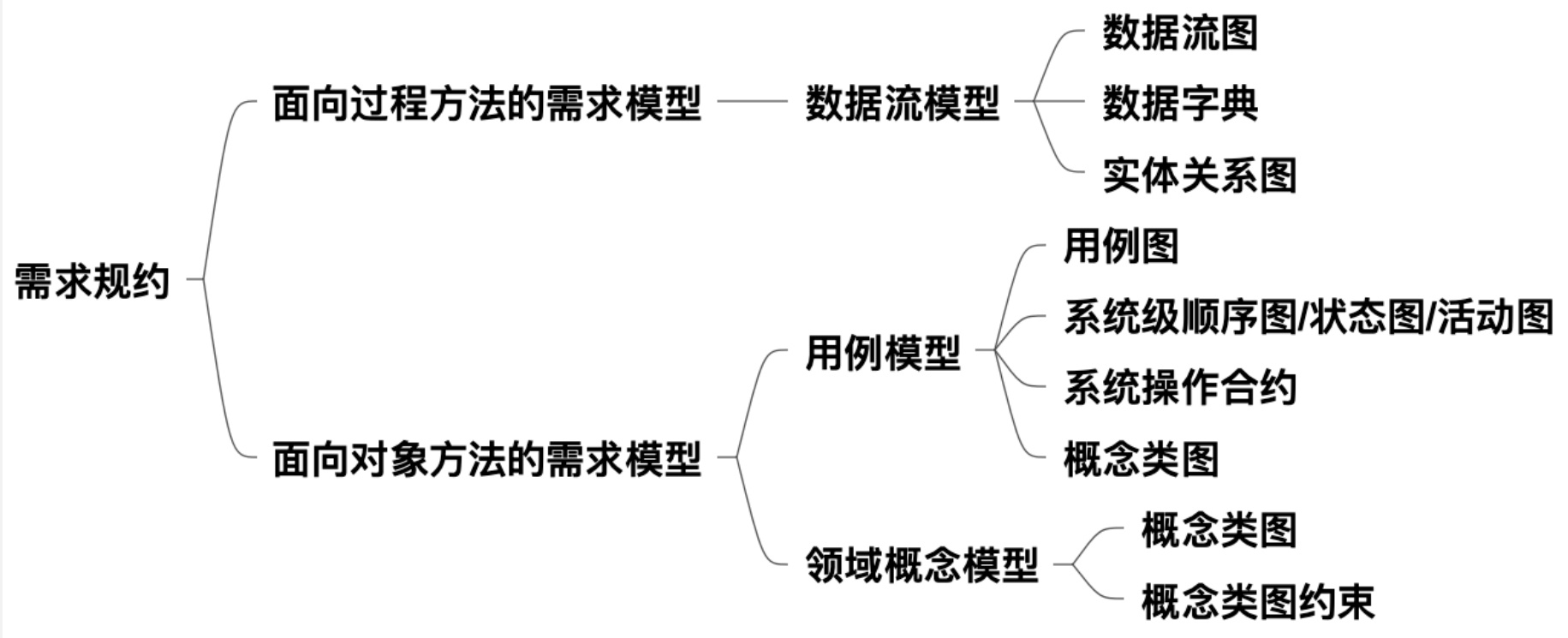
（回顾）需求精化与规约的核心问题

两层次跨越：

- **内容：用户初始需求->系统的开发需求**
- **形态：结构化自动语言需求->模型化需求**



Part 4 需求精化与规约



Part 4 需求精化与规约

- 需求精化与规约的任务
- 需求精化与规划的核心问题
- **面向对象方法的需求模型**
- 面向过程方法的需求模型

主观题 10分

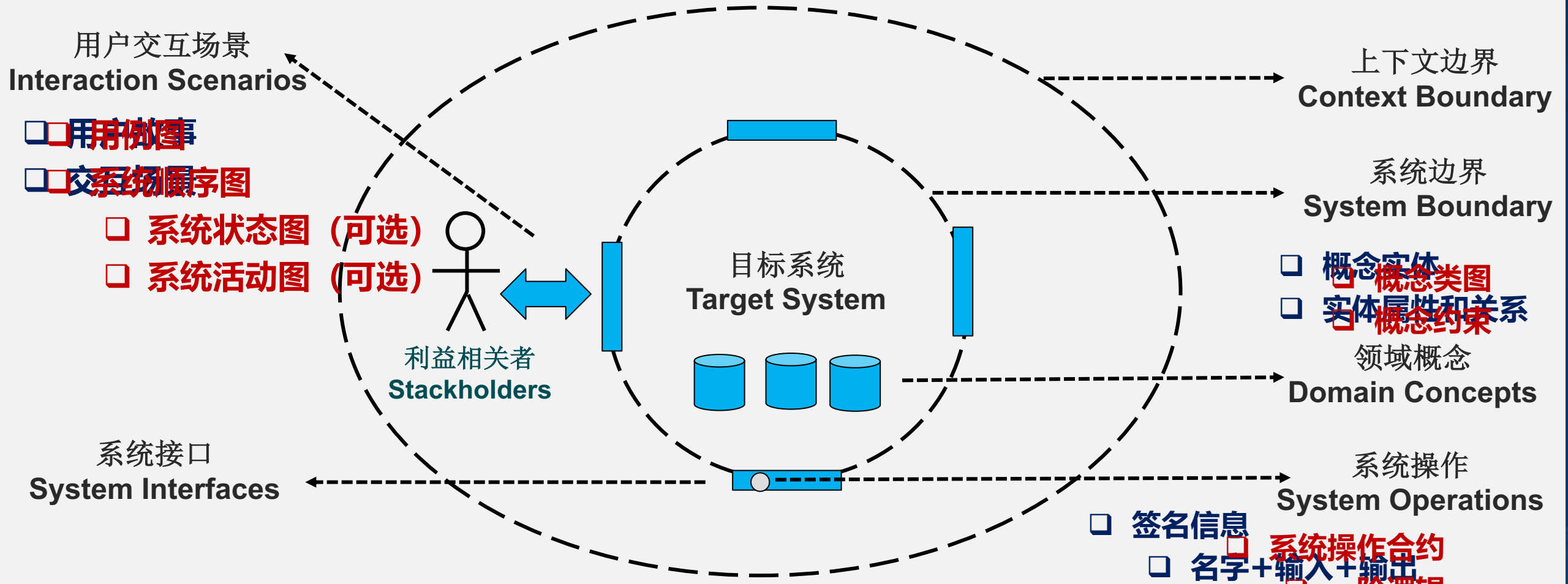
设置



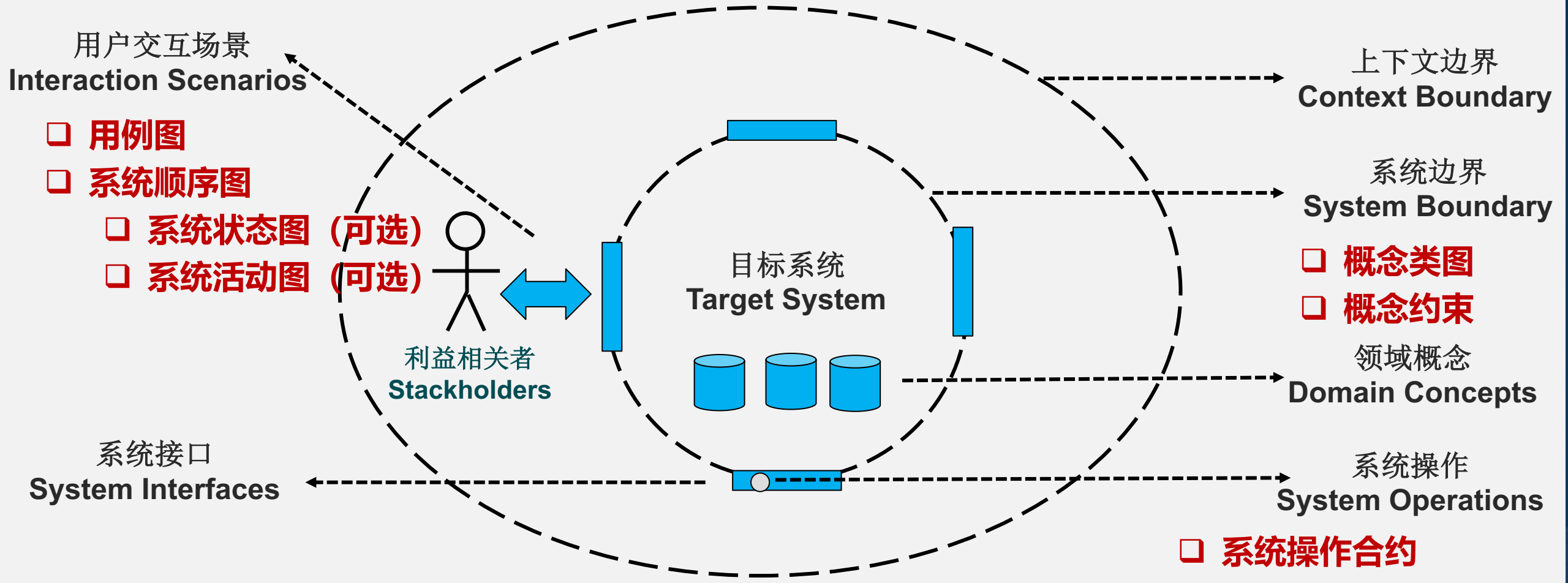
- **OO需求模型分几个部分?**
- **每个部分都包含什么内容?**

作答

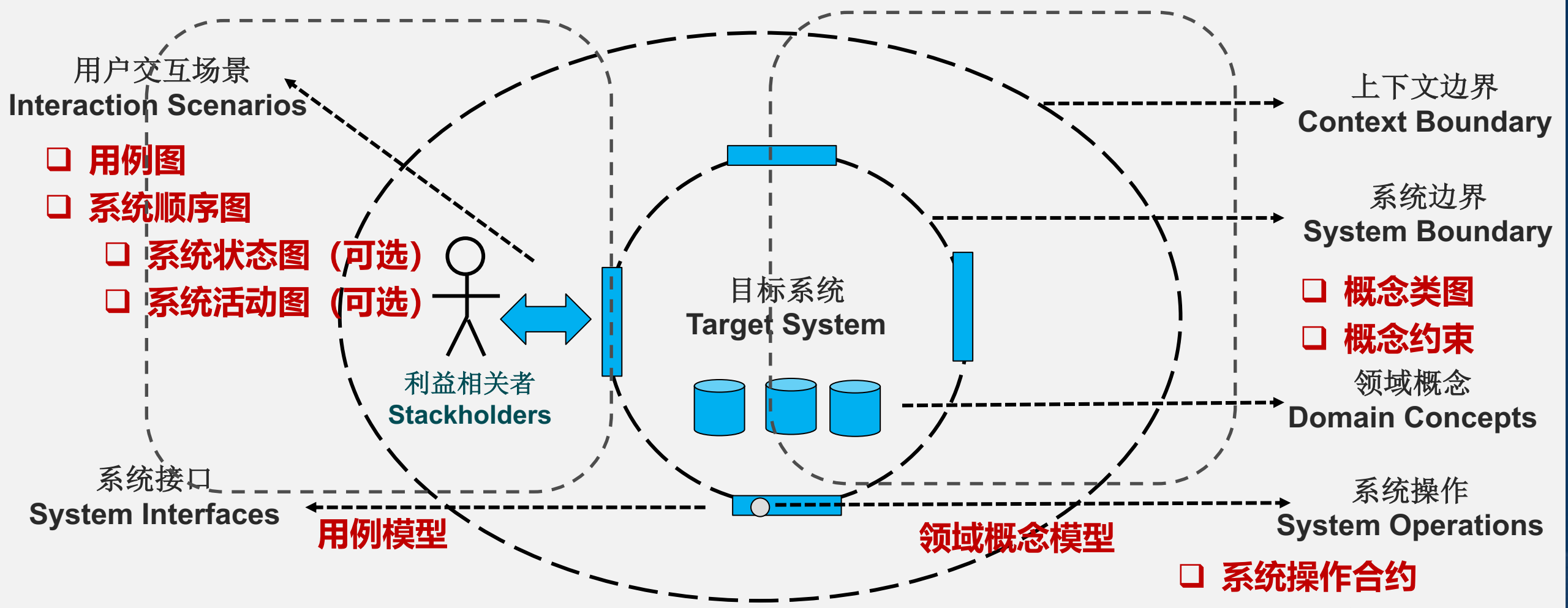
面向对象系统需求的建模



面向对象系统需求的建模



面向对象系统需求的建模



面向对象的需求模型

- 用例图
- 系统顺序图
  - 系统状态图（可选）
  - 系统活动图（可选）
- 系统操作合约
  - 一阶逻辑
  - 对象约束语言

用例模型
（行为）

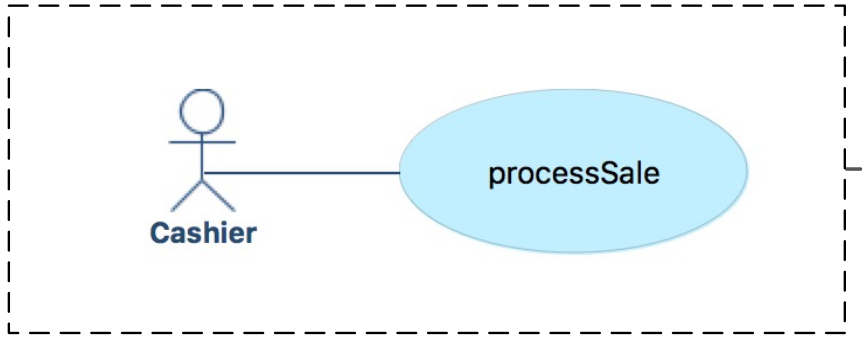
- 概念类图
- 概念复杂约束

领域概念模型
（结构）

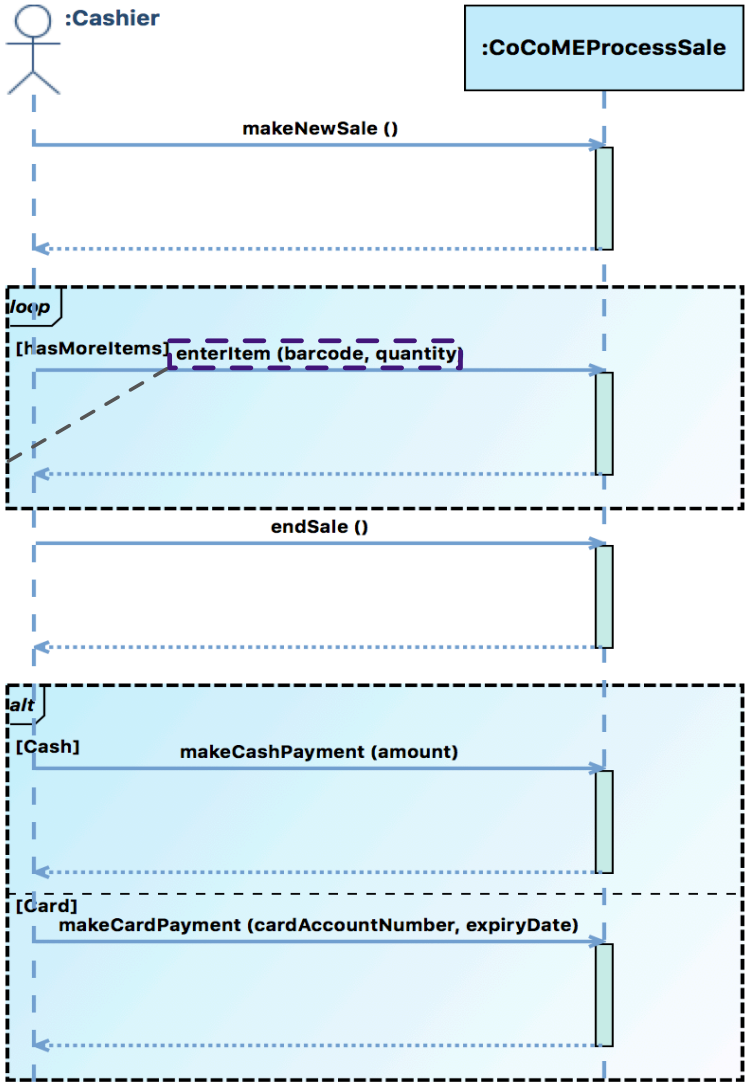


OO需求模型

超市购物系统



1. 用例图



2. 系统顺序图

```
Contract CoCoMEProcessSale::enterItem(barcode : Integer, quantity : Integer) : Boolean {

    /* Definition: find specific Item instance by barcode */
    definition:
        item:Item = Item.allInstance()->any(i:Item | i.Barcode = barcode)

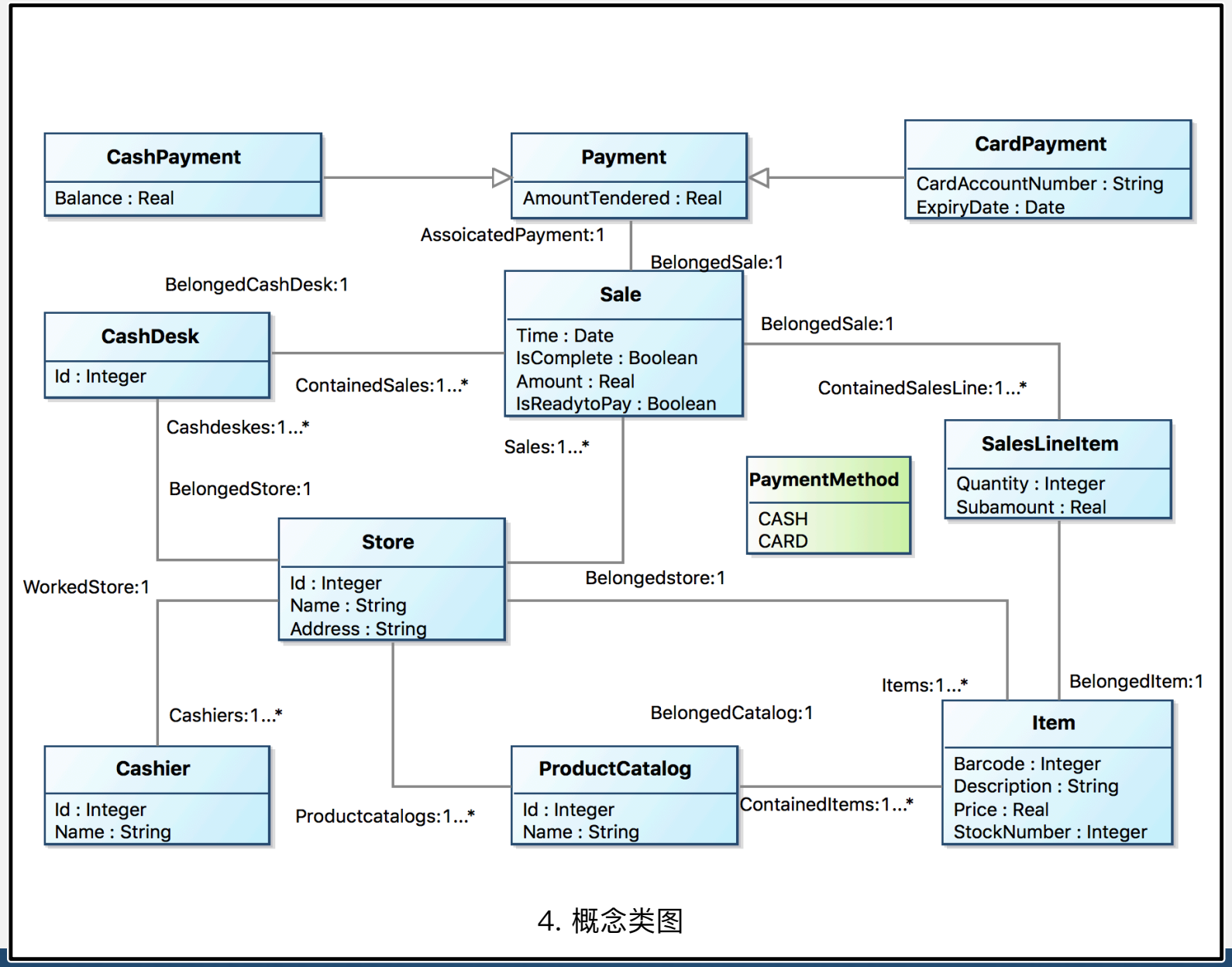
    /* Precondition: there is a sale underway */
    precondition:
        currentSale.oclIsUndefined() = false and
        currentSale.IsComplete = false

     * A salesLineItem instance sli was created (instance creation).
    postcondition:
        let sli:SalesLineItem in
        sli.oclIsNew() and
        self.currentSaleLine = sli and
        sli.BelongedSale = currentSale and
        currentSale.ContainedSalesLine->includes(sli) and
        sli.Quantity = quantity and
        sli.BelongedItem = item and
        item.StockNumber = item.StockNumber@pre - quantity and
        sli.Subamount = item.Price * quantity and
        SalesLineItem.allInstance()->includes(sli) and
        result = true

}
```

3. 操作合约

超市购物系统



4. 概念类图

UML: Unified Modeling Language

- **Unified(统一)**
  - ✓提取不同方法中最好建模技术，如OMT(James Rumbaugh)，Booch method(Grady Booch)和OOSE(Ivar Jacobson)
  - ✓采用**统一、标准化**的表示方式
- **Modeling(建模)**
  - ✓对现实系统和软件系统进行**可视化建模**
  - ✓建立系统模型
- **Language(语言)**
  - ✓**图形化语言**：语法、语义和语用
  - ✓包括规则，约束 扩展机制



www.omg.org

OMG组织致力于对象技术标准化工作

UML本质是什么？

✓ UML是一种标准的图形化建模语言，它是面向对象分析与设计的一种标准表示。它不是一种可视化的程序设计语言，而是一种可视化的建模语言；它不是工具或知识库的规格说明，而是一种建模语言规格说明，是一种表示的标准；它不是过程，也不是方法，但允许任何一种过程和方法使用它。

✓ UML捕获系统静态结构和动态行为的信息。其中静态结构定义了系统中对象的属性和方法，以及这些对象间的关系。用来动态行为则定义了对象在不同时间、状态下的变化以及对象间的相互通信。

✓ 此外，UML可以将模型组织为包的结构组件，使得大型系统可分解成易于处理的单元。

UML与软件开发方法、过程的关系

✓ UML是独立于过程的，它适用于各种软件开发方法、软件生命周期的各个阶段、各种应用领域以及各种开发工具。UML规范没有定义一种标准的开发过程，但它更适用于迭代式的开发过程。它是为支持现今大部分面向对象的开发过程而设计的。

✓ UML不是一种程序设计语言，但用UML描述的模型可以和各种编程语言相联系。我们可以使用代码生成器将UML模型转换为多种程序设计语言代码，或者使用逆向工程将程序代码转换成UML。把正向代码生成和逆向工程这两种方式结合起来就可以产生双向工程，使得既可以在图形视图下工作，也可以在文本视图下工作。

面向对象建模语言

❑概念

- ✓基于**面向对象的概念和抽象**，提供图形化的**图符**，用来**表示**软件系统的一种语言

❑目的

- ✓**用于建模**：绘制和描述软件系统模型(**需求模型和设计模型**)
- ✓**支持交流**：便于开发人员之间的交流、沟通和讨论

❑组成

- ✓**语法**：图形化的**符号**表示
- ✓**语义**：形式或半形式的**含义**
- ✓**语用**：如何使用语言来**建立模型、提供策略和原则**

面向对象需求分析方法的优势和特色

❑接近自然

- ✓面向对象提供了一系列更加**贴近现实世界**（而非计算机世界）的概念和抽象来描述软件需求

❑提供统一的概念和抽象方式

- ✓提供统一的概念和抽象，方便用户和软件开发人员**用同一个概念模型来理解问题、分析问题和解决问题**
- ✓**无需采用模型转换的方式**，而是采用不断精化模型的方法来开展软件开发，从而**简化软件开发的复杂度**

UML应用范围

✓ UML以面向对象的方式来描述系统。最广泛的应用是对软件系统进行建模，但它同样适用于许多非软件系统领域的系统。

✓ 理论上来说，任何具有静态结构和动态行为的系统都可以使用UML进行建模。

✓ 当UML应用于大多数软件系统的开发过程时，它从需求分析阶段到系统完成后的测试阶段都能起到重要作用。

UML

HOME SITE MAP LEGAL

OMG Standards Development Organization

ABOUT RESOURCE HUB OMG SPECIFICATIONS PROGRAMS MEMBERSHIP MEMBERS AREA

ABOUT THE UNIFIED MODELING LANGUAGE SPECIFICATION VERSION 2.5.1

2.5.1 • UML • SPECIFICATIONS

UML® Unified Modeling Language

A specification defining a graphical language for visualizing, specifying, constructing, and documenting the artifacts of distributed object systems.

Specification

Title:	Unified Modeling Language
Acronym:	UML®
Version:	2.5.1
Document Status:	formal ⓘ
Publication Date:	December 2017
Categories:	Modeling Software Engineering Platform
IPR Mode ⓘ	RF-Limited ⓘ

TABLE OF CONTENTS

- About the Specification
- Issues associated with this specification
- Specification Documents
  - Normative Documents
  - Normative Machine Readable Documents
  - Informative Documents
- History
  - Formal Versions
- Links

An OMG® Unified Modeling Language® Publication

OBJECT MANAGEMENT GROUP

UNIFIED MODELING LANGUAGE

OMG® Unified Modeling Language® (OMG UML®)

Version 2.5.1

OMG Document Number:	**formal/2017-12-05**
Date:	**December 2017**
Normative URL:	**https/www.omg.org/spec/UML/**
Machine Readable:	
Normative:	**https://www.omg.org/spec/UML/20161101/PrimitiveTypes.xmi** **https://www.omg.org/spec/UML/20161101/UML.xmi** **https://www.omg.org/spec/UML/20161101/StandardProfile.xmi** **https://www.omg.org/spec/UML/20161101/UMLDI.xmi**

UML图

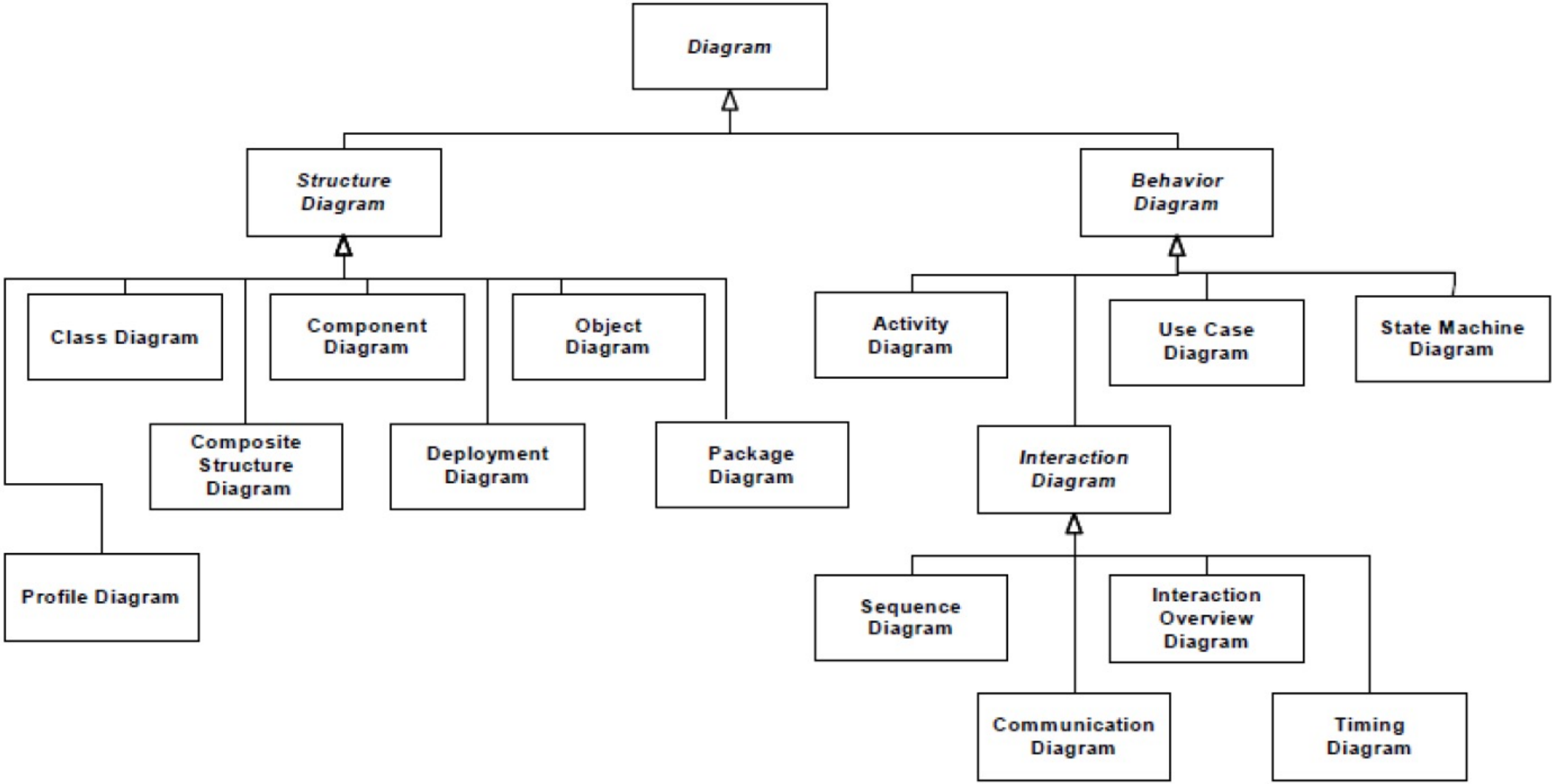


Figure A.5 The taxonomy of structure and behavior diagrams

UML的视点及图

视点	图（diagram）	说明
结构	包图（package diagram）	从包层面描述系统的静态结构
	类图（class diagram）	**从类层面描述系统的静态结构**
	对象图（object diagram）	从对象层面描述系统的静态结构
	构件图(component diagram)	描述系统中构件及其依赖关系
行为	**状态图(statechart diagram)**	**描述状态的变迁**
	活动图(activity diagram)	**描述系统活动的实施**
	通信图(communication diagram)	描述对象间的消息传递与协作
	顺序图(sequence diagram)	**描述对象间的消息传递与协作**
部署	部署图（deployment diagram）	描述系统中工件在物理运行环境中的部署情况
用例	**用例图（use case diagram）**	**从外部用户角度描述系统功能**

面向对象的需求模型

- 用例图
- 系统顺序图
  - 系统状态图（可选）
  - 系统活动图（可选）
- 系统操作合约
  - 一阶逻辑
  - 对象约束语言

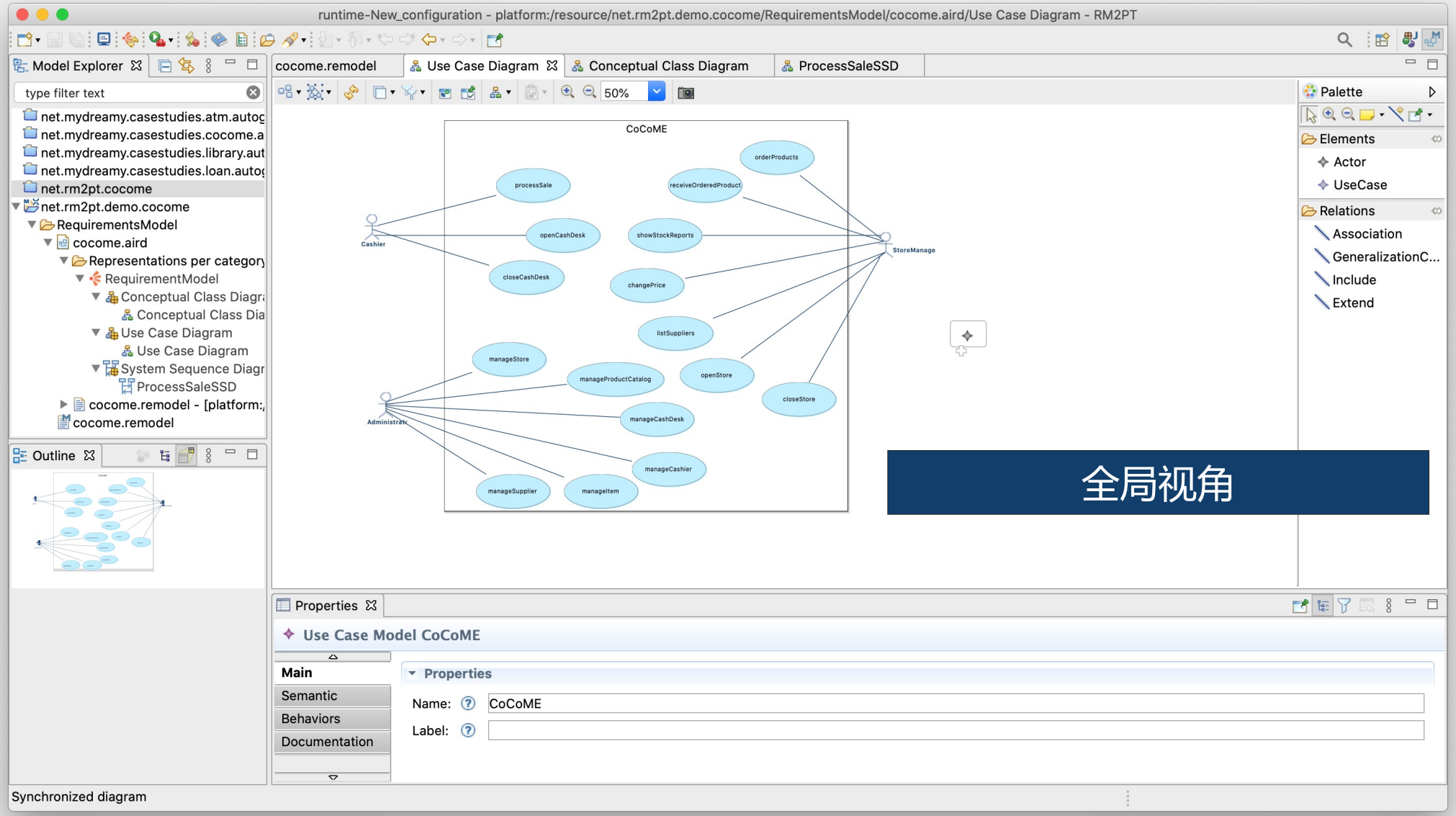
用例模型
（行为）

- 概念类图
- 概念复杂约束

领域概念模型
（结构）

需求
模型

RM2PT 建模展示 – 用例图



主观题 10分

设置



- **参考用户故事，猜想一下用例图里面都有什么？**

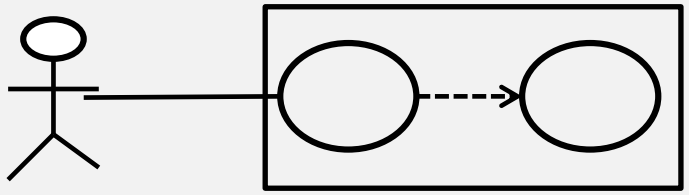
作答

模型描述描述 -用例图

❑用途

- ✓描述软件系统的**边界**以及软件**外部参与者**所观察到的**系统功能**
- ✓ “观察到”是指外部使用者与系统存在**交互**，即信息输入和输出

❑图的构成



- ✓**参与者**（Actor）：系统的外部使用者
- ✓**用例**（Use Case）：观察到的功能和交互集合
- ✓**边**：执行者与用例间、用例之间、执行者之间的关系

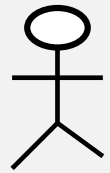
参与者(Actor)

❑何为参与者

- ✓系统**之外的实体**，他们使用软件系统功能、与软件系统交换信息
- ✓可以是**一类用户**，也可以是其他**软件系统或物理设备**

❑参与者是UML中的类

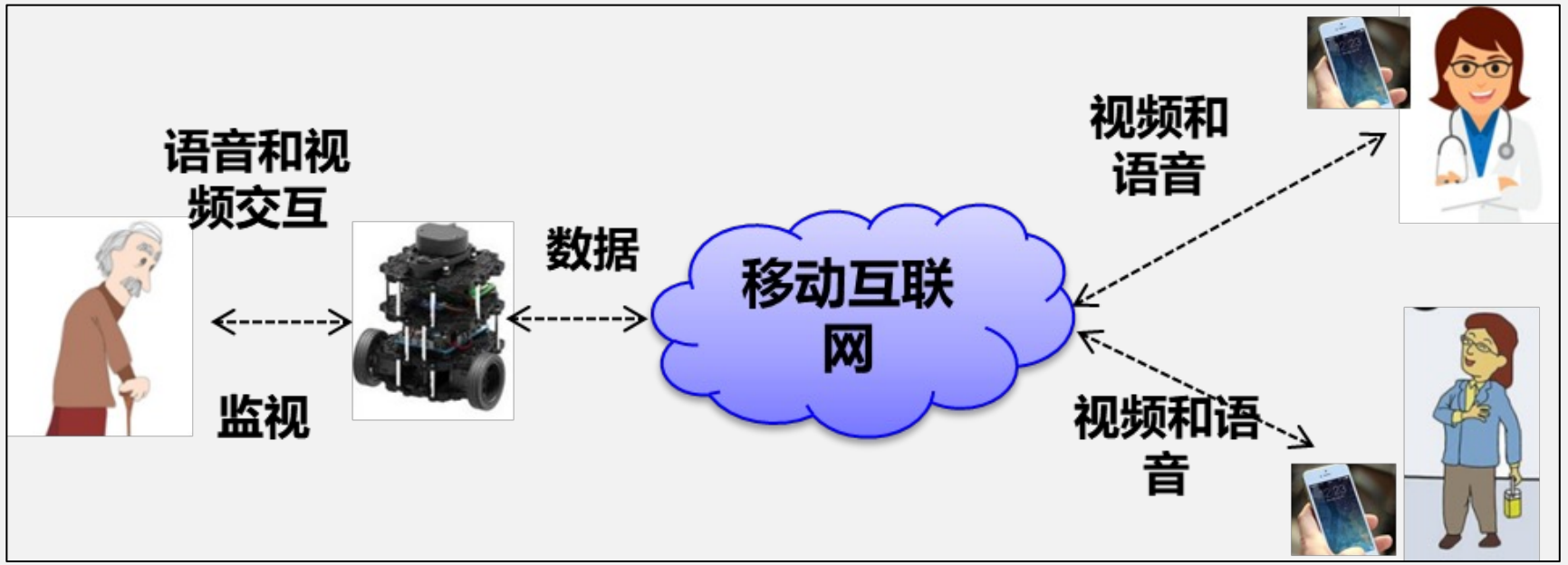
- ✓代表**一类用户**或者外部实体，而**非具体的对象实例**



Actor
Name

示例：“空巢老人看护软件”的参与者

- 老人
- 医生
- 家属
- 机器人
- 管理员
- …



参与者通常对应于软件系统的利益相关者

识别参与者

- ✓ 参与者是从现实世界中抽象出来的一种形式，却不一定确切对应现实中的某个特定对象。一个现实中的对象可以根据对系统的不同目的抽象成多个参与者；多个现实中的对象也可以按照对系统的相同目的而抽象为一个参与者。

- ✓ 例如，一个人可以是一个网站的管理员和普通用户，他的这两种身份对系统的目的和操作是不同的，因此他一个人对应于两个参与者。再比如，一个网站可以有成千上万个用户，他们对于网站所进行的操作和拥有的权限是相同的，那么他们全体被抽象为一个参与者。

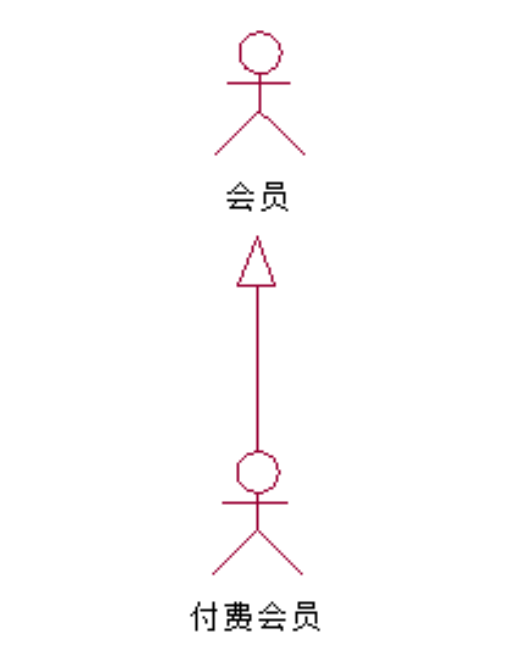
识别参与者

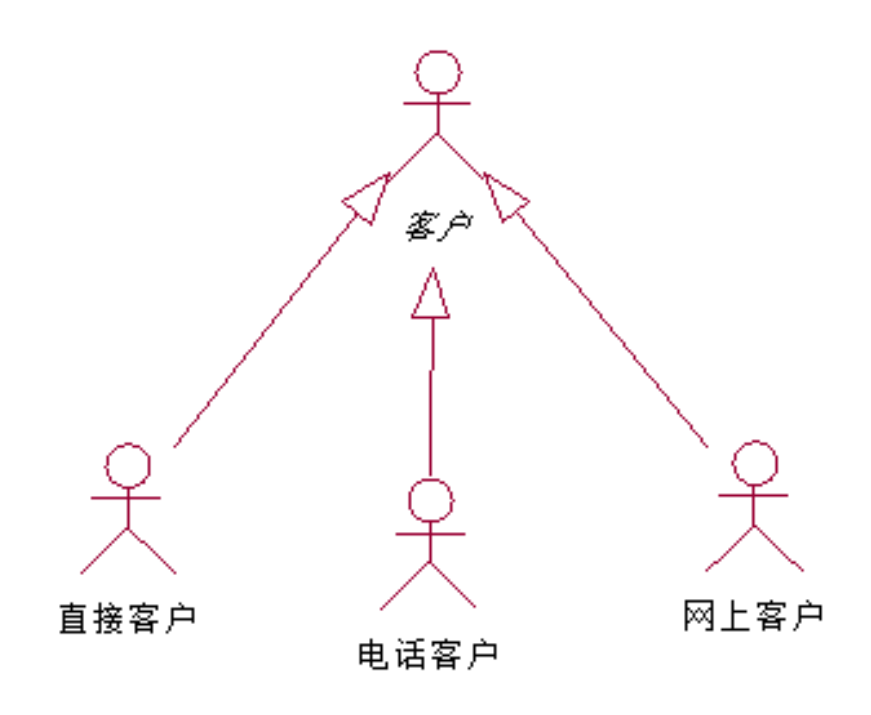
✓为了识别出一个系统所涉及到的参与者，我们可以向用户提出如下问题。

（1）谁将使用系统的主要功能?
（2）谁将需要系统的支持来完成他们的日常工作?
（3）谁必须维护、管理和确保系统正常工作?
（4）谁将给系统提供信息、使用信息和维护信息?
（5）系统需要处理哪些硬件设备?
（6）系统使用了外部资源吗?
（7）系统需要与其他系统交互吗?
（8）谁或者什么对系统产生的结果感兴趣

参与者之间的泛化关系

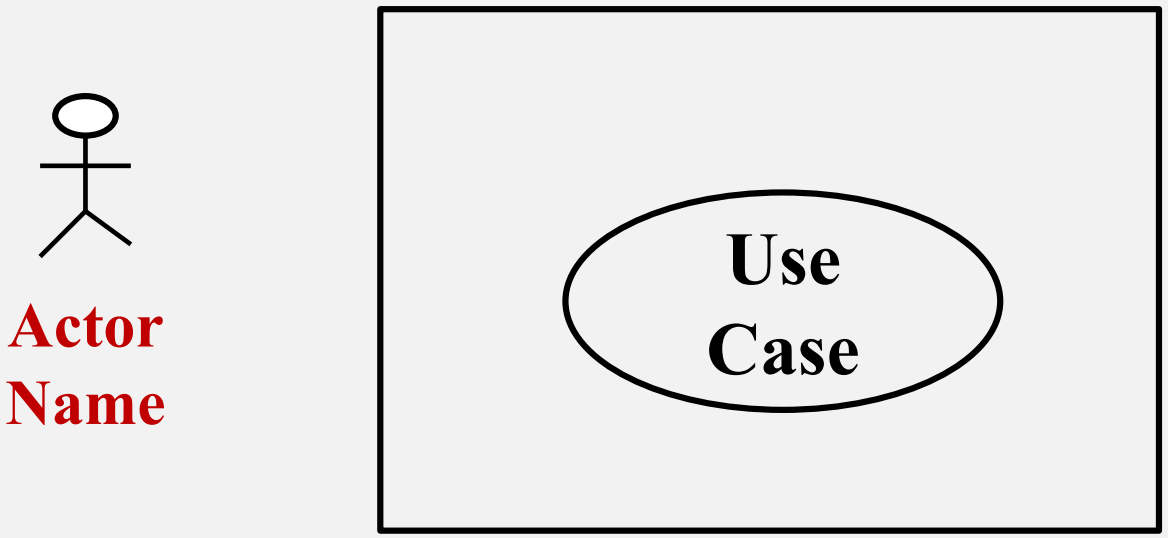
参与者的泛化关系





系统边界框

- 表示整个软件系统的边界
  - ✓边界框内的用例构成了系统的内容，如用例
  - ✓外面的是系统之外的参与者



用例(Use Case)

❑用例

- ✓表示参与者为达成一项**相对独立**、**完整**的**业务目标**而要求软件系统完成的**功能**



❑用例的表现形式

- ✓表现为执行者与系统之间的**业务交互动作的序列**
- ✓对于执行者而言，**交互目的**或者效果在于**达成其业务目标**
- ✓对于待开发系统而言，交互的过程即是某项**相对独立**、**完整**的**外部可见功能**的实现过程

用例的特征

用例是可以被执行者感受到的、系统的一个完整的功能。在UML中把用例定义成系统完成的一系列动作序列（场景），动作的结果能够被特定的参与者察觉到。

用例具有下述特征：

- ✓ 用例代表某些**用户可见的功能，实现一个具体的用户目标；**
- ✓ **用例总是被参与者启动，并向参与者提供可识别的值；**
- ✓ **用例可大可小，但它必须是完整的（不是基本的CRUD操作）**
- ✓ **用例在以后的开发过程中，可以进行独立的功能检测。**

用例的特点

用例往往是动宾短语

- ✓ 表达的是**一个交互序列**，因此可用**动词词组或动宾短语**来命名
- ✓ 用例的存在意义在于**实现参与者的目的**，动宾短语可以简明扼要地表达意愿和目的

✅ 登录系统、取款、选课

❌ 登录器、取款单、课程表、购票处

用例独立和完备性

- ✓ 与其他用例**无交互**，可独自完成参与者的某项目的
- ✓ 本质上体现了**参与者完整愿望**，设计的用例必须要实现一个完整的目的（**用户故事**）

✅ **ATM机“取钱”**

❌ “输入密码”不能完整地实现参与者的愿望，仅是取钱过程中的一个步骤

✅ **“取车”**可以作为一个用例

❌ “打开车库门”不可以作为一个用例

用例的特点 （续）

用例由参与者启动

- ✓ 用例是参与者请求或触发的一系列行为，因此不存在没有参与者的用例
- ✓ 用例一般不应该自启动，多个用例之间也不应该互相启动

例子:

- 对于“取钱”这一用例而言，主参与者是取款人而不是自动取款机
- 自动触发的用例：还款日更新

用例要有可观测的执行结果

- ✓ 不是所有的执行过程都最终成为用例
- ✓ 用例必须返回一些可以观测的执行结果
- ✓ 缺少了返回消息的用例是错误的

例子

- 登录系统时应该及时通知用户“登录成功”或者跳转到把参与者作为成员的页面
- 密码出错，也应该及时给出提示“密码错误”以便用户改正或重试等等

主观题 10分

设置



- **参考自然语言需求模版，猜想一下用例图里面画完用例是否已经建模完成?**

作答

用例描述

一般的用例描述主要包括以下几部分内容：

- ✓ **用例名称**：描述用例的意图或实现的目标，一般为动词词语或动宾短语。
- ✓ **用例编号**：用例的唯一标识符，在其他位置可以使用该标识符来引用用例。
- ✓ **参与者**：描述用例的参与者，包括主要参与者和其他参与者。
- ✓ **用例描述**：对用例的一段简单的概括描述。
- ✓ **触发器**：触发用例执行的一个事件。例如，“生成订单”用例的触发器是“用户提交了一个新订单时”。
- ✓ **前置条件**：用例执行前系统状态的约束条件。
- ✓ **基本事件流（典型过程）**：用例的常规活动序列，包括参与者发起的动作与系统执行的响应活动。
- ✓ **扩展事件流（替代过程）**：记录如果典型过程出现异常或变化时的用例行为，即典型过程以外的其他活动步骤。
- ✓ **后置条件**：用例执行后系统状态的约束条件。
- ✓ **补充约束**：用例实现时需要考虑的业务规则、实现约束等信息。

用例的识别

通过提出以下问题，帮助确定用例

（1）参与者要**向系统请求什么功能**?

（2）每个**参与者的特定任务**是什么?

（3）参与者需要读取、创建、撤销、修改、或存储系统的**某些信息**吗?

（4）是否任何一个参与者都要向系统通知有关突发性的、外部的改变？或者必须通知参与者关于系统中发生的事件?

（5）这些事件代表了哪些功能?

（6）系统需要哪些输入/输出?

（7）哪些用例支持或维护系统?

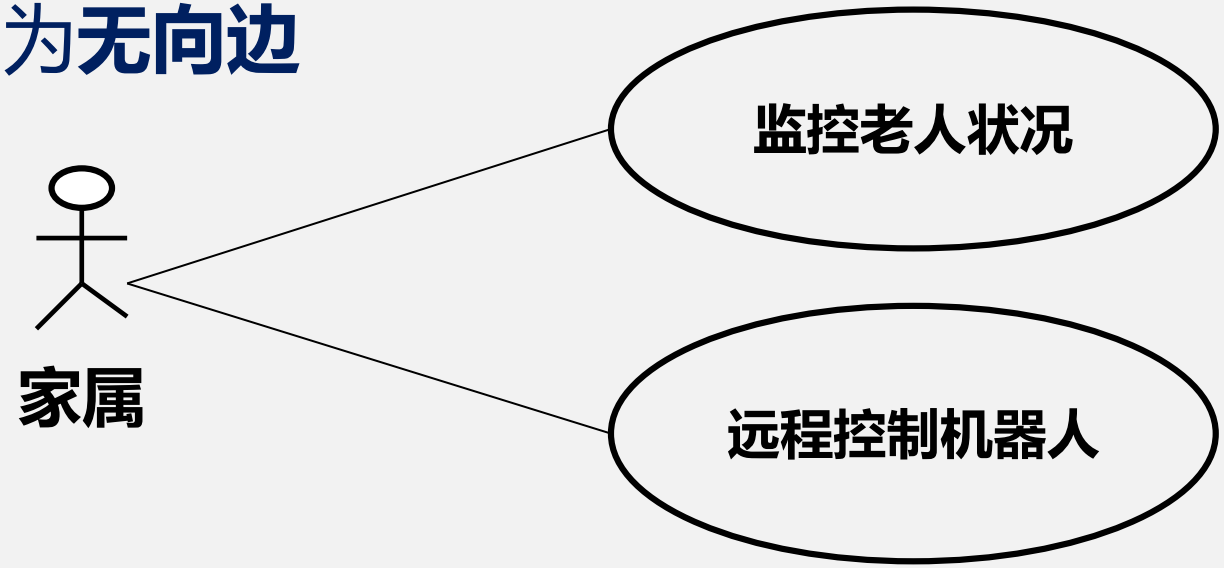
（8）是否所有的功能需求都被用例使用了?

用例的实例是系统的一种实际使用方法，通常把用例的实例称为脚本或场景

执行者与用例间的关系

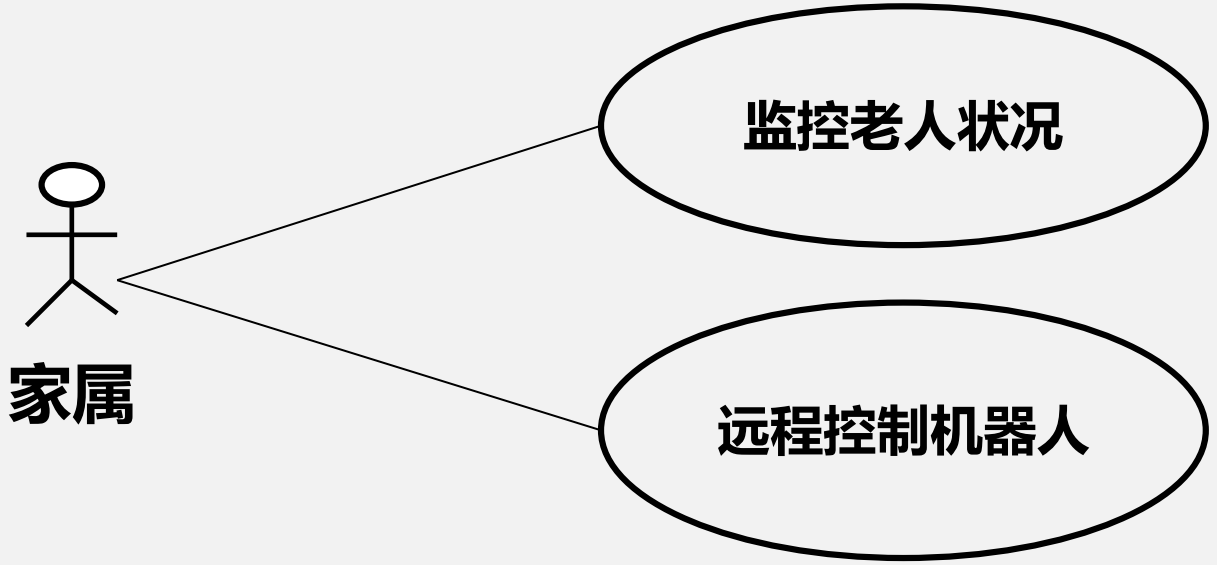
❑执行者与用例间关系的内涵

- ✓执行者与用例之间存在**交互**
- ✓执行者触发用例执行，向用例**提供信息或从用例获取信息**
- ✓触发用例执行的执行者称为**主动执行者**，仅从用例获取信息的执行者称为**被动执行者**
- ✓执行者与用例间的边通常为**无向边**



示例：用例

- “家属”参与者所看到的用例
  - ✓监控老人状况
  - ✓远程控制机器人



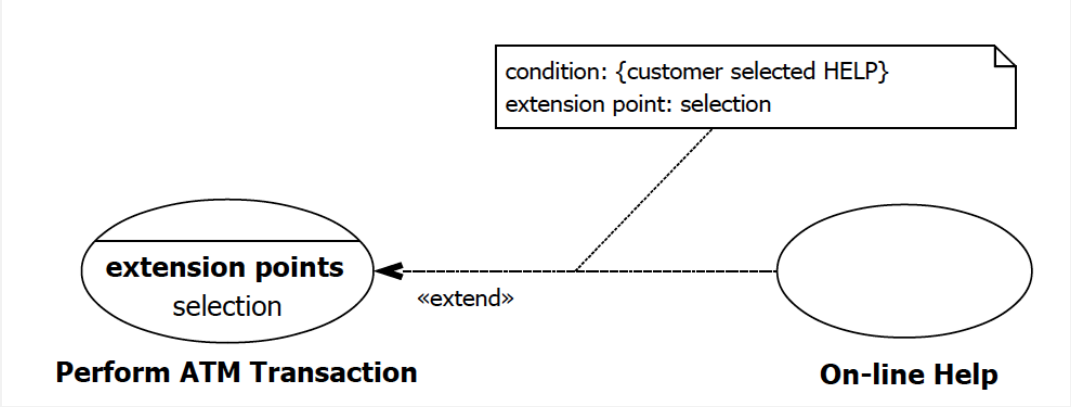
用例间的关系

① 包含(Include)
② 扩展(Extend)



例子：用例之间的关系

1.扩展关系



2.包含关系

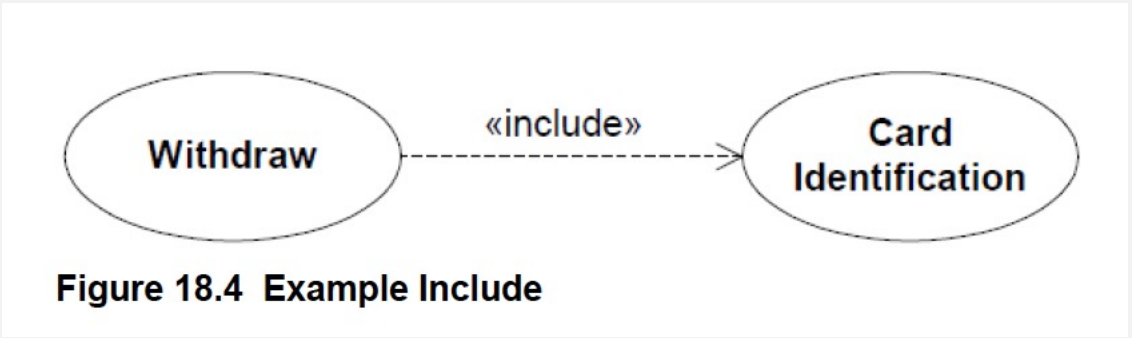


Figure 18.4 Example Include

主观题 10分

设置



- **参考上面例子，说明一下include与exclude的关系，特别include是不是中文的包含关系?**
- **（如图书管理用例include借书、还书等用例）**

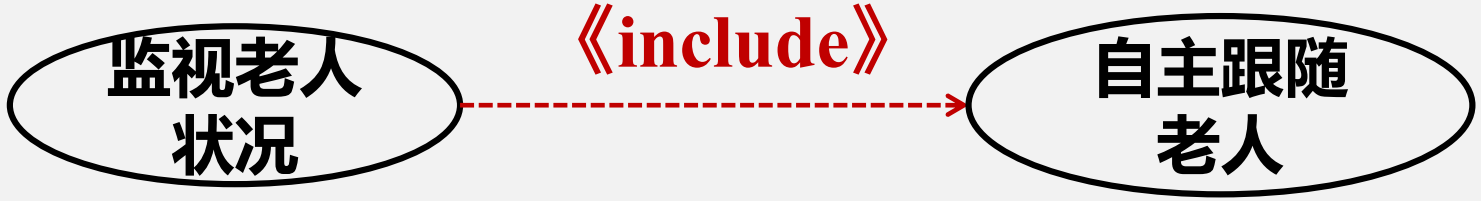
作答

用例间的关系-包含关系

❑如果用例B是用例A的某项子流程，则称用例A包含用例B

✓包含关系用于提取多个用例中的**公共子流程**，以避免重复和冗余

✓体现了功能分解和组织的思想



用例模型

✓ **包含关系**

如果在**若干用例中有某些相同的动作**，则可以把这些**相同的动作提取出来单独构成一个用例**。这样当某个用例使用该抽象用例时，就好像这个用例包含了抽象用例中的所有动作。

用例间的关系-扩展关系

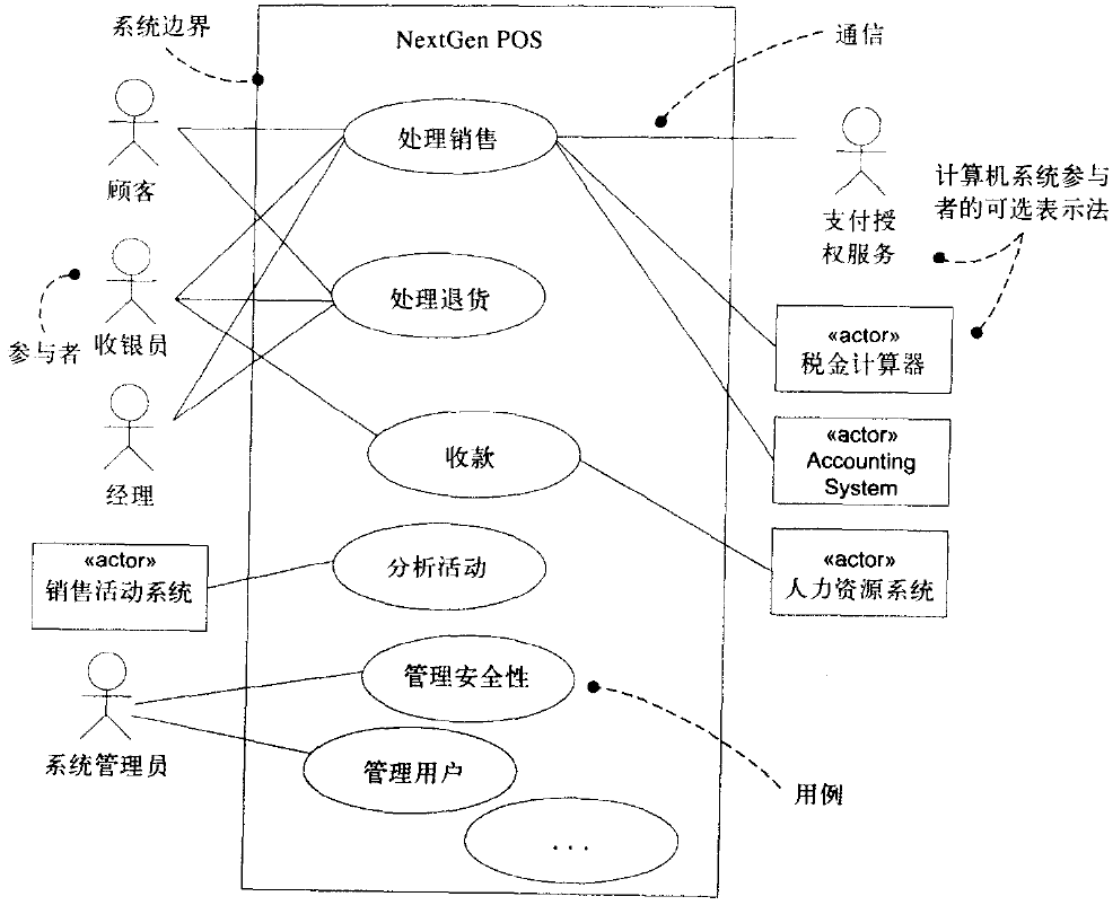
- 如果用例A与B，B的动作序列是在A的动作序列中的**某些执行点上插入附加动作序列**而构成的，则称**用例B扩展用例A**

- 用来区隔正常业务处理功能和带有例外处理的功能
  - ✓避免例外处理逻辑搅乱或湮灭正常处理逻辑
  - ✓体现了功能增强

包含和扩展有何本质性的区别?



超市管理系统 – 用例模型



用例UC1：处理销售

范围：NextGen POS应用

级别：用户目标

主要参与者：收银员

涉众及其关注点：

- 收银员：希望能够准确、快速地输入，而且没有支付错误，因为如果少收货款，将从其薪水中扣除。
- 售货员：希望自动更新销售提成。
- 顾客：希望以最小代价完成购买活动并得到快速服务。希望便捷、清晰地看到所输入的商品项目和价格。希望得到购买凭证，以便退货。
- 公司：希望准确地记录交易，满足顾客要求。希望确保记录了支付授权服务的支付票据。希望有一定的容错性，即使在某些服务器构件不可用时（如远程信用卡验证），也能够完成销售。希望能够自动、快速地更新账务和库存信息。
- 经理：希望能够快速执行超控操作，并易于更正收银员的不当操作。
- 政府税收代理：希望能从每笔交易中抽取税金。可能存在多级税务代理，比如国家级、州级和县级。
- 支付授权服务：希望接收到格式和协议正确的数字授权请求。希望准确计算对商店的应付款。

前置条件：收银员必须经过确认和认证。

超市管理系统 - 用例模型

成功保证（或后置条件）：存储销售信息。准确计算税金。更新账务和库存信息。记录提成。生成票据。记录支付授权的批准。

主成功场景（或基本流程）：

1. 顾客携带所购商品或服务到收银台通过POS机付款。
2. 收银员开始一次新的销售交易。
3. 收银员输入商品条码。
4. 系统逐条记录出售的商品，并显示该商品的描述、价格和累计额。价格通过一组价格规则来计算。

收银员重复3～4步，直到输入结束。

5. 系统显示总额和所计算的税金。
6. 收银员告知顾客总额，并请顾客付款。
7. 顾客付款，系统处理支付。
8. 系统记录完整的销售信息，并将销售和支付信息发送到外部的账务系统（进行账务处理和提成）和库存系统（更新库存）。
9. 系统打印票据。
10. 顾客携带商品和票据离开（如果有）。

扩展（或替代流程）：

*a. 经理在任意时刻要求进行超控操作：
   1. 系统进入经理授权模式。
   2. 经理或收银员执行某一经理模式的操作。例如，变更现金结余，恢复其他登录者中断的销售交易，取消销售交易等。
   3. 系统回复到收银员授权模式。

*b. 系统在任意时刻失败：

为了支持恢复和更正账务处理，要保证所有交易的敏感状态和事件都能够从场景的任何一步中完全恢复。

   1. 收银员重启系统，登录，请求恢复上次状态。
   2. 系统重建上次状态。
      2a. 系统在恢复过程中检测到异常：
         1. 系统向收银员提示错误，记录此错误，并进入一个初始状态。
         2. 收银员开始一次新的销售交易。

1a. 客户或经理需要恢复一个中断的销售交易。
   1. 收银员执行恢复操作，并且输入ID以提取对应的销售交易。
   2. 系统显示被恢复的销售交易状态及其小计。
      2a. 未发现对应的销售交易。
         1. 系统向收银员提示错误。
         2. 收银员可能会开始一个新销售交易，并重新输入所有商品。
   3. 收银员继续该次销售交易（可能要输入更多的商品或处理支付）。

超市管理系统 - 用例模型

2-4a. 顾客告诉收银员其免税状况（例如，年长者、本国人等）。
1. 收银员进行核实，并输入免税状况编码。
2. 系统记录该状况编码（在计算税金时使用）。

3a. 无效商品ID（在系统中未发现）：
1. 系统提示错误并拒绝输入该ID。
2. 收银员响应该错误。
2a. 商品ID可读（例如，数字型的UPC（通用产品代码））：
1. 收银员手工输入商品ID。
2. 系统显示商品项目的描述和价格。
2a. 无效商品ID：系统提示错误。收银员尝试其他方式。
2b. 系统内不存在该商品ID，但是该商品附有价签：
1. 收银员请求经理执行超控操作。
2. 经理执行相应的超控操作。
3. 收银员选择手工输入价格，输入价签上的价格，并且请求对该价目进行标准计税。（因为没有产品信息，计税引擎无法确定如何计税。）
2c. 收银员通过执行寻找产品帮助以获取正确的商品ID及其价格。
2d. 另外，收银员可以向其他员工询问商品ID或价格，然后手工输入ID或价格（参见以上内容）。

3b. 当有多个商品项目属于同一类别的时候（如5个汉堡包），不必记录每个商品项目的唯一标识：
1. 收银员可以输入类别的标识和商品的数量。

3c. 需要手工输入类别和价格（例如，花卉或纸牌及其价格）：
1. 收银员手工输入特定的类别代码及其价格。

3-6a. 顾客要求收银员从所购商品中去掉一项：
所去除商品的价格必须小于收银员权限，否则需要经理执行超控操作。
1. 收银员输入商品ID并将其删除。
2. 系统删除该项目并显示更新后的累计额。
2a. 商品价格超过了收银员权限：
1. 系统提示错误，并建议经理超控。
2. 收银员请求经理超控，完成超控后，重做该操作。

3-6b. 顾客要求收银员取消销售交易：
1. 收银员在系统中取消销售交易。

3-6c. 收银员延迟销售交易：
1. 系统记录销售交易信息，使其能够在任何POS登录中恢复操作。
2. 系统显示用来恢复销售交易的“延迟票据”，其中包含商品项目和销售交易ID。

4a. 系统定义的商品价格不是顾客预期的价格（顾客对此产生抱怨并要求减价）：
1. 收银员请求经理批准。
2. 经理执行超控操作。

超市管理系统 - 用例模型

3. 收银员手工输入超控后的价格。
4. 系统显示新价格。

5a. 系统检测到与外部税务计算系统服务的通信故障：
1. 系统在POS机节点上重启此服务，并继续操作。
1a. 系统检测到该服务无法重启。
1. 系统提示错误。
2. 收银员手工计算和输入税金，或者取消该销售交易。

5b. 顾客声称他们符合打折条件（例如，是雇员或重要顾客）：
1. 收银员提出打折请求。
2. 收银员输入顾客ID。
3. 系统按照打折规则显示折扣总计。

5c. 顾客要求兑现账户积分，用于此次销售交易：
1. 收银员提交积分请求。
2. 收银员输入顾客ID。
3. 系统应用积分直到价格为0，同时扣除结余积分。

6a. 顾客要求现金付款，但所携现金不足：
1. 顾客要求使用其他支付方式。
1a. 顾客要求取消此次销售交易，收银员在系统上取消该销售交易。

7a. 现金支付：
1. 收银员输入收取的现金额。
2. 系统显示找零金额，并弹出现金抽屉。
3. 收银员放入收取的现金，并给顾客找零。
4. 系统记录该现金支付。

7b. 信用卡支付：
1. 顾客输入信用卡账户信息。
2. 系统显示其支付信息以备验证。
3. 收银员确认。
3a. 收银员取消付款步骤。
1. 系统回复到“商品输入“模式。
4. 系统向外部支付授权服务系统发送支付授权请求，并请求批准该支付。
4a. 系统检测到与外部系统协作时的故障：
1. 系统向收银员提示错误。
2. 收银员请求顾客更换支付方式。
5. 系统收到批准支付的应答并提示收银员，同时弹出现金抽屉（以便放入签名后的信用卡支付票据）。
5a. 系统收到拒绝支付的应答：
1. 系统向收银员提示支付被拒绝。
2. 收银员请求顾客更换支付方式。

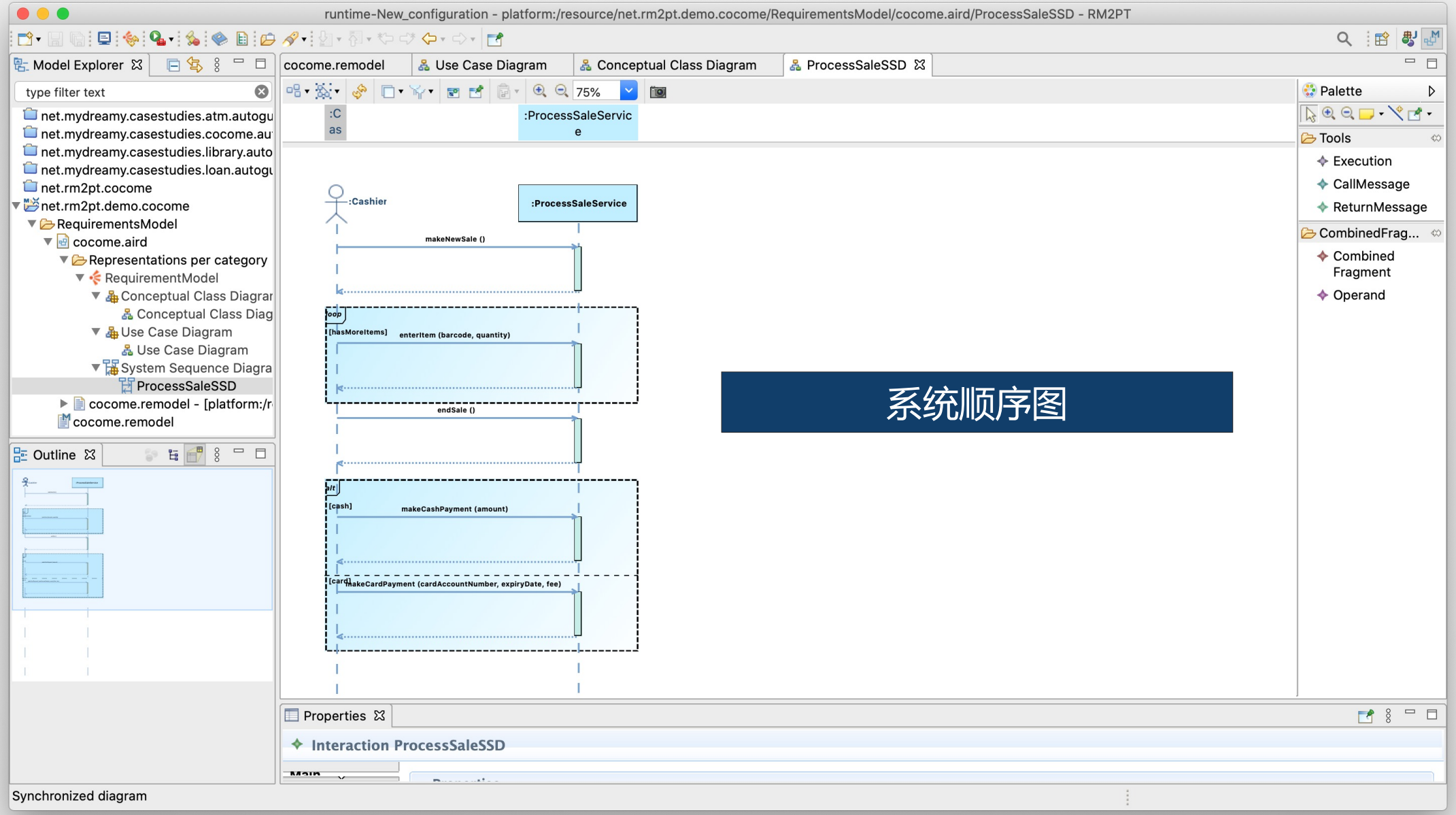
超市管理系统 - 用例模型

5b. 应答超时。
1. 系统提示收银员应答超时。
2. 收银员重试，或者请求顾客更换支付方式。
6. 系统记录信用卡支付信息，其中包括支付批准。
7. 系统显示信用卡支付的签名输入机制。
8. 收银员请求顾客签署信用卡支付。顾客输入签名。
9. 如果在纸质票据上签名，则收银员将该票据放入现金抽屉并关闭抽屉
7c. 支票支付……
7d. 记账支付……
7e. 收银员取消支付步骤：
1. 系统回到“商品输入”模式。
7f. 顾客出示优惠券：
1. 在处理支付之前，收银员记录每张优惠券，系统扣除相应金额。系统记录已使用的优惠券以备账务处理之用。
1a. 输入的优惠券不适用于所购商品：
1. 系统向收银员提示错误。
9a. 存在产品回扣：
1. 系统对每个具有回扣的商品给出回扣表单和票据。
9b. 顾客索要赠品票据（不显示价格）：
1. 收银员请求赠品票据，系统给出赠品票据。
9c. 打印票据。
1. 如果系统能够检测到错误，给出提示。
2. 收银员更换纸张。
3. 收银员请求打印其他票据。

特殊需求：

- 使用大尺寸平面显示器触摸屏UI。文本信息可见距离为1米。
- 90%的信用卡授权响应时间小于30秒。
- 由于某些原因，我们希望在访问远程服务（如库存系统）失败的情况下具有比较强的恢复功能。
- 支持文本显示的语言国际化。
- 在步骤3和步骤7中能够加入可插拔的业务规则。
- ……

RM2PT 建模展示 – 系统顺序图

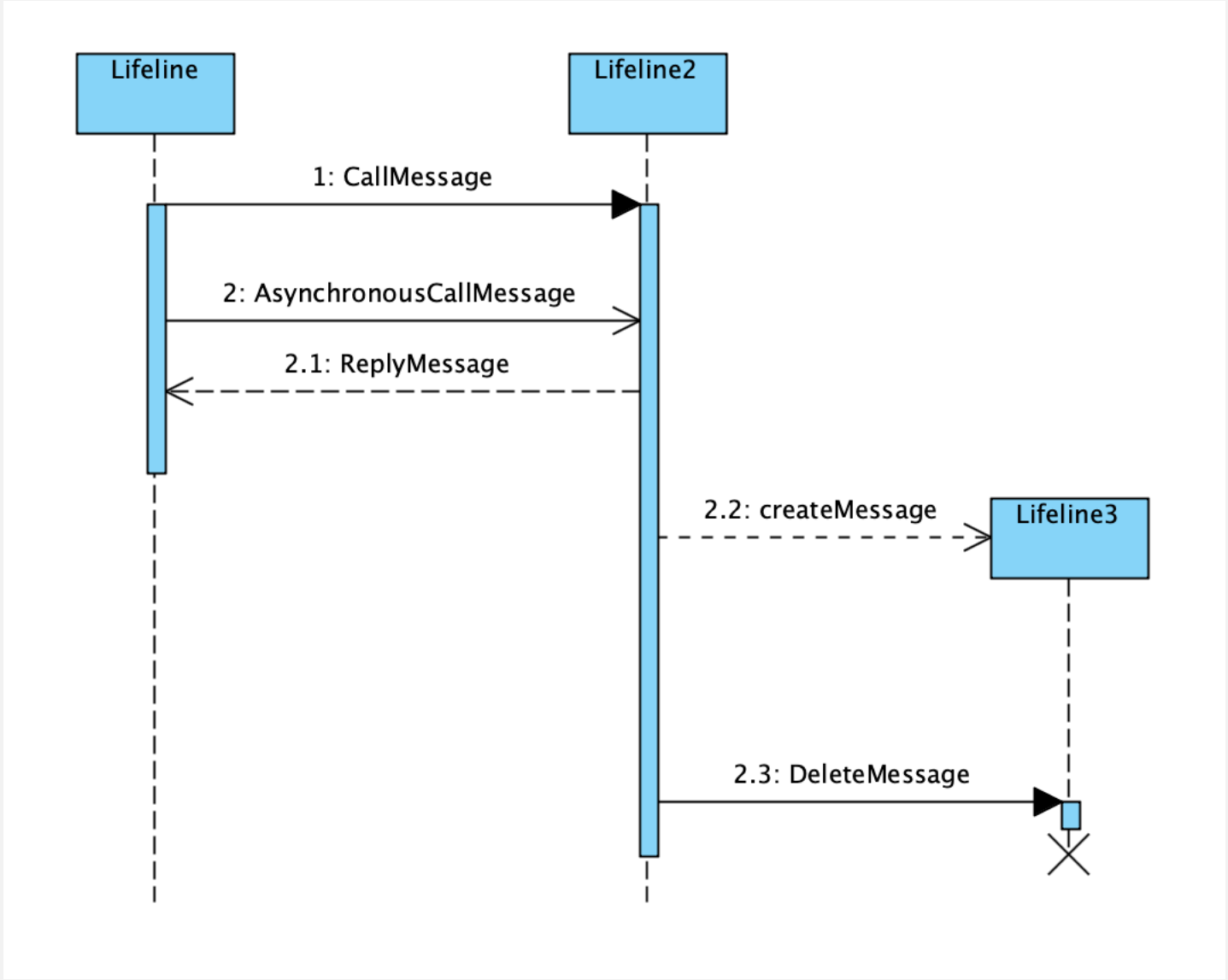


顺序图 (Sequence Diagram)

- ✓ **顺序图**是**按时间顺序**显示**对象交互（组件交互/用户与系统）**的图。它显示了参与交互的对象和所**交换信息的先后顺序**，用来**表示用例中的行为**，并将这些行为建模成信息交换。

- ✓ 顺序图主要包括**四个元素**：**对象**、**生命线**、**激活**和**消息**。在UML中，顺序图将交互关系表示为一张二维图。其中**纵向是代表时间**维度，时间向下延伸，按时间依次列出各个对象所发出和接收的消息。**水平方向是代表对象的维度**，排列着**参与交互的各个独立的对象**。

顺序图


Lifeline
Lifeline2
1: CallMessage
2: AsynchronousCallMessage
2.1: ReplyMessage
2.2: createMessage
Lifeline3
2.3: DeleteMessage

顺序图的概念

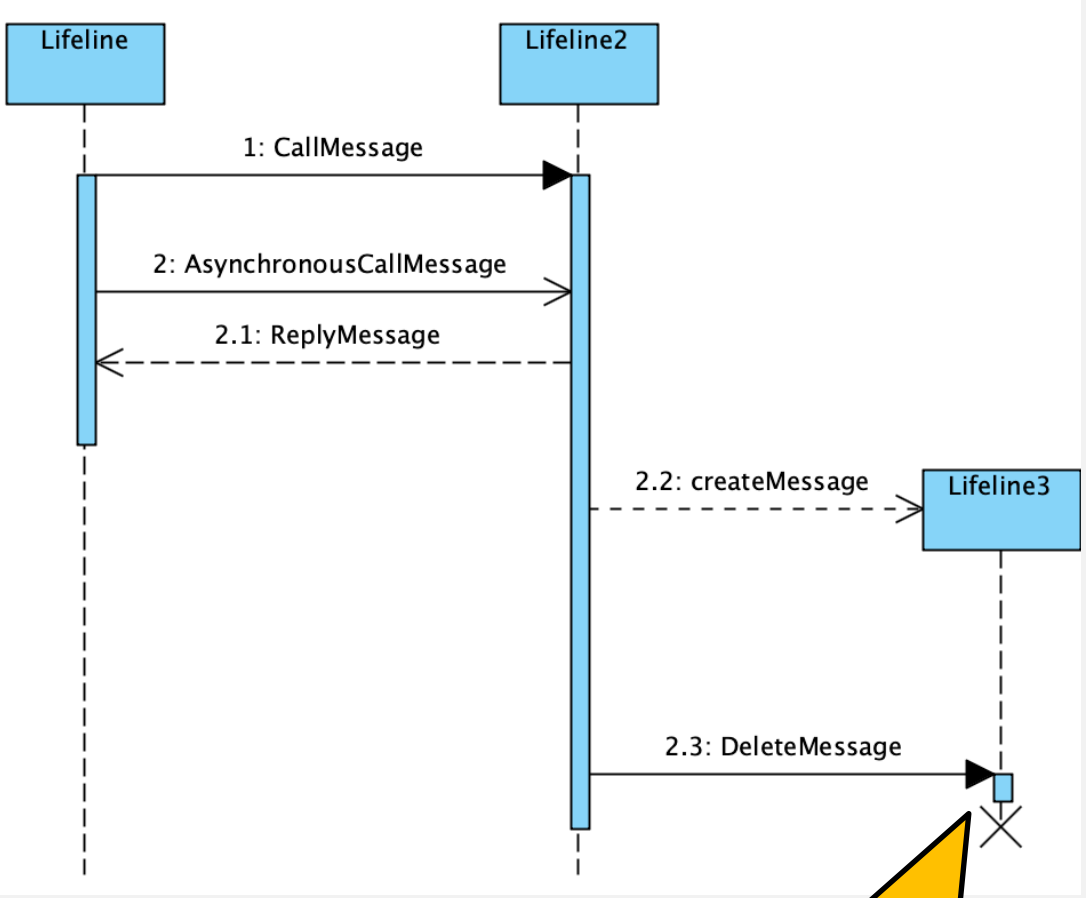
顺序图的三种主要作用：

- ✓ **细化用例的表达**。使用顺序图的一大用途，就是将**用例所描述的需求与功能**转化为更加**正式、层次更加分明的细化表达**。
- ✓ **有效地描述类职责的分配方式**。我们可以根据顺序图中各**对象之间的交互关系**和发送的消息来进一步**明确对象所属类的职责**。
- ✓ **丰富系统的使用语境的逻辑表达**。系统的使用语境即为系统可能的使用方式和使用环境。

顺序图的组成元素

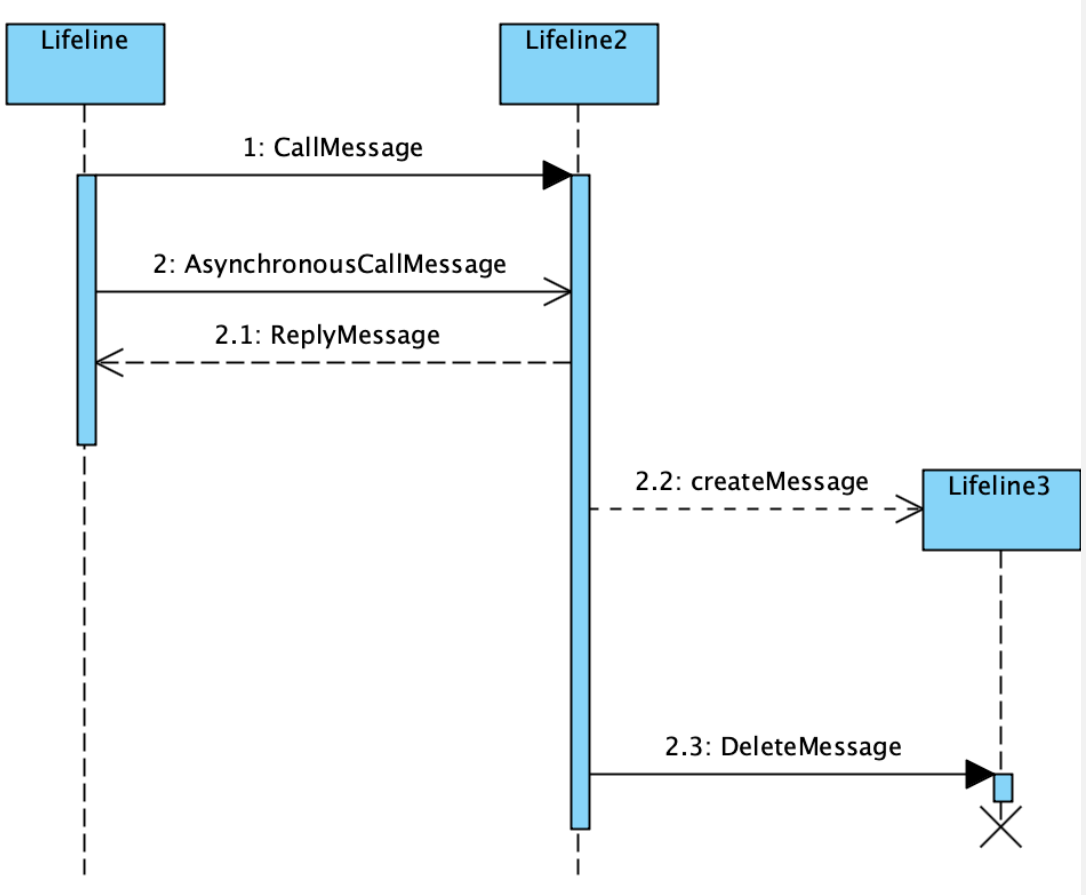
- 对象(Object)
- 生命线 (Lifeline)
- 激活 (Execution)
- 消息 (Message)

对象（Object）



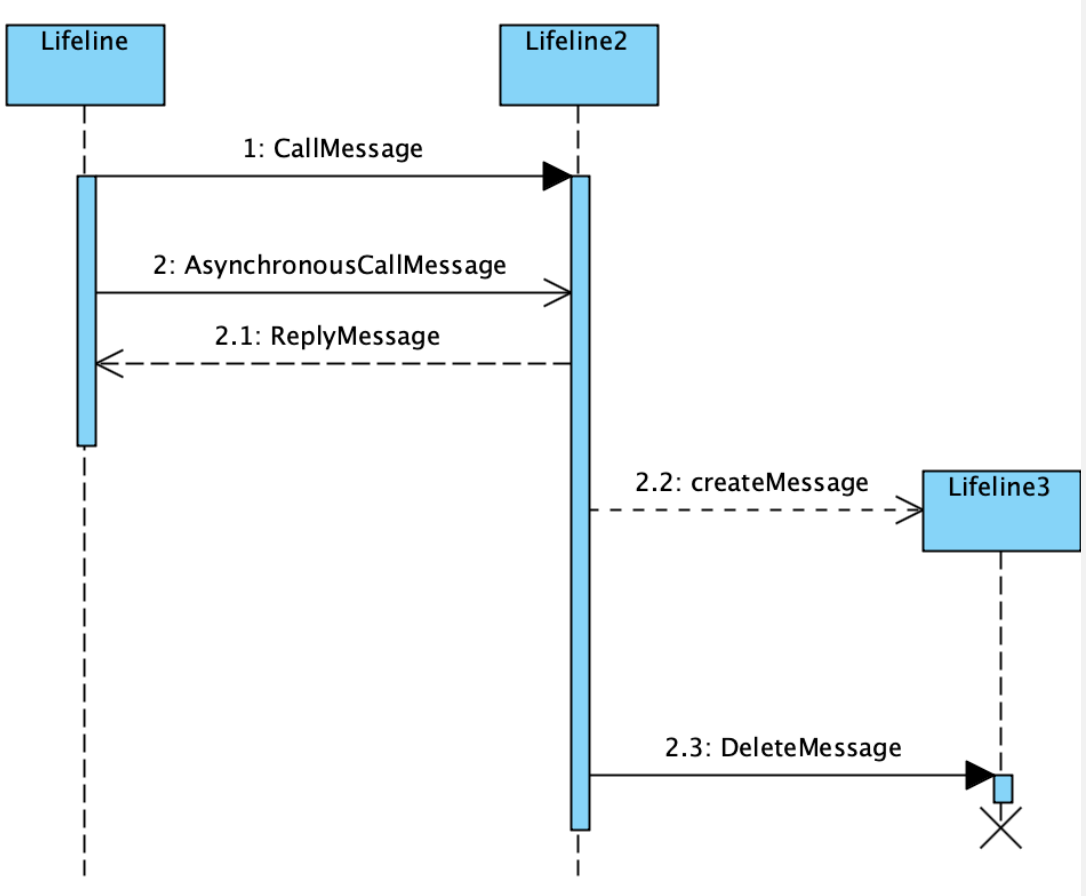
- ✓ 顺序图中的对象与对象图中的概念一样，都是**类的实例**。顺序图中的对象可以是系统的参与者或者任何有效的系统对象
- ✓ 对象的创建由头符号来表示，即在对象创建点的**生命线顶部**使用显示**对象名**和**类名的矩形框**来标记
- ✓ 在位置上，一个被放置于顺序图顶端的对象，意味着在**交互开始前**，系统已经拥有这样一个对象了。如果一个对象出现在其它位置上（不在顶端），则说明这个对象是在**交互执行到某些步骤时**被创建出来的。被创建出来的对象可以在接下来的时间里被其他对象的消息所激活，也可以以同样的方式被销毁

生命线（Lifeline）



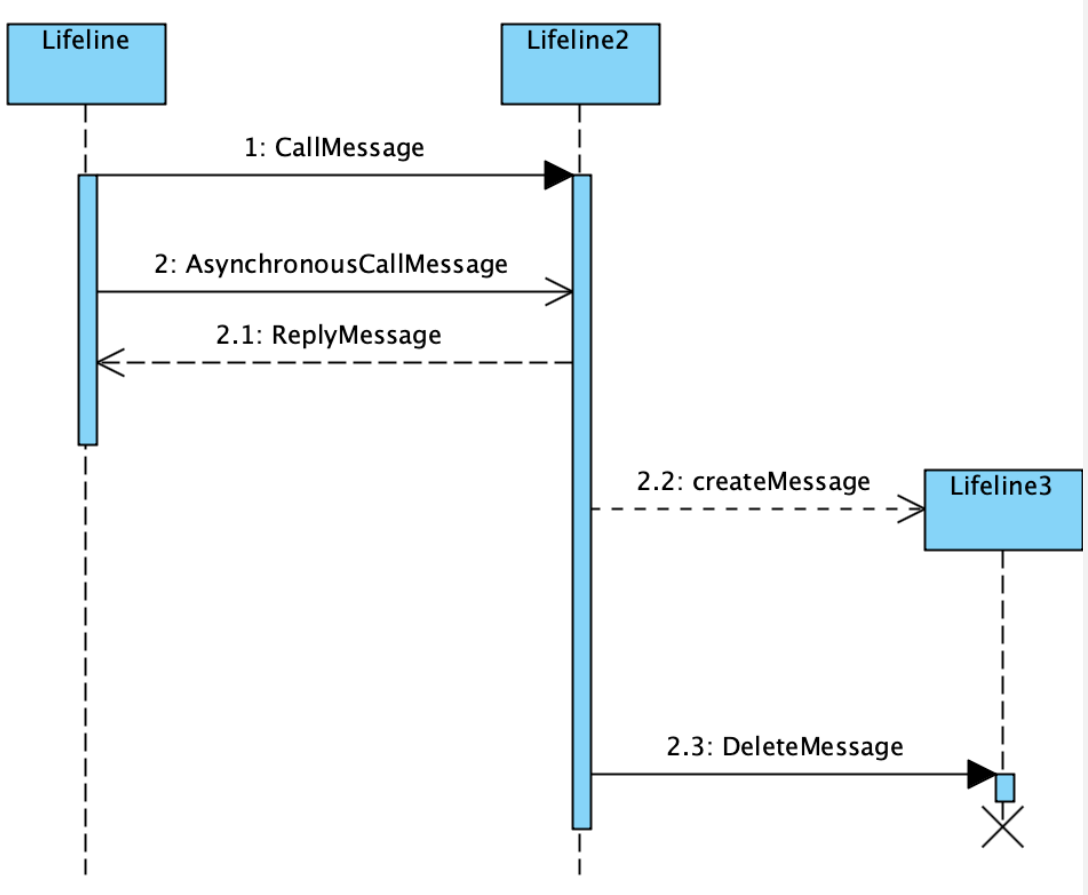
- ✓ 生命线代表了一次交互中的一个参与对象在**一段时间内存在，** 期间对象一直可以被访问（可随时发送消息）
- ✓ 生命线位于每个对象的底部中心位置，**显示为一条垂直的虚线，**与时间轴平行，带有一个显示对象的头符号
- ✓ 对于在交互过程中被创建的对象，其生命线从**接收到新建对象**的消息时开始
- ✓ 对于在交互过程中被销毁的对象，其生命线在**接收到销毁对象**的消息时或在**自身消息返回后**结束，同时用一个“X”标记表明生命线的结束

激活 （Execution）



- ✓ **激活期**，表示**一个对象执行一个动作**所经历的**时间段**，既可以是直接执行，也可以是安排下级过程执行。同时，激活也可以表示对应对象在这段时间内不是空闲的，它正在完成某个任务，或正被占用。
- ✓ **激活**在UML中用一个**细长的矩形表示**，显示在生命线上，矩形的顶部表示对象所执行动作的开始，底部表示动作的结束。

消息 (Message)

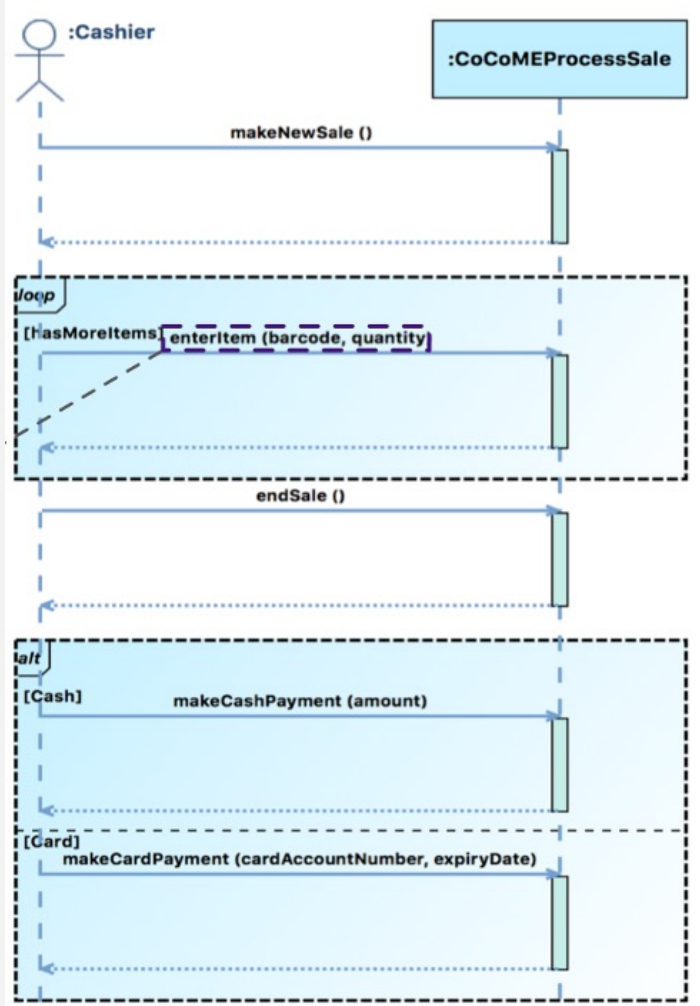


- ✓ **消息**是从一个对象（发送者）向另一个对象（接收者）发送**信号**，或由一个对象（发送者或调用者）调用另一个对象（接收者）的**操作**
- ✓ **消息**是对象间协同工作的信息载体，代表了一系列**对象实体间通信**
- ✓ 在顺序图中，消息表示为从一个对象的生命线指向另一个对象的生命线的**箭头**

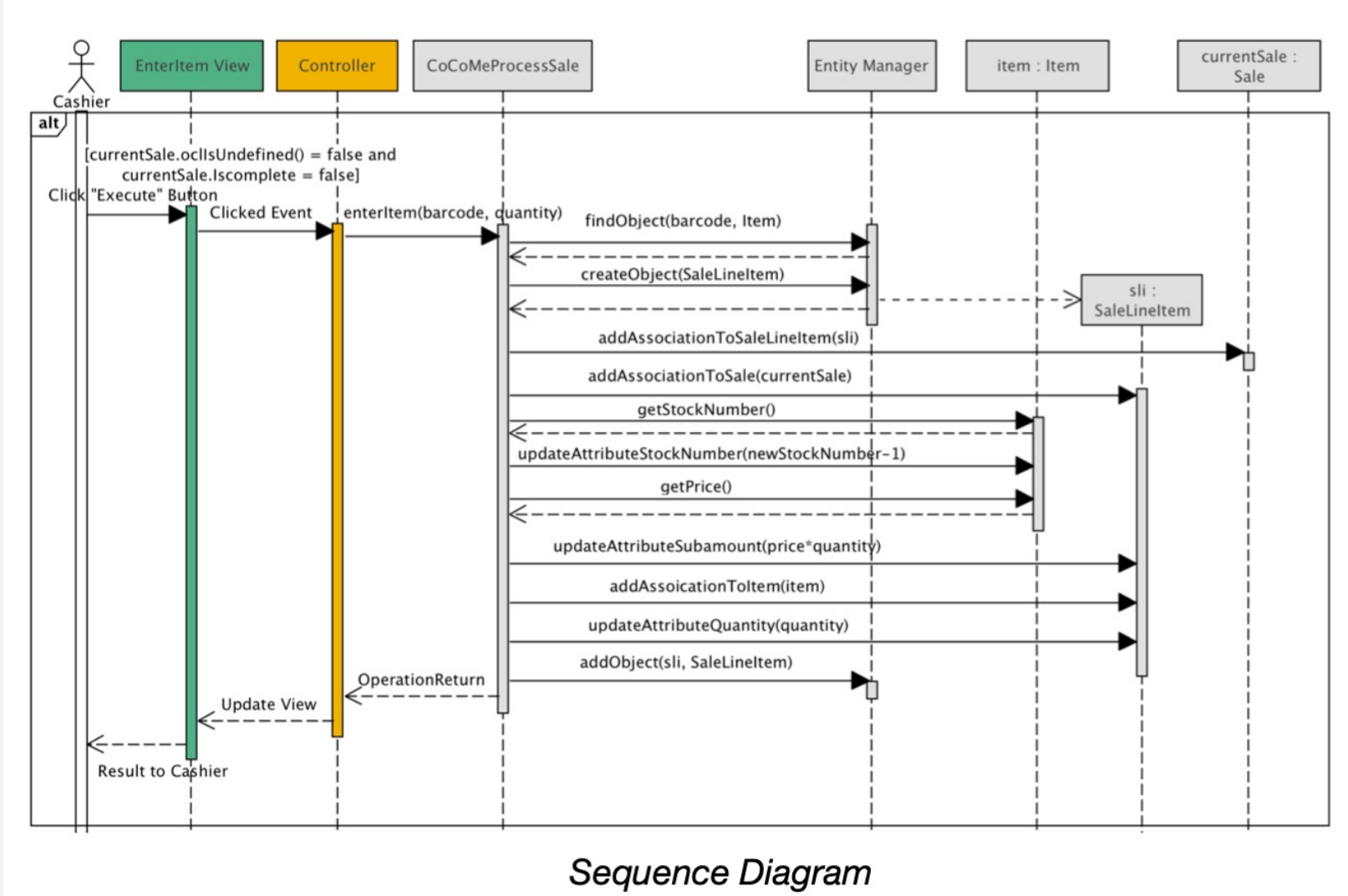
交互消息方式有哪些?

- **同步消息**：发送方必须等待接受方的响应，才能继续
- **异步消息**：发送方发出消息后继续工作，无需等待

系统顺序图（需求）VS 对象顺序图（设计）



System Sequence Diagrams



Sequence Diagram

面向对象的需求模型

- 用例图
- 系统顺序图
  - 系统状态图（可选）
  - 系统活动图（可选）
- 系统操作合约
  - 一阶逻辑
  - 对象约束语言

用例模型
（行为）

- 概念类图
- 概念复杂约束

领域概念模型
（结构）

需求
模型

主观题 10分

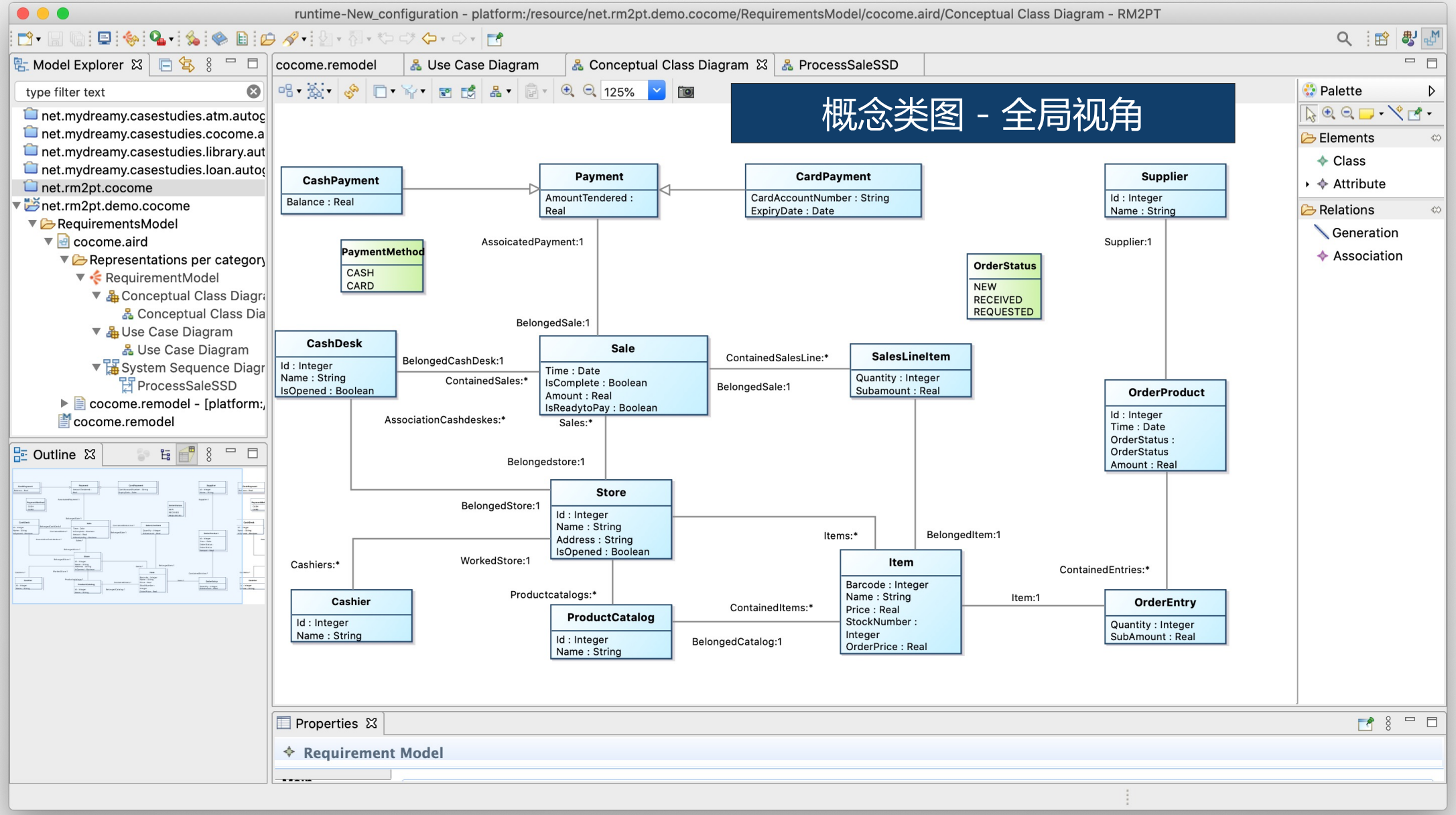
设置



- **参考E-R图，猜想概念类图里面都应该有什么内容?**

作答

RM2PT 建模展示 – 概念类图



面向对象的核心概念

- 对象 (Object)
- 类 (Class)
- 继承 (Inheritance)
- 多态 (Polymorphism)
- 覆盖/重写 (Override)
- 重载(Overload)
- 消息 (Message)
- 聚合 (Aggregation)和组合(Composition)

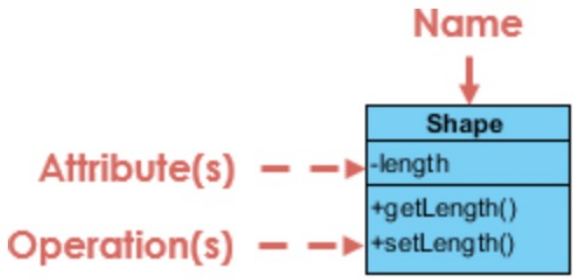
对象 (Object)

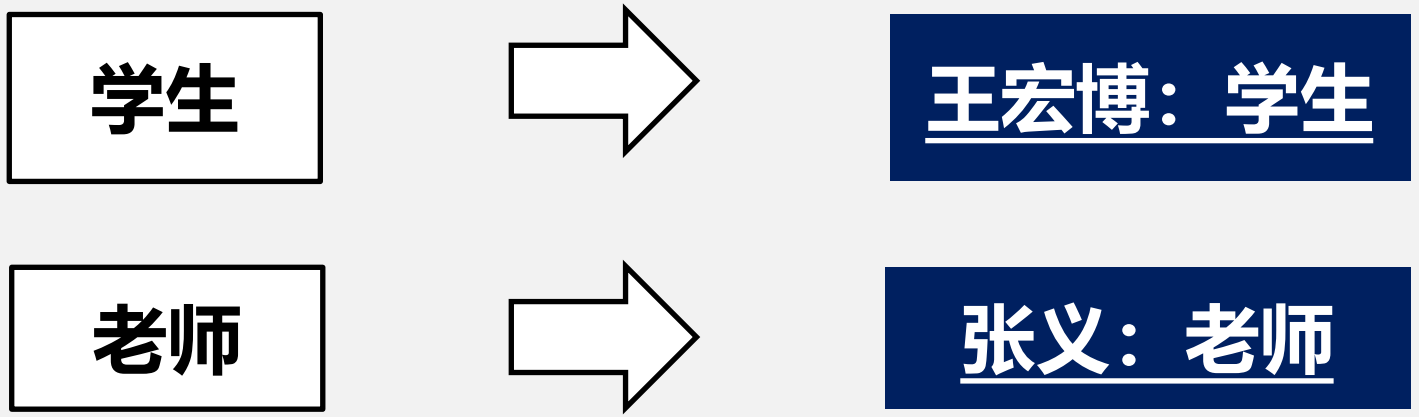
- **个体或者事物抽象表示**
  - ✓现实世界和计算机世界
  - ✓应用领域对象、软件实体
- **对象的属性和操作**
  - ✓**属性(Attribute)**：**对象的性质**，**其值**定义了**对象状态**
  - ✓**操作(Operation)**：也称**方法**，**对象行为**，表示**对象提供的服务**
- **示例**
  - ✓用对象表示**应用领域的一个事物**（如NAO机器人），也可以用它来表示在计算机软件中的**某个运行元素或单元**（如运行实例）

对象是具体、有意义的、存在的实体

类(Class)

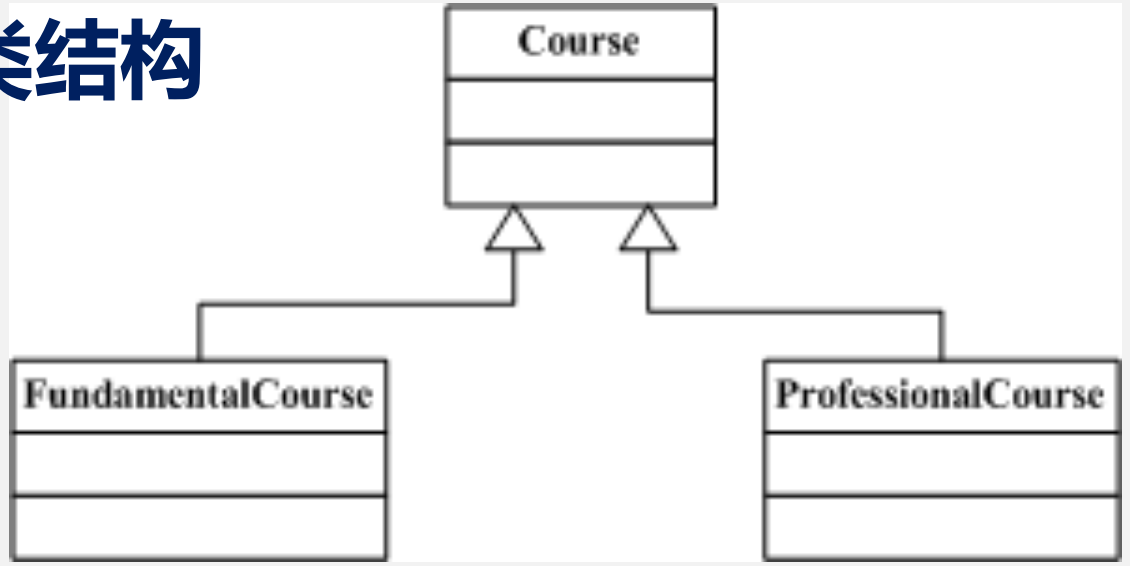
- **类是对一组具有相同特征对象的抽象**
  - ✓分类、组织机制，将具有相同特征的对象组织为一类
  - ✓封装了属性和操作
- **对象与类的关系**
  - ✓对象是类的**实例**，类是创建对象的**模板**
  - ✓类是**静态的抽象**；对象是**动态、可运行的实体**





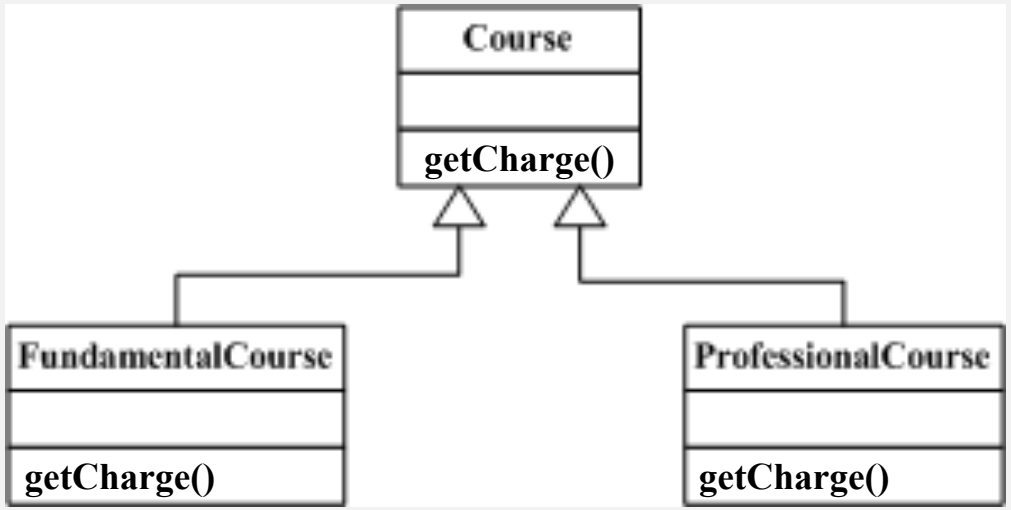
继承(Inheritance)

- **表示类与类间的一般与特殊关系**
  - ✓模拟现实世界类之间的遗传关系
- **子（特殊）类可共享父（一般）类的属性和操作**
  - ✓刻画**类间的内在联系**以及**对属性和操作的共享**
  - ✓**子类**也可以有自己的**独特属性和操作**
- **借助继承可形成系统的层次化类结构**
  - ✓示例：课程、公共课、专业课



多态(Polymorphism)

- **操作的外部接口定义形式相同，但是内部实现方式不一样**
  - ✓同一个操作作用于不同对象上可有不同解释，并产生不同结果
  - ✓课程、公共课、专业课三个类的“计费”操作实现不同
- **多态的执行方式**
  - ✓从具体子类开始，沿继承结构向上找，直至发现为止



实现多态的途径：覆盖/重写(Override) - 继承

- 子类**增加或重新定义**所继承的属性或方法，从而用新定义的属性和方法来**覆盖**所继承的、来自父类中的属性或方法

```
public class A  {
        String name;
        public String getValue(){
                return “Value is:” + name;
        }
    }

public class B extends A {
        String address;
        public String getValue () {
                return “Value is:” + address;
        }
    }
```

子类B重新定义所继承的方法getValue

实现多态的途径：参数重载(Overload) – 同一个类

- 一个类中有多个同名操作，但它们在操作数或操作数类型上有区别，系统根据实参引用不同方法

```
Public class A {
        int age;
        String name;

        public void setValue(int agePara){
           age = agePara;
        }

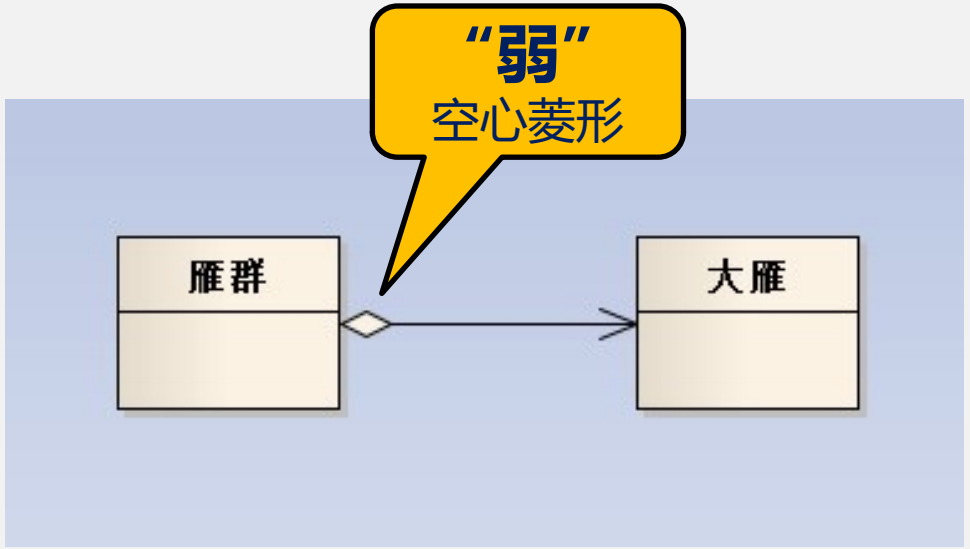
       public void setValue(String namePara){
            name = namePara;
       }
}
```

聚合(Aggregation)和组合(Composition)

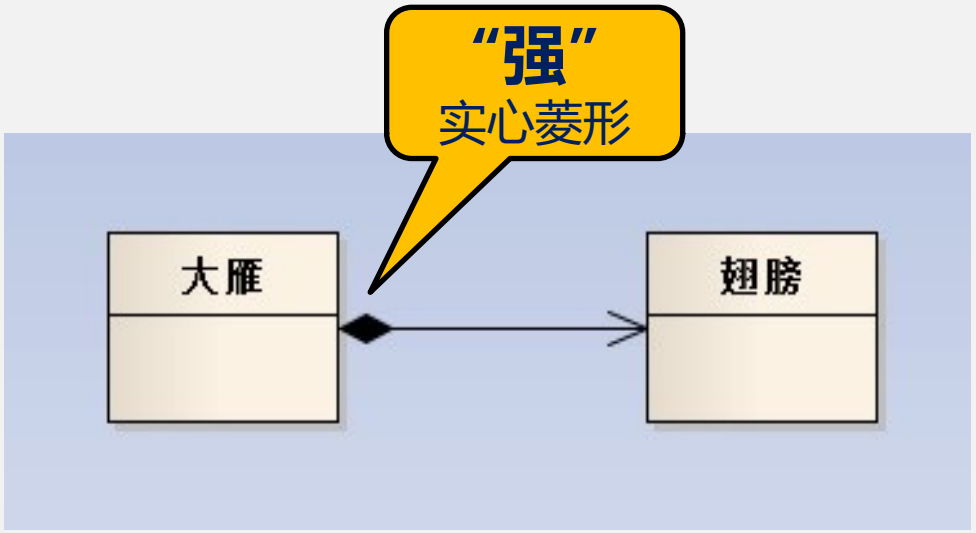
- **共性**
  - ✓均描述**整体-部分关系**，部分类对象是整体类对象的组成部分
- **差别**
  - ✓**聚合**：部分类对象可以是多个整体类对象的组成部分，即部分类对象可以为多个整体类对象**所共享**
  - ✓**组合**：部分类对象只能位于某个整体类对象之中，**一旦整体类对象消亡，其中部分类对象也无法生存。**从设计和实现的角度上看，**整体类必须具备完整的管理部分类生命周期的职责。**
- **示例**
  - ✓**聚合关系**：轮胎与汽车
  - ✓**组合关系**：房间与大楼

组合和聚合

- 聚合关系图



- 组合关系图



识别类与对象

找出侯选的类与对象

- 需要注意的事，**不一定每个名词都对应一个对象或类**，有时描述可能过细，那么该名词可能就是**其他对象的一个属性**。
- 另外鉴于需求陈述与用例描述不可能十分规范，我们还必须从这些**名词、名词短语**中排除同意或近义词的干扰。

✓**研究点：如何识别一个资源是类还是属性**

识别类与对象

系统实体的识别方法

- ✓ **可感知的物理实体**，例如汽车、书、信用卡；
- ✓ **人或组织的角色**，例如：学生、教师、经理、管理员、供应处；
- ✓ 应该**被记录的事件**，例如取款、飞行、订购；
- ✓ 两个或多个**对象的相互作用**，例如购买、结婚；
- ✓ **需要说明的概念**，例如保险政策、业务规则。

识别属性

1．分析（不可能在需求陈述中找到所有的属性）

2．确定属性 （属性的修改通常并不影响系统结构）

注意以下问题：

（1）误把对象当作属性

（2）误把关联类的属性当作对象的属性

（3）误把内部状态当成属性

（4）过于细化

（5）存在不一致的属性

（6）属性不能包含一个内部结构

（7）属性在任何时候只能有一个在其允许范围内的确切的值

识别关联关系

关联用于描述**类与类之间的连接**。

- ✓ **关联名称**：用于**标记关联**。
- ✓ **关联角色**：用于反映类在这**一关联关系中扮演了什么角色**。如果在关联上没有标出角色名，则隐含地用类的名称作为角色名。
- ✓ **关联导向性**：是指关联关系只在指定方向上成立，可以使用一个指向符号表示。

识别关联

关联的多重性：表示是由类产生的对象之间存在的**数量关系**

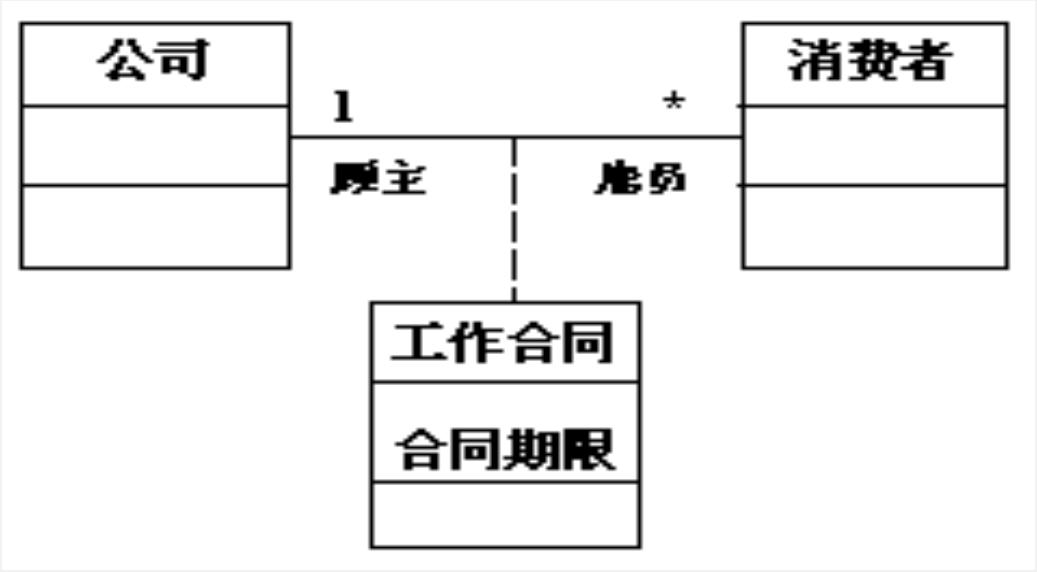
主要有以下几种：

- ✓ 1..1或1　　表示1个对象
- ✓ 0..1　　表示0到1个对象
- ✓ 0..*或*　　表示0到多个对象
- ✓ 1..*　　表示1到多个对象

识别关联

（1）关联类

关联有可能具有自己的属性或操作，对此需要引入一个关联类来进行记录。这时，关联关系中的每个连接与关联类的一个对象相联系，关联类通过一条虚线与关联连接。



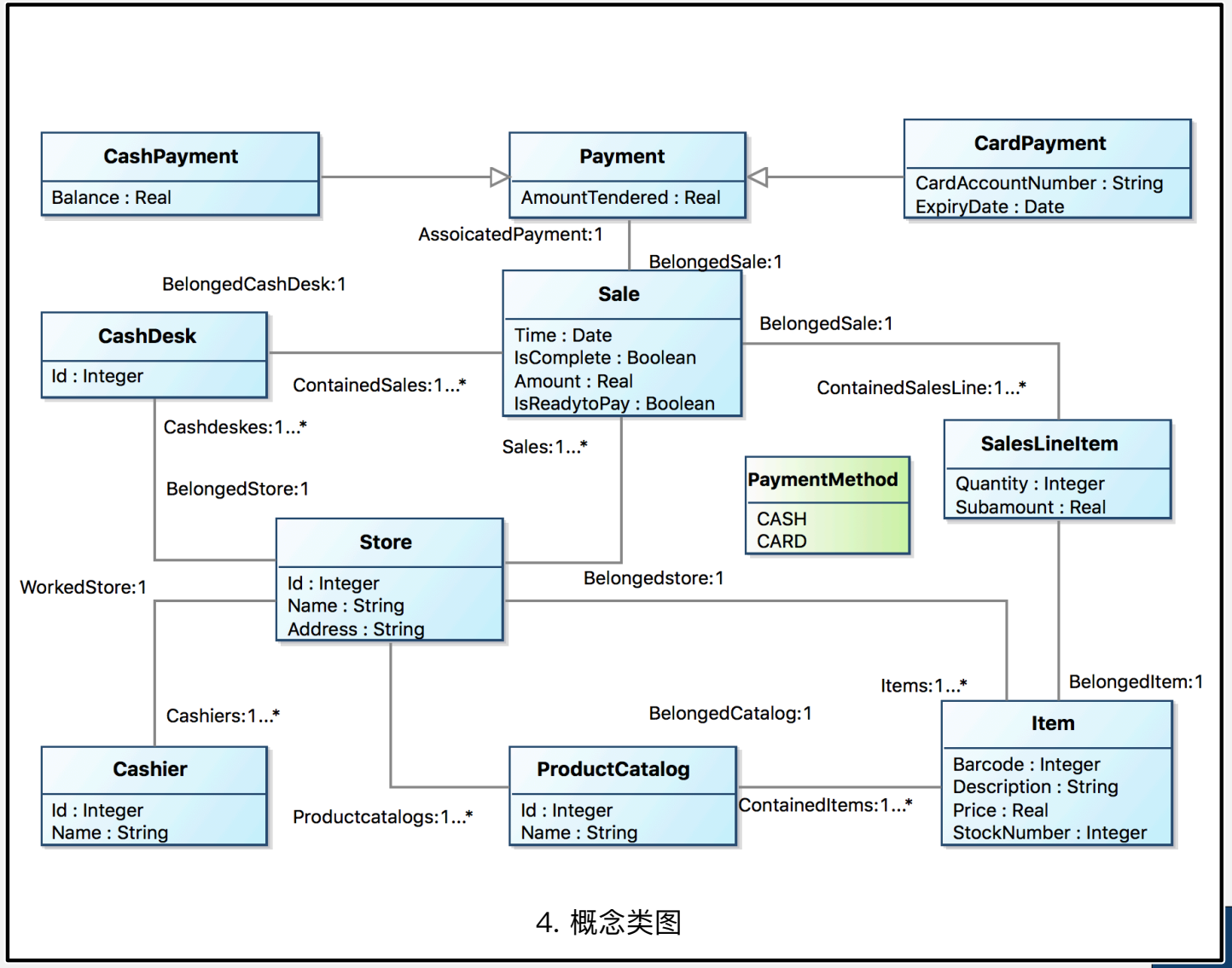
识别关联

(2) 聚合和组合

聚合是一种特殊形式的关联。聚合表示类之间的关系是整体与部分的关系。整体与部分没有相同的生命周期。整体不存在了，部分仍能存在。

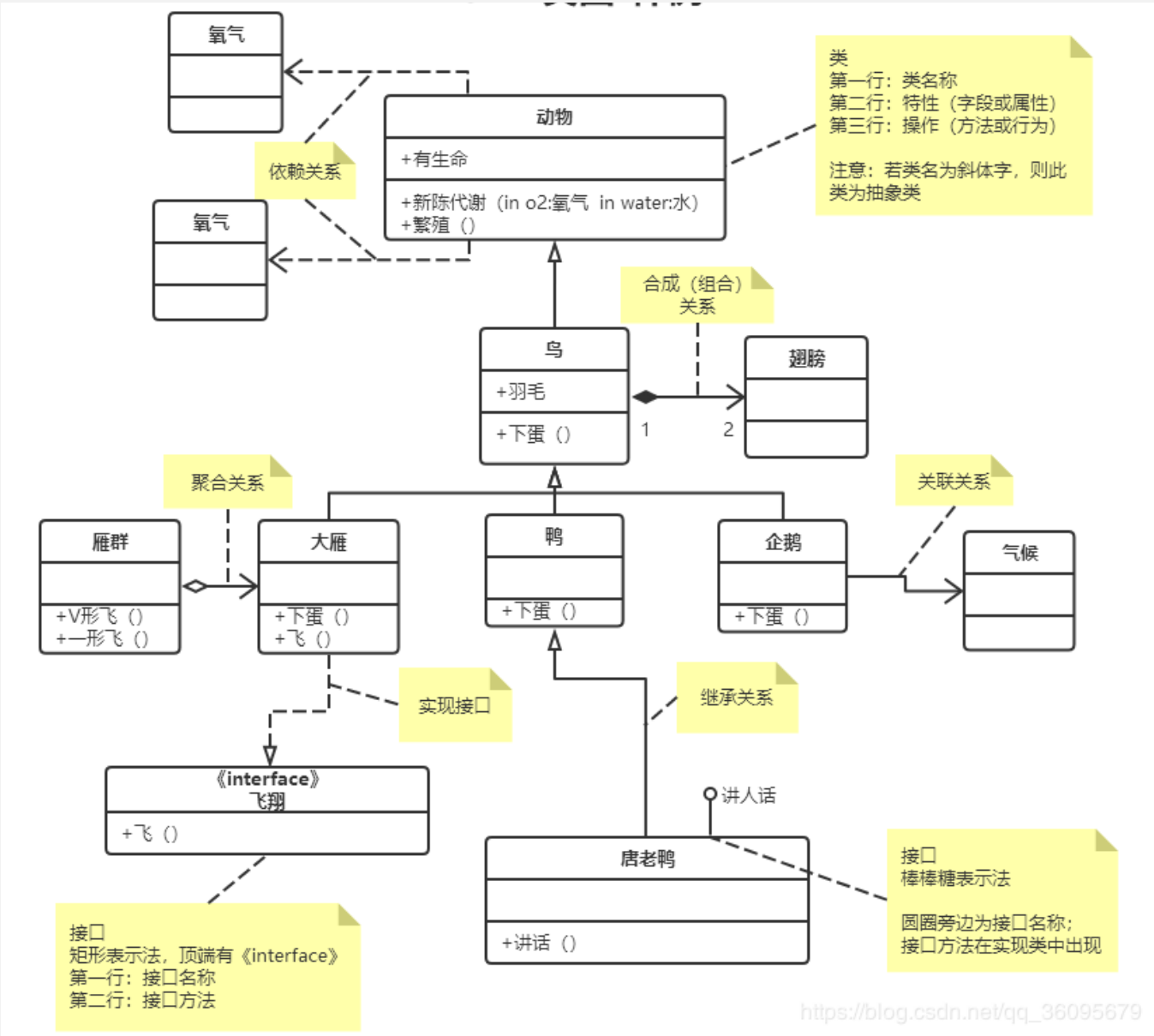
组合是一种特殊形式的关联。组合表示类之间的关系是整体与部分的关系。整体与部分具有相同的生命周期。整体不存在了，部分也不能存在。

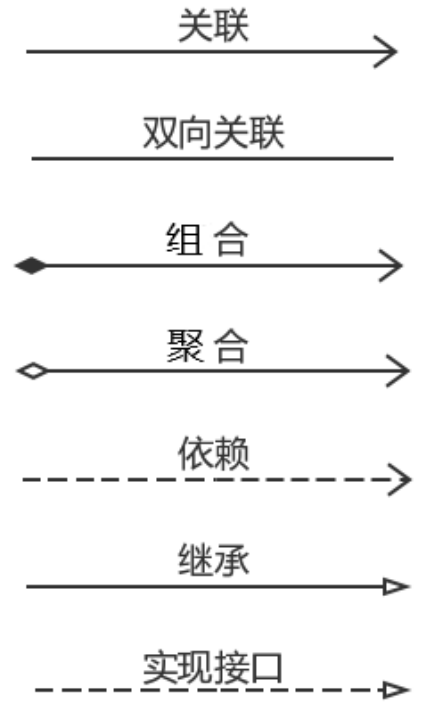
UML – 需求模型



4. 概念类图

小结





https://zhuanlan.zhihu.com/p/85960253

面向对象的需求模型

- 用例图
- 系统顺序图
  - 系统状态图（可选）
  - 系统活动图（可选）
- 系统操作合约
  - 一阶逻辑
  - 对象约束语言

用例模型
（行为）

- 概念类图
- 概念复杂约束

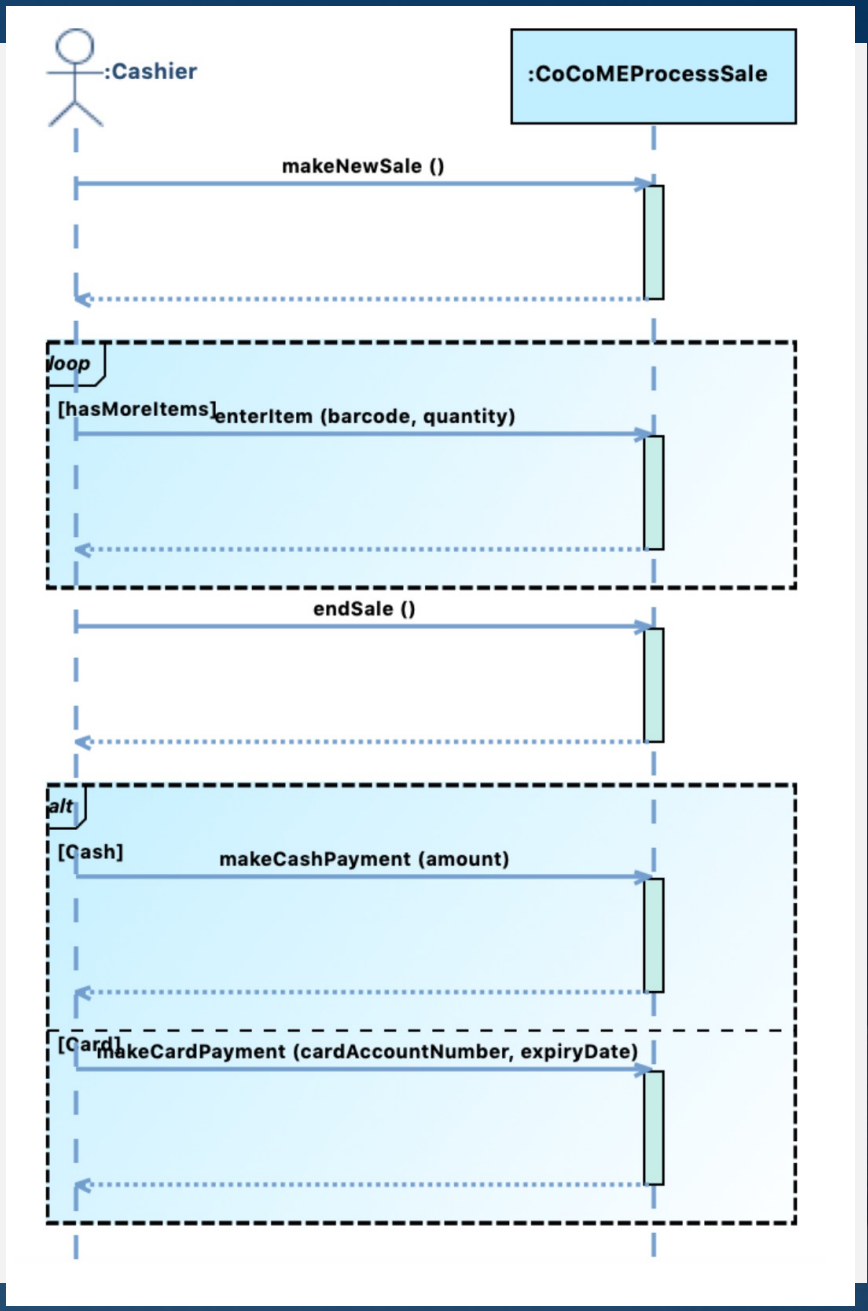
领域概念模型
（结构）

需求
模型

系统操作

- 系统顺序图（SSD）用来细化了用例的交互过程。而SSD展示了系统事件，即涉及系统的事件或者I/O消息。既然有输入的系统事件，那么就意味着系统具有用来处理这个系统事件的系统操作。

- 在这个系统顺序图中，用户在还有更多商品的时候循环向系统发送enterItem(itemID, quantity)事件，那么就意味着系统需要一个名为enterItem的系统操作来处理这个事件，该系统操作接受barcode和quantity作为输入，并将系统操作的处理结果通过消息对应的返回消息传递给用户。



系统操作合约

系统操作合约使用**前置和后置条件**的形式，描述领域模型里对象的详细状态变化，并作为系统操作的结果，从而对系统行为做了一个更为精确的描述。

- **操作签名**：操作的名称和参数。
- 交叉引用：涉及的用例
- **前置条件**：执行操作前，系统的满足的状态条件，包括输入参数。
- **后置条件**：完成操作后，系统的满足的状态条件，包括返回值和系统操作之间的临时变量。

系统操作合约

这里给出enterItem系统操作契约作为参考：

- **操作签名**：enterItem(barcode: Integer, quantity: Integer)
- **交叉引用**：处理销售
- **前置条件**：销售正在进行，barcode对应Item的实例存在，quantity小于等于目前商品库存
- **后置条件**：
  - ✓ 创建了SalesLineItem类的实例sli（创建了实例）
  - ✓ sli对象与当前的sale对象建立了链接（建立了链接）
  - ✓ sli对象的quantity属性值等于输入参数quantity（修改了属性）
  - ✓ sli实例与barcode对应商品Item建立链接Link（建立了链接）

系统操作合约——后置条件

- 后置条件描述了**系统内对象状态的变化**。这种变化包括创建实例、形成或者消除关联以及改变属性。

- 请注意，后置条件**不是在操作过程中执行的活动**，而是在**操作完成后系统状态的约束（描述什么东西变化了）**

- 概括来说，后置条件可以分为以下三种类型：
  - ✓ （**对象**）创建了或者删除了对象
  - ✓ （**连接**）对象之间连接的变化了（创建或者删除）
  - ✓ （**属性**）属性状态变化了

面向对象的基本操作

Primitive Operations

Table 1: Primitive Operations

	Primitive Operation	Return Type
Object	**findObject**(*ClassName*:*String*, *condition*:*String*)	Object
	findObjects(*ClassName*:*String*, *condition*:*String*)	Set(Object)
	createObject(*ClassName*:*String*)	Object
	addObject(*ClassName*:*String*, *ob*:*Class*)	Boolean
	releaseObject(*ClassName*:*String*, *ob*:*Class*)	Boolean
Attribute	**getAttribute**(*ob*:*Class*, *attriName*:*String*)	PrimeType
	setAttribute(*ob*:*Class*, *attriName*:*String*, *mathExp*:*String*)	Boolean
Link	**findLinkedObject**(*o*:*Class*, *assoName*:*String*, *condition*:*String*)	Object
	findLinkedObjects(*o*:*Class*, *assoName*:*String*, *condition*:*String*)	Set(Object)
	addLinkOnetoMany(*ob*:*Class*, *assoName*:*String*, *addOb*:*Class*)	Boolean
	addLinkOnetoOne(*ob*:*Class*, *assoName*:*String*, *addOb*:*Class*)	Boolean
	removeLinkOnetoMany(*ob*:*Class*, *assoName*:*String*, *removeOb*:*Class*)	Boolean
	removeLinkOnetoOne(*ob*:*Class*, *assoName*:*String*)	Boolean

早期UML图缺点

- **早期UML图不够精细，缺少对模型中关于对象附加约束的精确定义**
  - ✓ 用自然语言描述约束 -> 歧义
  - ✓ 形式化语言 -> 消除歧义，但是过于理论



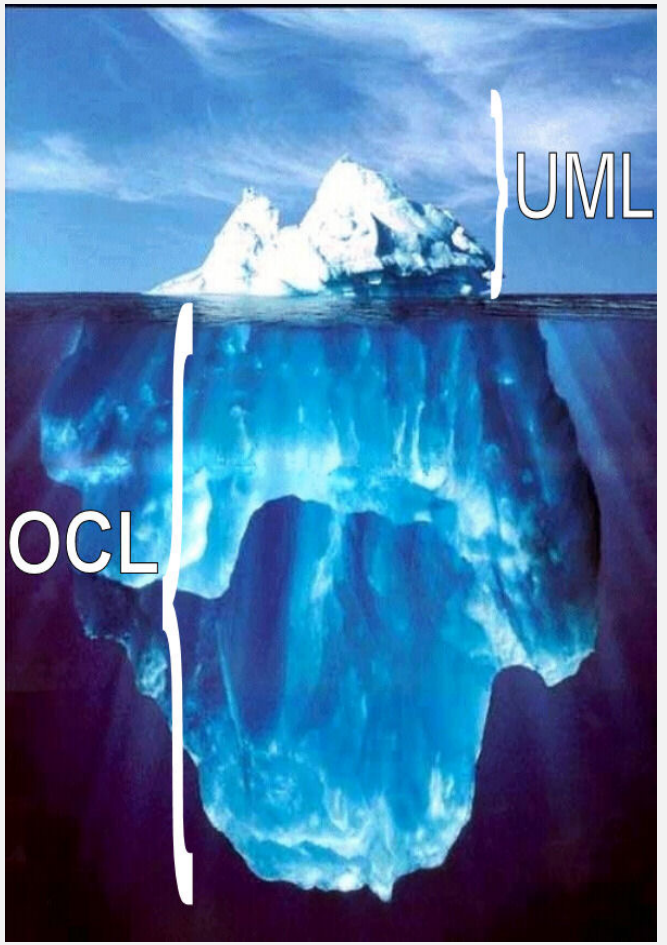
- **急需一种易读易写的形式化语言**

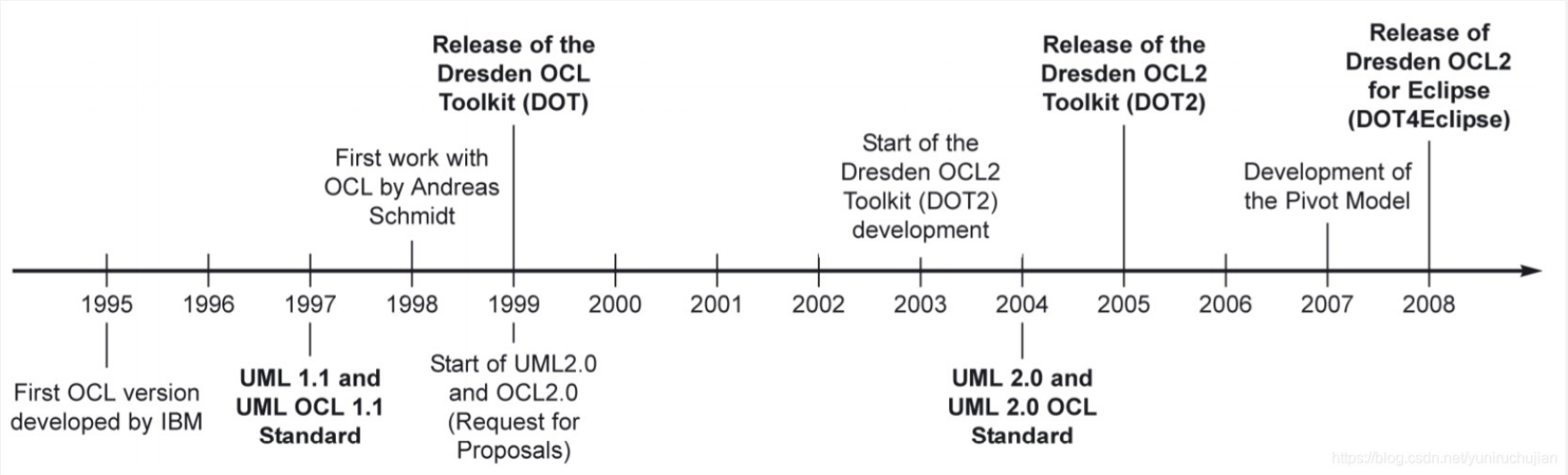
trying to be formal but easy to learn and use

- **能表达UML图中任何元素（对象或值）的约束，且能对系统中的元素进行快速查询**

OCL（对象约束语言）

- OCL (Object Constraint Language）克服UML图形表示的局限性，**给出建模元素的相关细节**
- OCL 是对象管理组（OMG）发行的一种标准的面向对象的分析和设计语言，于1997年集成到UML标准中（1.1版本）





OCL合约

© 2008 Carmen Avila & Yoonsik Cheon

OCL 2.0 QUICK REFERENCE

The University of Texas at El Paso

OCL Constructs

context

Specifies the context for OCL expressions.

```
context Account
```

inv

States a condition that must always be met by all instances of a context type.

```
context Account
  inv: balance >= 0
```

pre

States a condition that must be true at the moment when an operation starts its execution.

```
context Account::deposit(amt: Integer): void
  pre: amt > 0
```

post

States a condition that must be true at the moment when an operation ends its execution.

```
context Account::deposit(amt: Integer): void
  post: balance = balance@pre + amt
```

init

Specifies the initial value of an attribute or association role.

```
context Account::balance: Integer
  init: 0
```

derive

Specifies the value of a derived attribute or association role.

```
context Account::interest: Real
derive: balance * .03
```

body

Defines the result of a query operation.

```
context Account::getBalance(): Integer
  body: balance
```

def

Introduces a new attribute or query operation.

```
context Account
  def: getBalance(): Integer = balance
```

package

Specifies explicitly the package in which OCL expressions belong.

```
package BankSystem::Accounting
context Account
  inv: balance >= 0
endpackage
```

OCL Expressions

self

Denotes the contextual instance.

```
context Account
  inv: self.balance >= 0
```

result

In a postcondition, denotes the result of an operation.

```
context Account::getBalance(): Integer
  post: result = balance
```

@pre

In a postcondition, denotes the value of a property at the start of an operation.

```
context Account::deposit(amt: Integer): void
  post: balance = balance@pre + amt
```

Navigation

Navigation through attributes, association ends, association classes, and qualified associations.

```
context Account
  inv: self.balance >= 0 -- dot notation
  -- collection operator (->)
  inv: owners->size() > 0
  -- association class, TransInfo
  inv: transactions.TransInfo->forall(amount > 0)
  -- qualified association, owners
  inv: not owners['primary'].isOclUndefined()
```

if-then-else expression

Conditional expression with a condition and two expressions.

```
context Account::interestRate: Real
derive: if balance > 5000 then .03 else .02 endif
```

let-in expression

Expression with local variables.

```
context Account::canWithdraw(amt: Integer): boolean
  def: let newBalance: Integer = balance - amt
       in newBalance > minimumBalance
```

Messaging (^)

Indicates that communication has taken place.

```
context Account::deposit(s: Sequence(Integer)): void
  pre: s->forAll(amt: Integer | amt > 0)
  post: balance = balance@pre + s->sum()
  post: s->forAll(amt: Integer | self^deposit(amt))
```

OCL Standard Library

Basic Types

Type	Values	Operations
Boolean	false, true	or, and, xor, not, =, <>, implies
Integer	-10, 0, 10, …	=, <>, <, >, <=, >=, +, -, *, /, mod(), div(), abs(), max(), min(), round(), floor()
Real	-1.5, 3.14, …	
String	'Carmen'	=, <>, concat(), size(), toLower(), toUpper(), substring()

OclAny

Supertype of all UML and OCL types

Operation	Description

Date: February 2010

OMG

OBJECT MANAGEMENT GROUP

Object Constraint Language

Version 2.2

OMG Document Number: formal/2010-02-01
Standard document URL: http://www.omg.org/spec/OCL/2.2
Associated Schema Files*: http://www.omg.org/spec/OCL/20090501
http://www.omg.org/spec/OCL/20090501/EssentialOCL.emof
http://www.omg.org/spec/OCL/20090501/OCL.cmof

* original file(s): ptc/09-05-03 (XMI)

OCL合约

```
//Signature
Contract CoCoMEProcessSale::enterItem
    (barcode : String, quantity : Real) : Boolean {

//Definition Section
definition:
    //Find Object
    item:Item = Item.allInstance()->any(i:Item | i.Barcode = barcode)

//Pre-condition Section
precondition:
    currentSale.oclIsUndefined() = false and
    currentSale.IsComplete = false and
    item.oclIsUndefined() = false and
    item.StockNumber > 0

//Post-condition Section
```

OCL合约

```
postcondition:
    //Create an Object
    let sli:SalesLineItem in
    sli.oclIsNew() and
    //Add Links
    self.currentSaleLine = sli and
    sli.BelongedSale = currentSale and
    currentSale.ContainedSalesLine->includes(sli) and
    sli.BelongedItem = item and
    //Modify Attributes
    sli.Quantity = quantity and
    sli.Subamount = item.Price * quantity and
    item.StockNumber = item.StockNumber@pre - quantity and
    //Add an Object
    SalesLineItem.allInstance()->includes(sli) and
    result = true
}
```

面向对象的需求模型

- 用例图
- 系统顺序图
  - 系统状态图（可选）
  - 系统活动图（可选）
- 系统操作合约
  - 一阶逻辑
  - 对象约束语言

用例模型
（行为）

- 概念类图
- 概念复杂约束

领域概念模型
（结构）

需求
模型

面向对象的需求模型

- 用例图
- 系统顺序图
  - 系统状态图（可选）
  - 系统活动图（可选）
- 系统操作合约
  - 一阶逻辑
  - 对象约束语言

用例模型
（行为）

- 概念类图
- 概念复杂约束

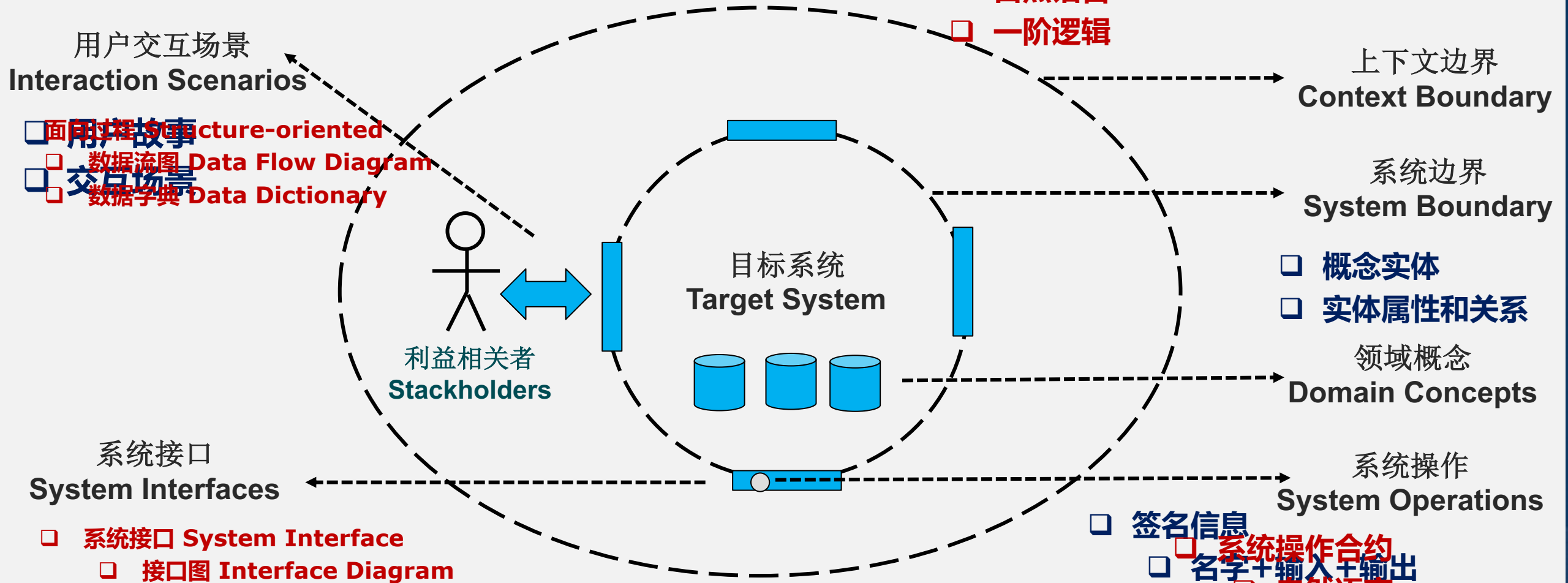
领域概念模型
（结构）

需求
模型

Part 4 需求精化与规约

- 需求精化与规约的任务
- 需求精化与规划的核心问题
- 面向对象方法的需求模型
- **面向过程方法的需求模型**

面向过程的需求建模



- 面向过程
  - 实体关系图 E-R Diagram
- 约束 Constraints
  - 自然语言
  - 一阶逻辑

- 用户故事
- 交互场景

- 面向过程 Structure-oriented
  - 数据流图 Data Flow Diagram
  - 数据字典 Data Dictionary

- 概念实体
- 实体属性和关系

- 系统接口 System Interface
  - 接口图 Interface Diagram
  - 接口定义语言 Interface Definition Language (IDL)

- 签名信息
  - 名字+输入+输出
- 状态约束
  - 前置条件 + 后置条件
  - 不变条件

- 系统操作合约
  - 自然语言
  - 一阶逻辑 FOL

面向过程的需求模型

- 数据流图 Data Flow Diagram
- 数据字典 Data Dictionary

- 接口图 Interface Diagram
- 接口定义语言 Interface Definition Language (IDL)
- 系统操作合约
  - 自然语言
  - 一阶逻辑 FOL

数据流模型
（行为）

- 实体关系图 E-R Diagram

实体关系模型
（结构）

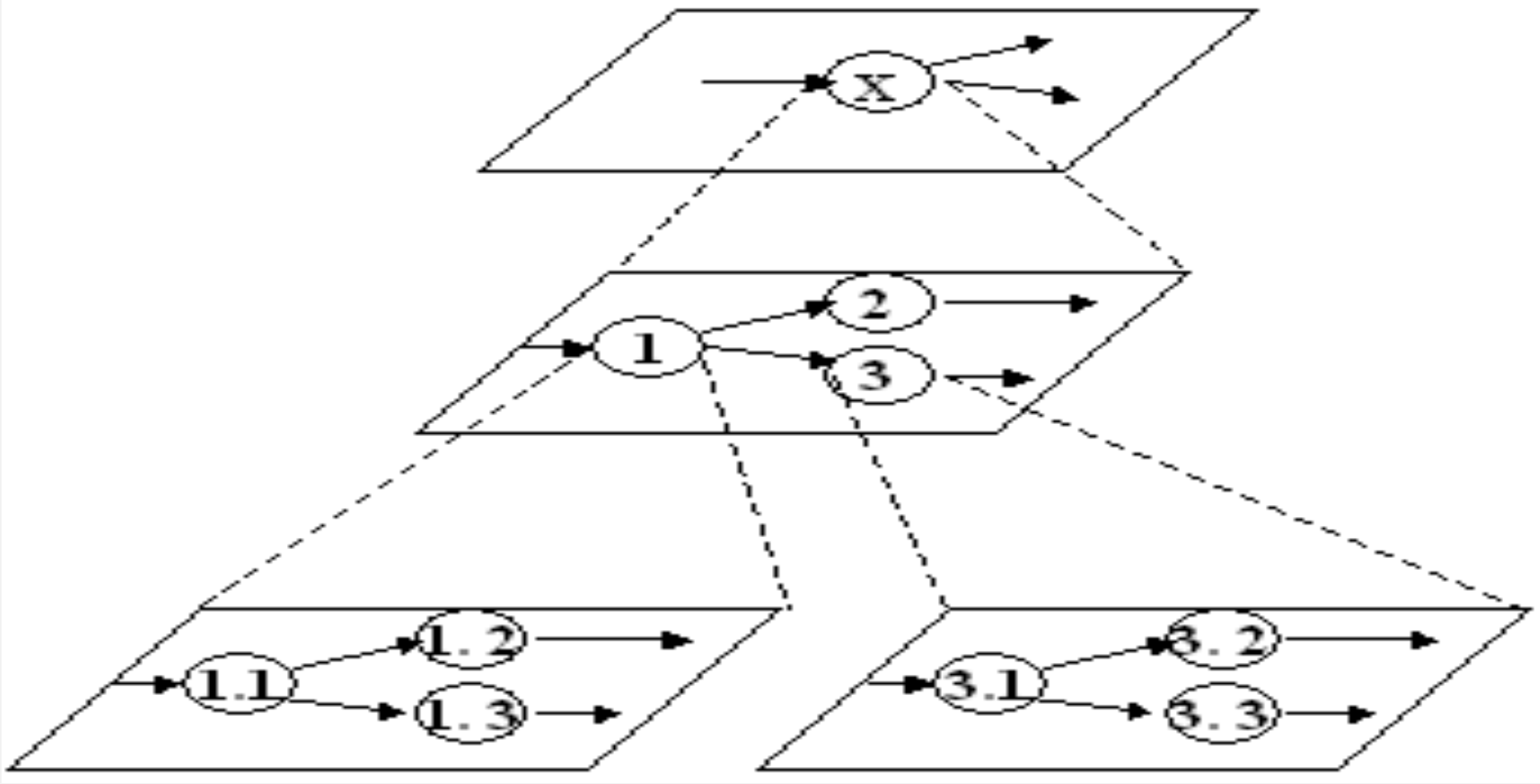
面向过程的需求模型

结构化分析思想

- **结构化**是把软件系统的**功能**当作**一个大模块**，根据分析与设计的不同要求，进行**模块分解或者组合。**
- **基本思路**就是把整个系统开发过程**分成若干阶段**，每个阶段进行若干活动，**每项活动应用一系列标准、规范、方法和技术**，完成一个或多个任务，**形成符合需求的软件产品**（成果）。

结构化分析方法概述

- 采用“自顶向下，逐步求精”的方式，X系统被分解成3个子系统：



结构化分析方法指导性原则

- 在开始建立分析模型之前**先理解问题**，而不应急于求成，甚至在问题未被很好地理解之前，就产生了一个解决错误问题的软件
- 开发模型，使用户能够了解将**如何进行人机交互**
- 记录每个需求的**起源和原因**，这样能有效地保证需求的**可追踪性和可回溯**性
- 使用**多个需求分析视图**，建立**数据、功能和行为模型**
- 给需求**赋予优先级**，优先开发重要的功能，提高开发生产效率
- 努力**避免含糊性**

结构化方法的局限性

- 不提供对**非功能需求**的有效理解和建模；
- 不提供对用户选择合适**方法的指导**，也没有对方法适用的特殊环境的忠告；
- 往往产生大量文档，系统需求的要素被隐藏在一大堆具体细节的描述中；
- 产生的模型不注意细节，**用户觉得难以理解**，因而很难验证模型的真实性。

结构化分析方法概述

结构化分析时一般使用以下工具来描述系统。

（1）数据流图（Data Flow Diagram，DFD）

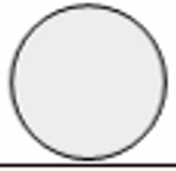
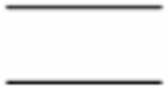
（2）数据字典（Data Dictionary，DD）

（3）结构化语言

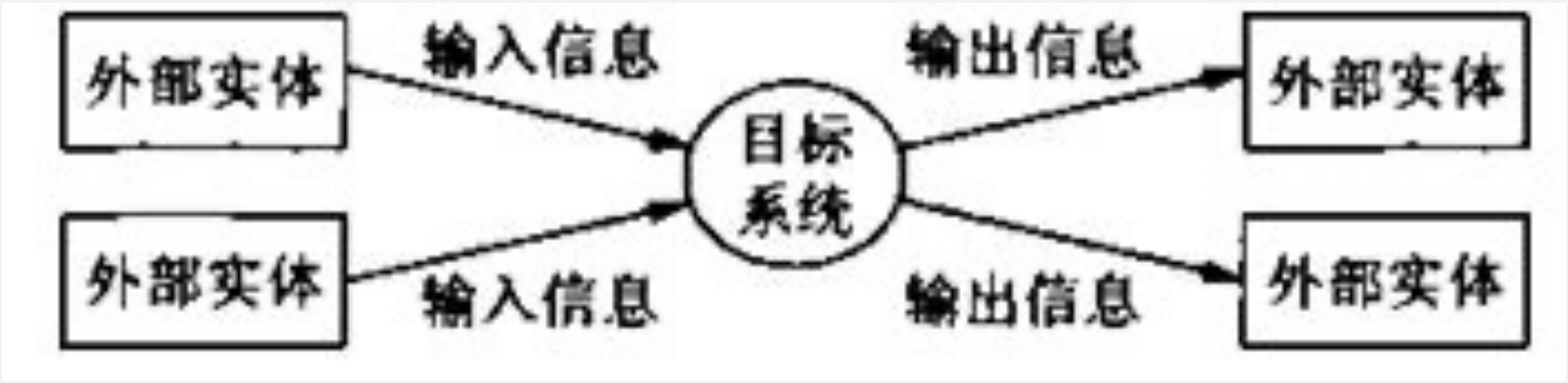
（4）判定表

（5）判定树

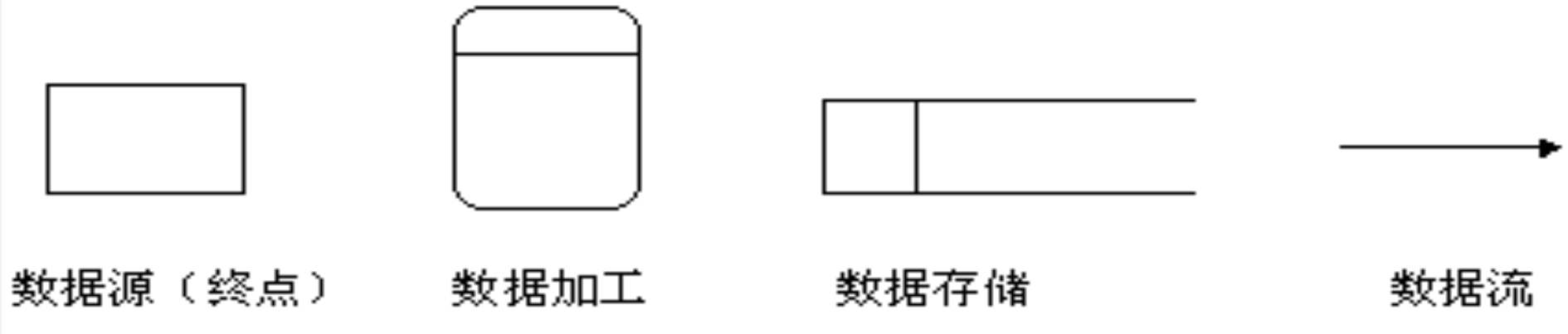
数据流图（DFD）

	Yourdon	Gane
外部实体		
数据流		
数据变换		
数据存储		

环境图（0级DFD）



数据流图的基本成份



1．数据流

2．加工

3．数据存储

4．源点和终点

数据流图的绘制

数据流图的绘制步骤

(1) 找出系统的**输入和输出**

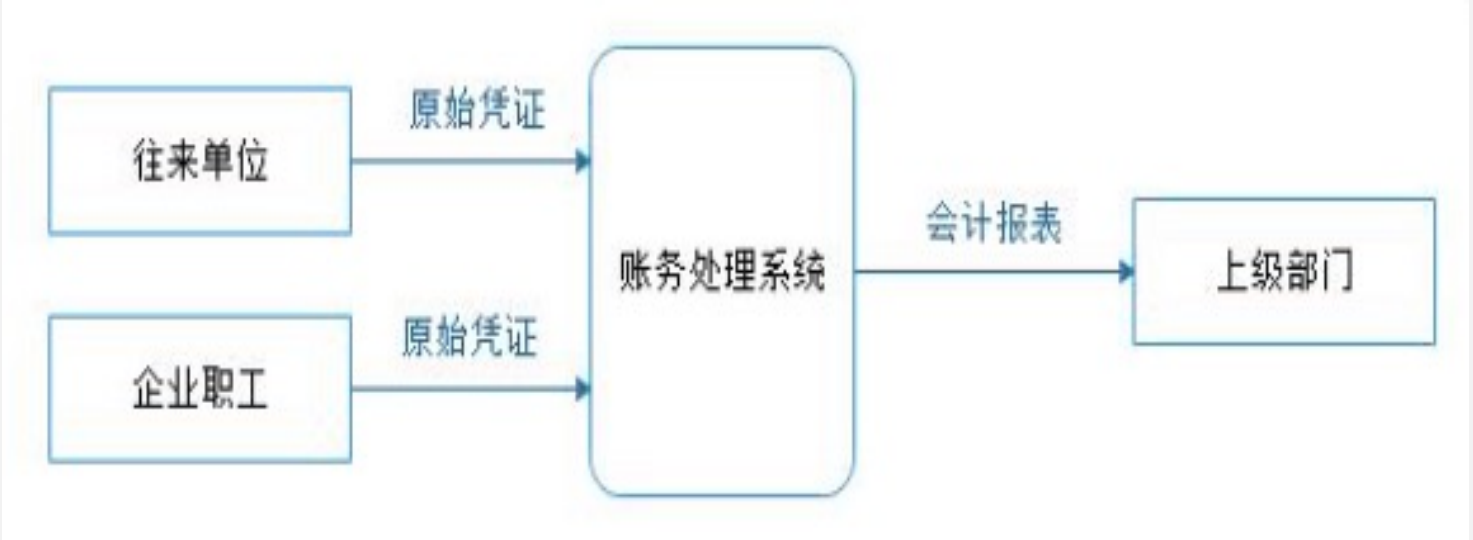
(2) 画数据流图的**内部**

(3) 为每一个**数据流命名**

(4) 为**加工命名**

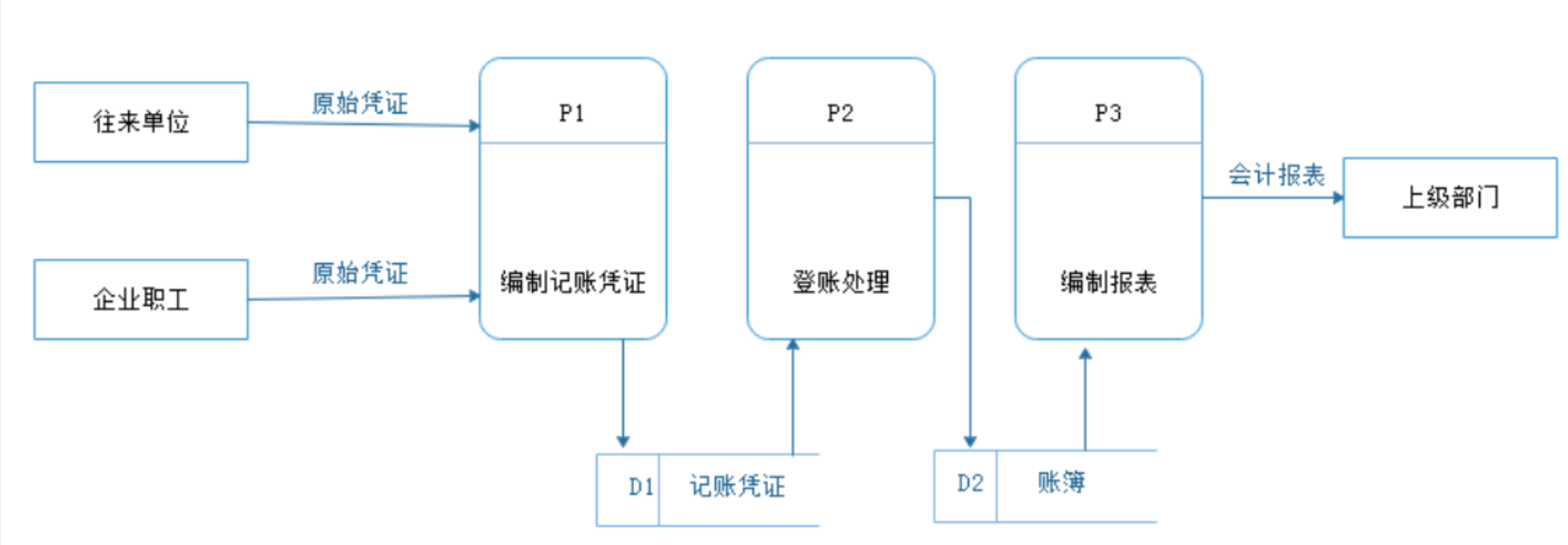
数据流图

- 财务处理系统数据流图**顶图**



数据流图

❑ 会计账务处理系统的数据流图

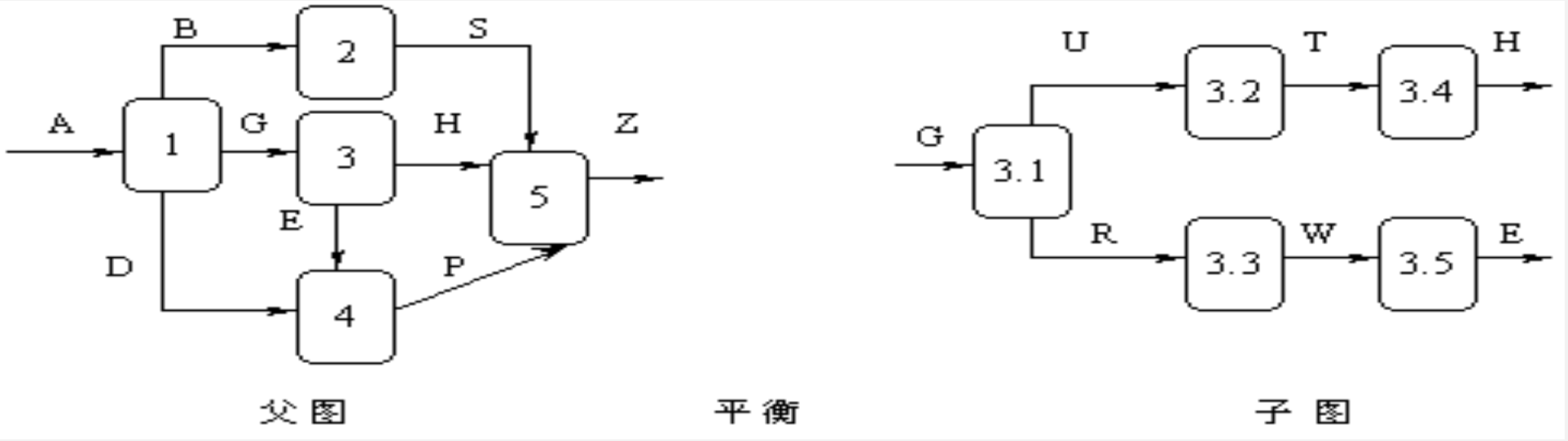


分层数据流图

- **顶层数据流图**描述了整个软件系统的作用范围，对系统的总体功能、输入和输出进行了抽象，**反映了系统和环境的关系**。
- **中间层次的数据流图**是通过**分解高层数据流和加工**得到的。
- **低层次数据流图**通常有很多中间层，**甚至7~9层**。这种分解可以不断重复，直到新的数据流图中**每个数据加工的功能明确、相关的数据流**被严格定义为止。

分层原则

- 父图与子图的关系
- **平衡规则**
- 分层程度



数据流图的特征与用途

用途

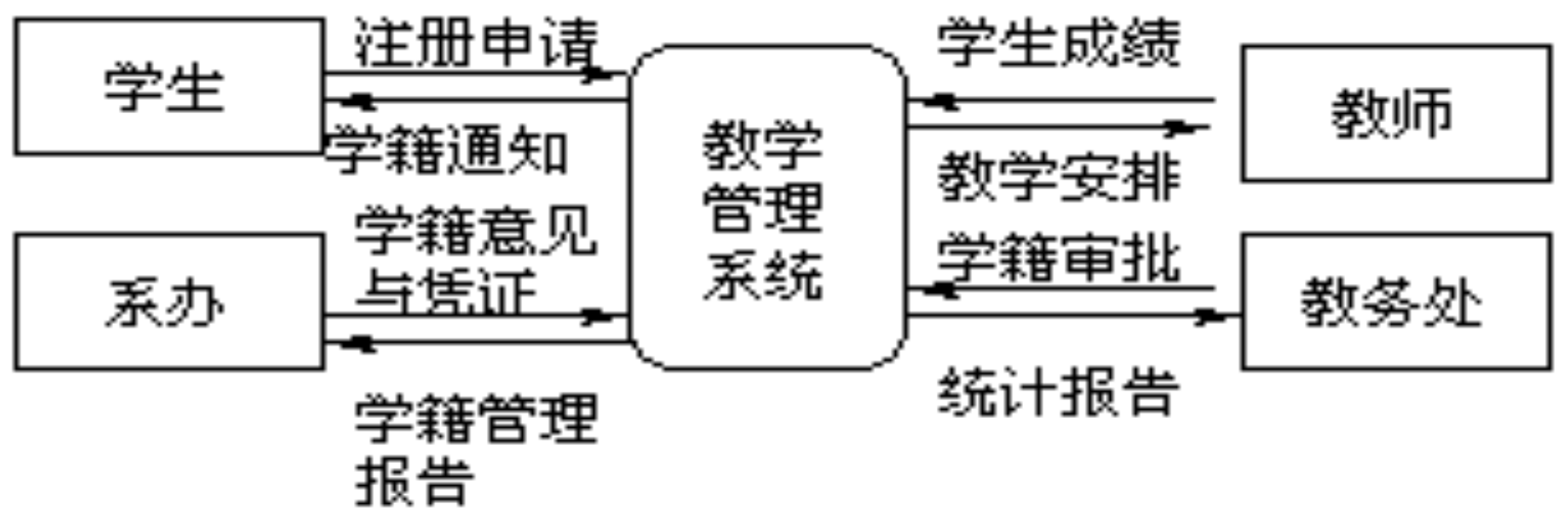
- 可以依靠数据流图来实现**从用户需求到系统需求**的过渡
- 数据流图可以按照功能将**系统分解为许多子系统**
- 用做开发者与用户之间的信息交流工具

特点

- 抽象性
- 概括性

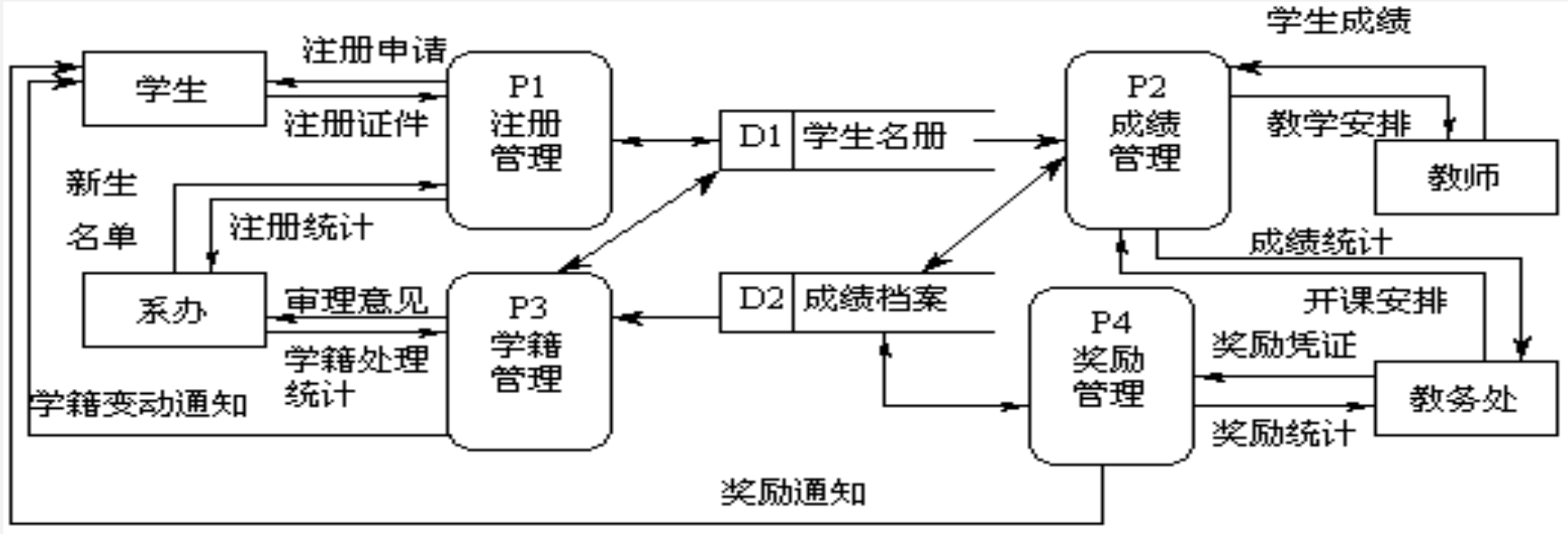
分层数据流图举例

- 教学管理数据流图**顶图**



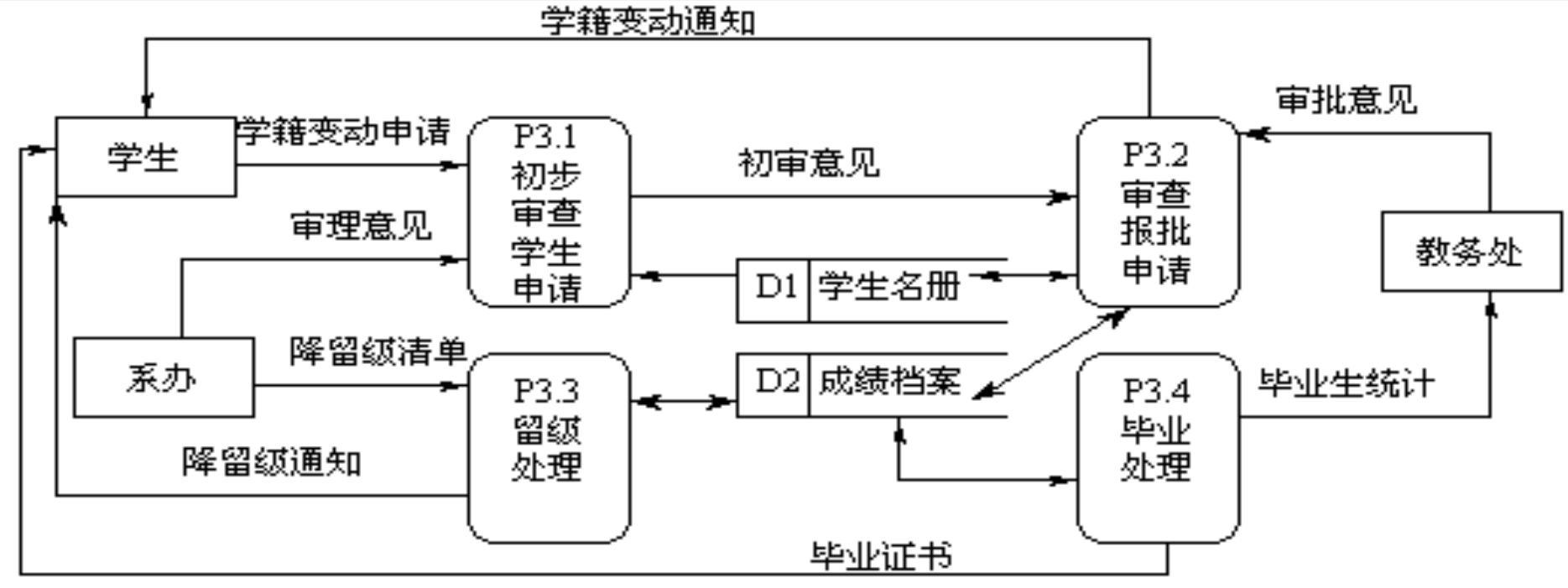
分层数据流图举例

- 教学管理数据流图**中图（模块）**



分层数据流图举例

- 学籍管理数据流图**底图**



数据字典的定义与用途

定义：数据字典是关于数据信息的集合，是数据流程图中所有元素严格定义的场所。

用途

- **数据流程图和数据字典**一起构成了**系统的逻辑模型**
- 有助于改进不同的开发人员或不同的开发小组之间的通信
- 是存储文件或**数据库设计的基础**
- 在实施阶段，还可参照**数据字典描述数据**

数据字典的种类

- 数据流
- 数据存储
- 数据项
- 加工

数据字典的定义方法

数据字典的定义式可使用的符号：

符号	含 义
=	由….组成（定义为….）
+	和（顺序关系的连接）例，x = a+b 表示X由a与b组成
{}	重复 例，x ={a+b} 表示X是由零次或多次重复的a与b组成
[/]	可选择（选一个）例，x =[a/b] 表示X由a或b中选择一个
( )	可选（也可不选）例，x =（a） 表示a是任选的，可在X中出现0次或1次
**	注释

数据字典 - 数据流

数据流名：注册申请
简述：每学期开学需要学生注册登记
别名：无
组成：注册申请 = 学号+姓名+入学日期+注册日期
数据量：2000次/开学一周
峰值：第一周每天下午1:00到5:00点有300次
注释：到2022年还将增加到3000人

数据字典 - 数据项

数据项： 学号
简述： 每个在校学生的学生编号
别名： 无
组成： 学号 = XX + XX + XXX
年级 专业 序号
值类型： 7位数字
取值范围：
注释：到2022年还将增加到3000人

数据字典 – 数据存储

文件名：成绩档案
简述：包括所有在册学生各门课程的考试成绩和学分信息
别名：无
组成：成绩档案 = 学号+姓名+课程名称+考试成绩+学分
数据量：2000*6次，考试结束一周内
峰值：学期最后一周每天下午1:00到5:00点有2000*6次
注释：到2022年还将增加到3000人

结构化语言 – 描述加工

结构化语言

例：计算某学生的考试成绩优秀率和中以下的比率

若成绩优秀比率大于70%

　若表现优良

则获得一等奖学金

　否则获得二等奖学金

若成绩优秀比率小于70%且大于50%

　若表现优良

则获得二等奖学金

　否则获得三等奖学金

判定表

判定表的构造步骤：

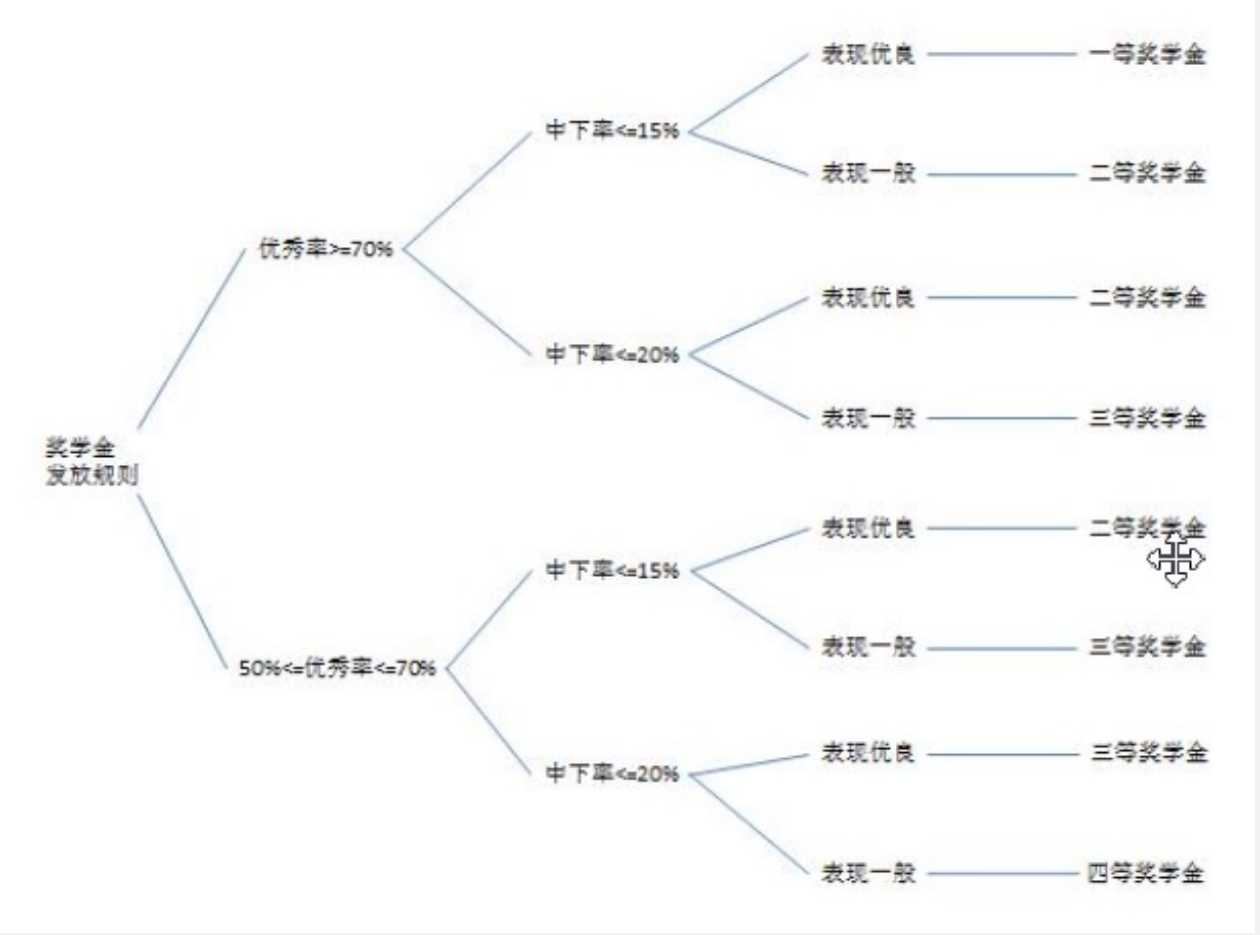
- **列出所有基本条件，填写判定表的左上限。**在本例中，奖学金的发放依据三个条件：成绩优秀比率、成绩中下比率、表现。
- **列出所有的基本操作，填写判定表的左下限。**在本例中，奖学金的发放分为4个等级。

条件	成绩比率 优>70%	Y	Y	Y	Y	N	N	N	N
	成绩比率 优>50%	-	-	-	-	Y	Y	Y	Y
	成绩比率 中<15%	Y	Y	N	N	Y	Y	N	N
	成绩比率 中<20%	-	-	Y	Y	-	-	Y	Y
	表现 优良	Y	N	Y	N	Y	N	Y	N
	表现 一般	N	Y	N	Y	N	Y	N	Y
操作	一等奖学金	√							
	二等奖学金		√	√		√			
	三等奖学金				√		√	√	
	四等奖学金								√

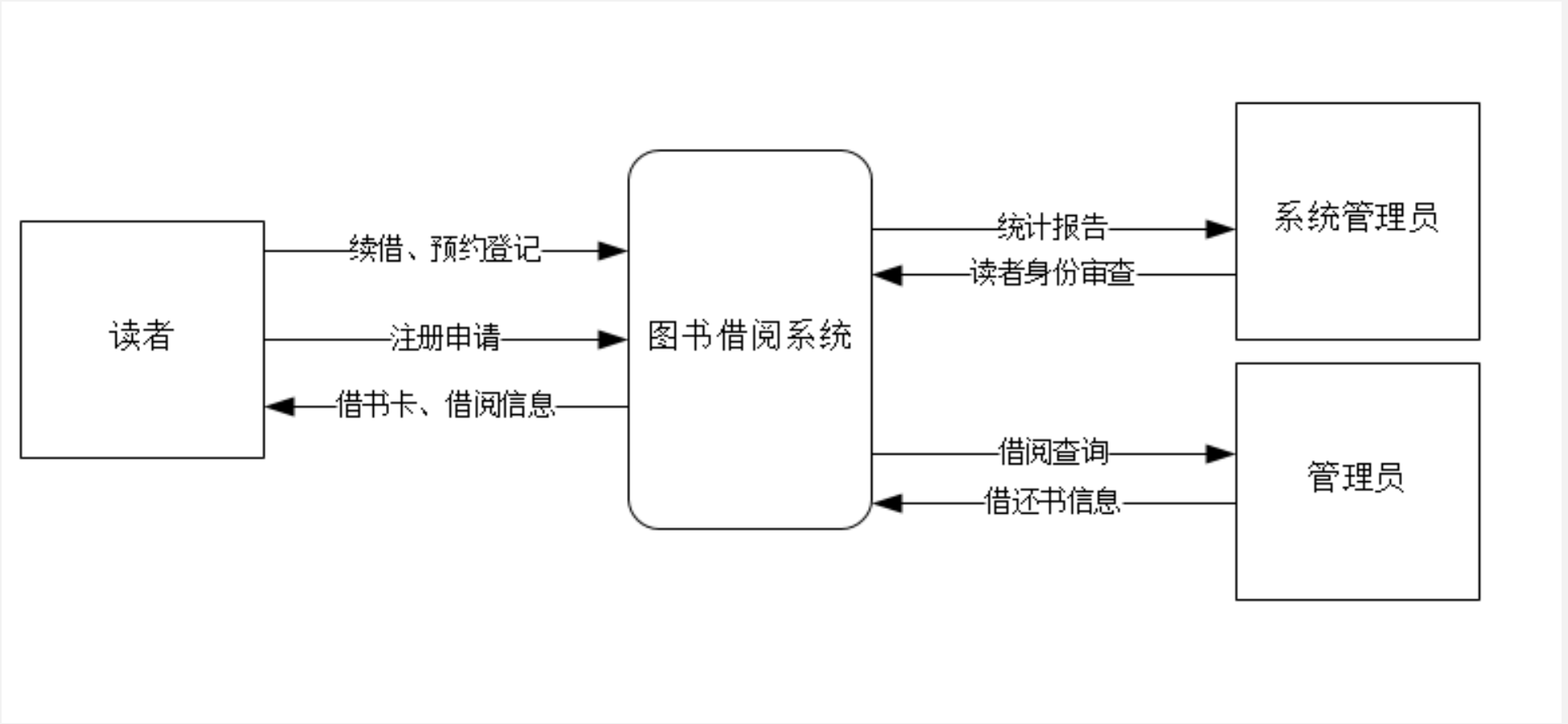
- **计算所有可能的条件组合**，确定规则个数，填**写判定表的右上限。**
- 将每一组合指定的操作，**添入右下限相应的位置。**

判定树

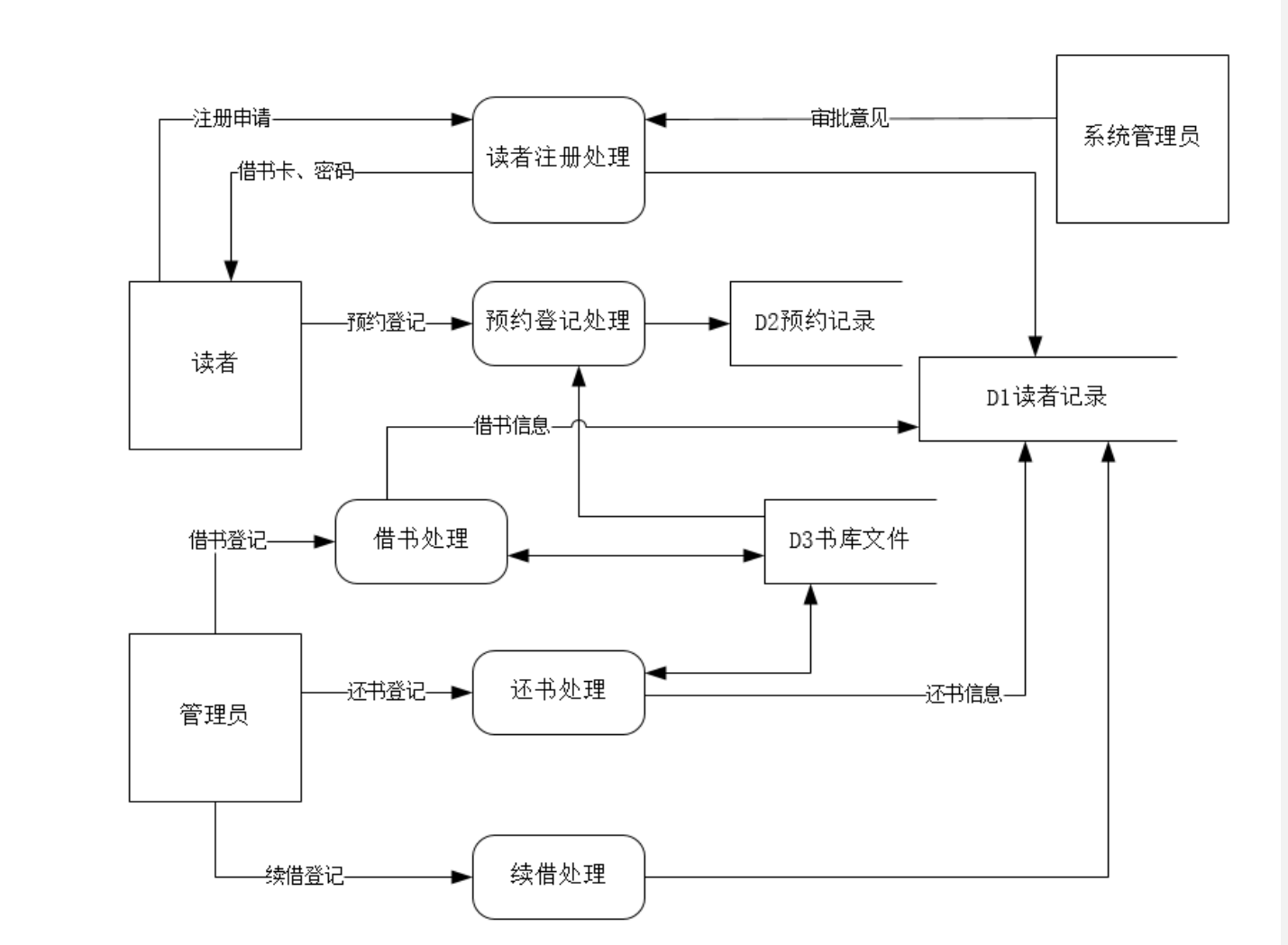
奖学金发放判定树



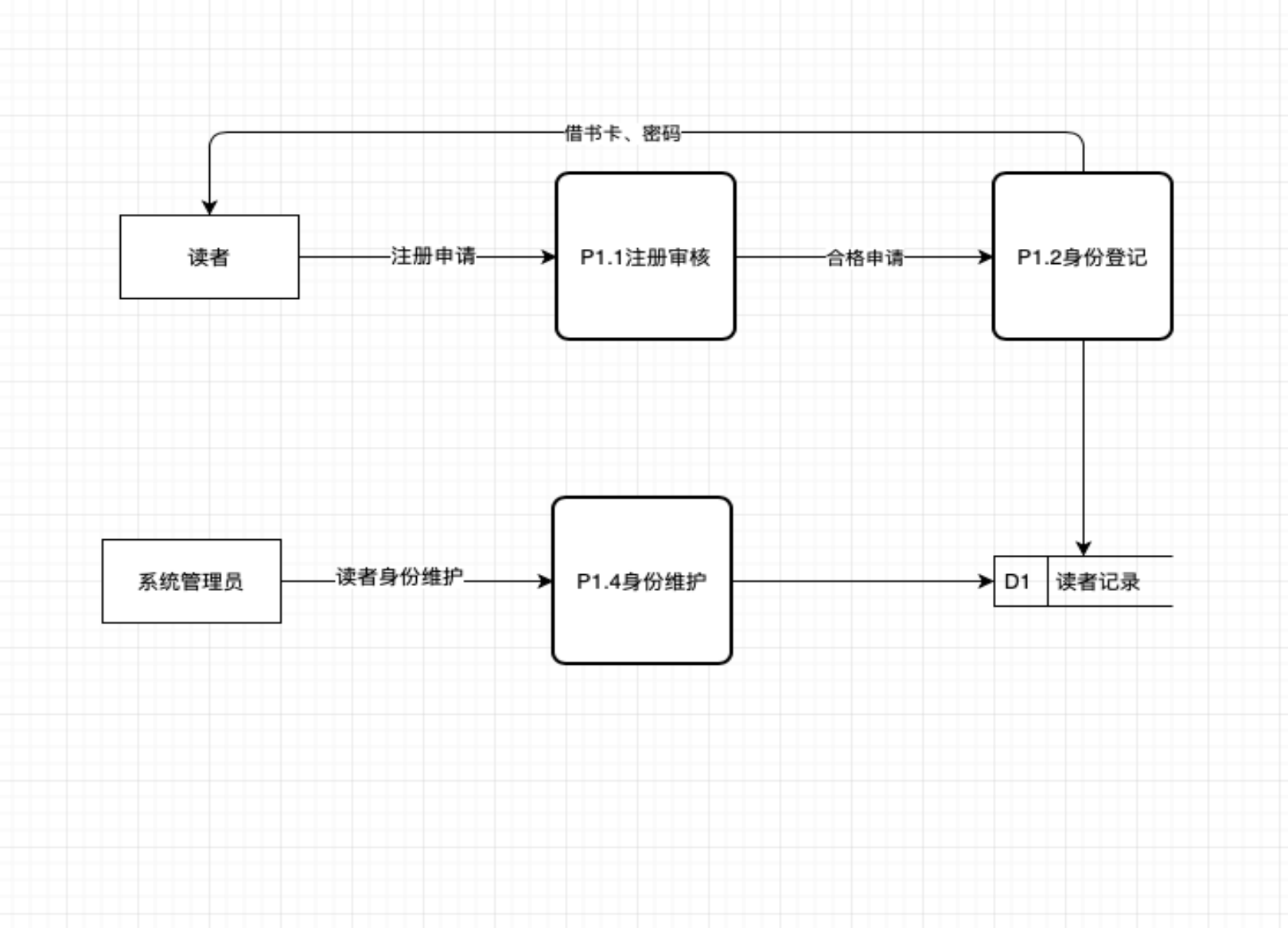
图书借阅管理系统顶层数据流图



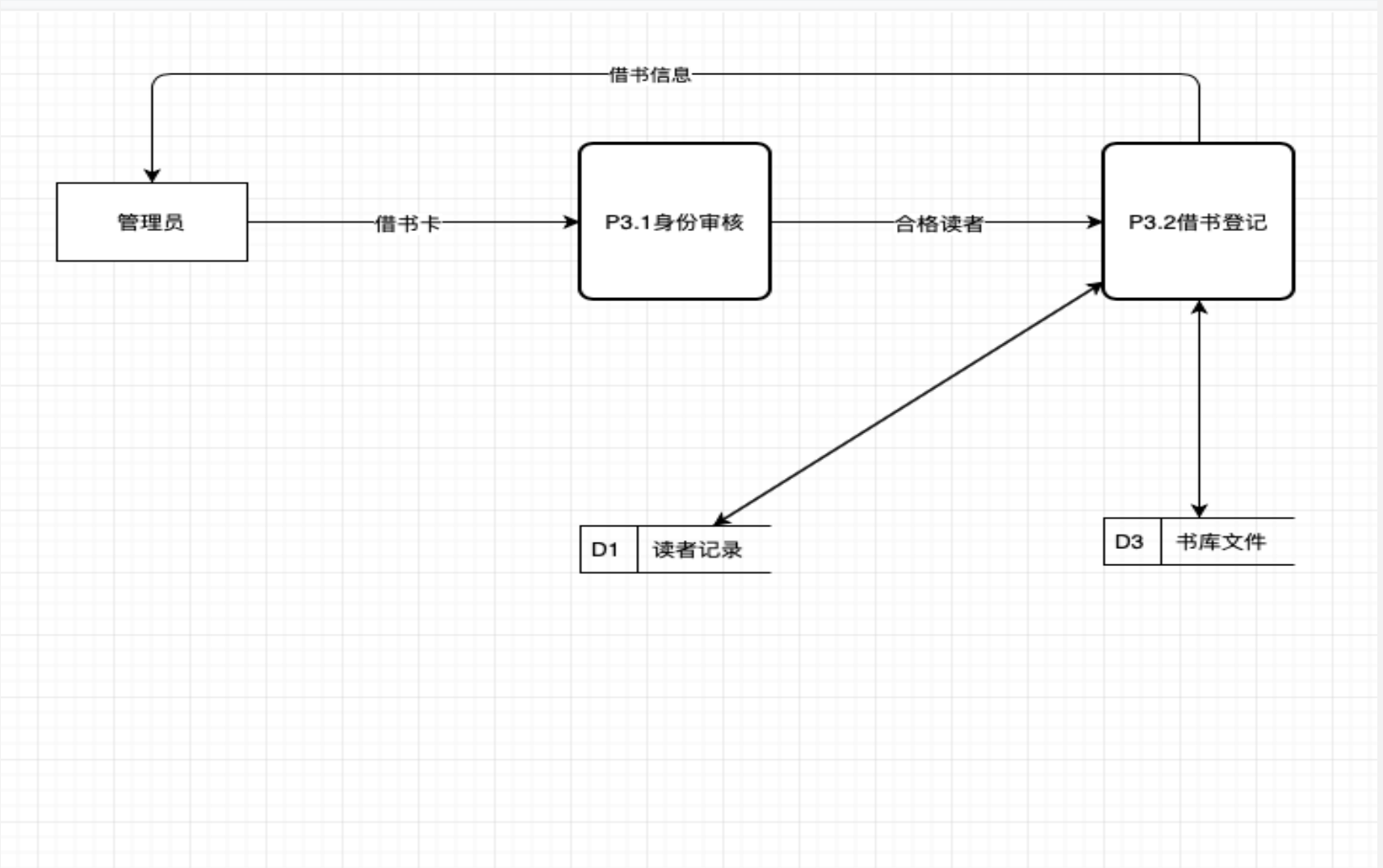
图书借阅管理系统中层数据流图



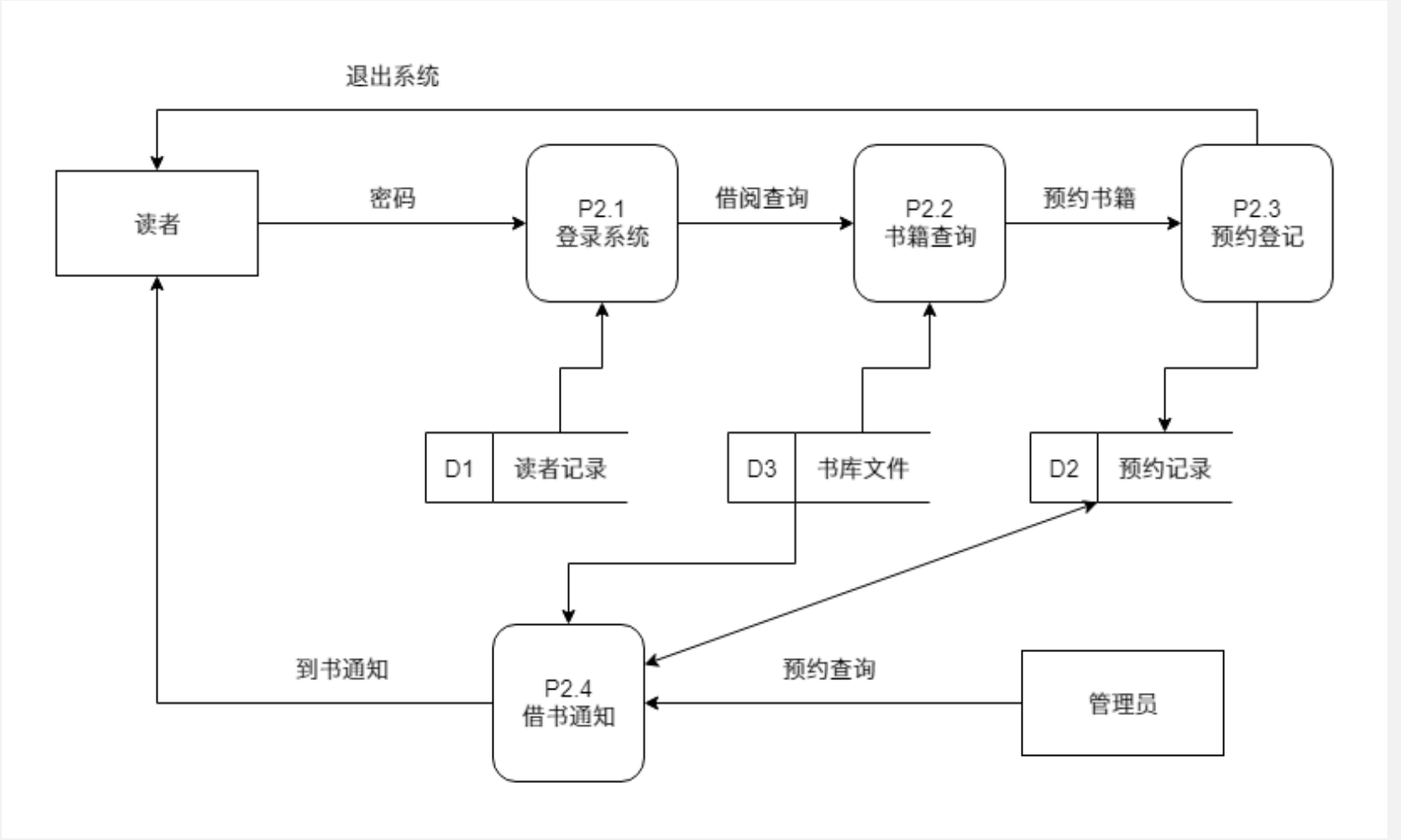
注册处理数据流图底图


借书卡、密码
读者
注册申请
P1.1注册审核
合格申请
P1.2身份登记
系统管理员
读者身份维护
P1.4身份维护
D1
读者记录

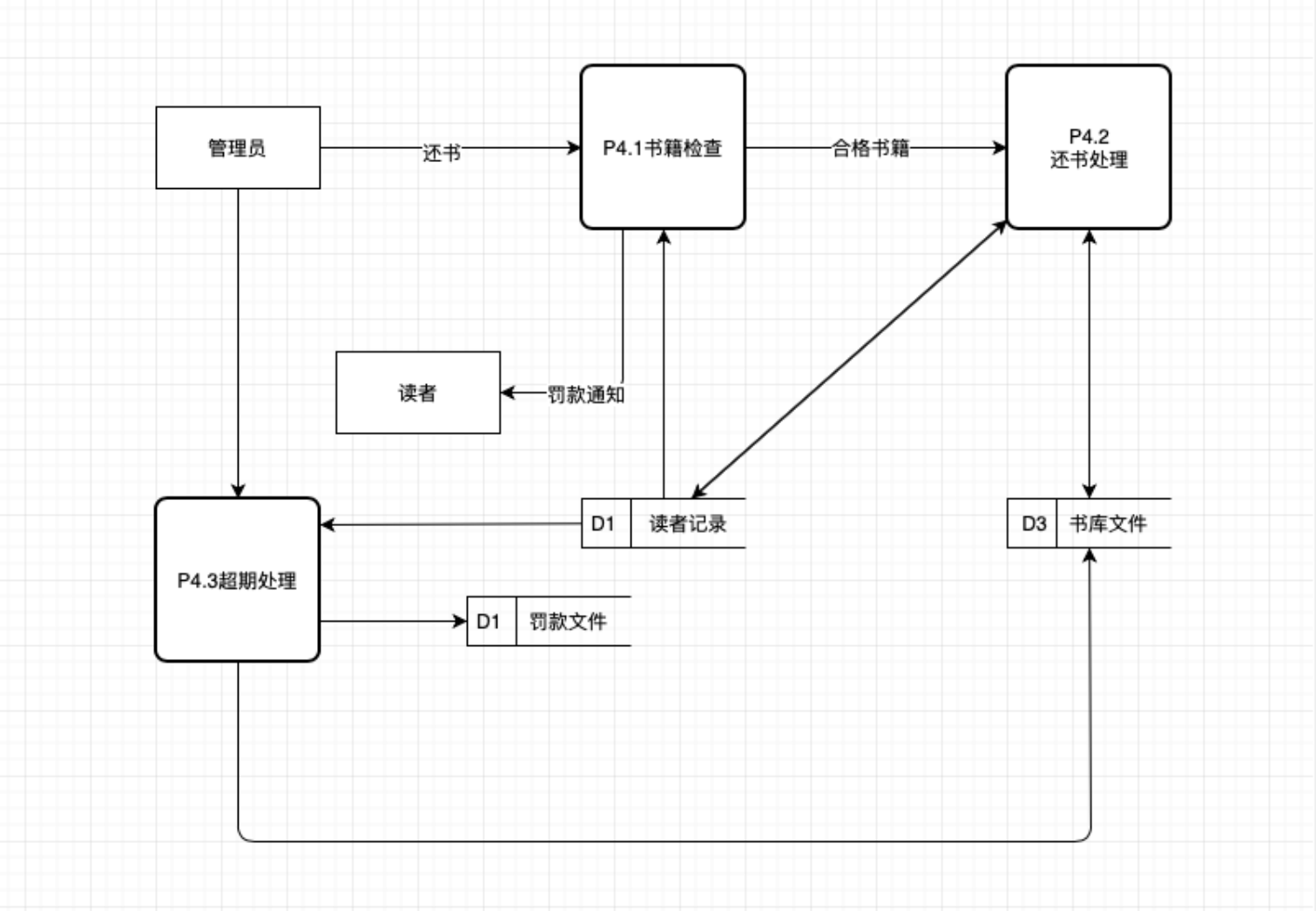
借书处理数据流图底图



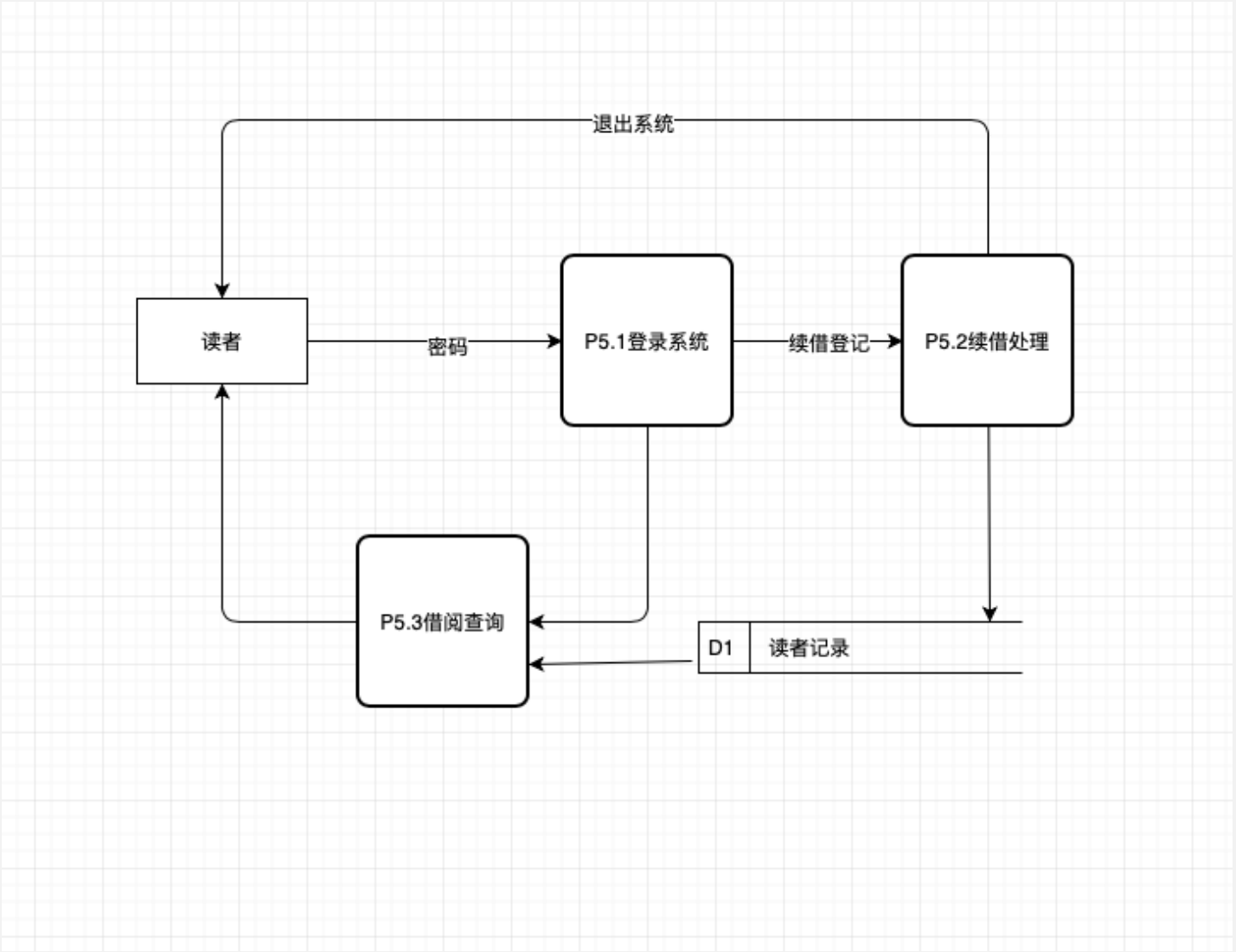
预约处理数据流图底图


退出系统
读者
密码
P2.1
登录系统
借阅查询
P2.2
书籍查询
预约书籍
P2.3
预约登记
D1
读者记录
D3
书库文件
D2
预约记录
到书通知
P2.4
借书通知
预约查询
管理员

还书处理数据流图底图


管理员
还书
P4.1书籍检查
合格书籍
P4.2
还书处理
读者
罚款通知
P4.3超期处理
D1
读者记录
D3
书库文件
D1
罚款文件

续借处理数据流图底图



基于模型的系统需求的建模



- 面向过程
  - 实体关系图 E-R Diagram
- 面向对象
  - 概念类图 Conceptual Class Diagram
- 约束 Constraints
  - 一阶逻辑
  - 对象约束语言

用户交互场景
Interaction Scenarios

- 用户故事
- 交互场景

面向过程 Structure-oriented
- 数据流图 Data Flow Diagram
- 数据字典 Data Dictionary

面向对象 Object-oriented
- 用例图 Use Case Diagram
- 系统顺序图 System Sequence Diagram

上下文边界
Context Boundary

系统边界
System Boundary

- 概念实体
- 实体属性和关系

领域概念
Domain Concepts

目标系统
Target System

利益相关者
Stackholders

系统接口
System Interfaces

- 系统接口 System Interface
  - 接口图 Interface Diagram
  - 接口定义语言 Interface Definition Language (IDL)

系统操作
System Operations

- 签名信息
  - 名字+输入+输出
- 状态约束
  - 前置条件 + 后置条件
  - 不变条件

- 系统操作合约
- 一阶逻辑 FOL
- 对象约束语言 OCL

北京航空航天大學
BEIHANG UNIVERSITY

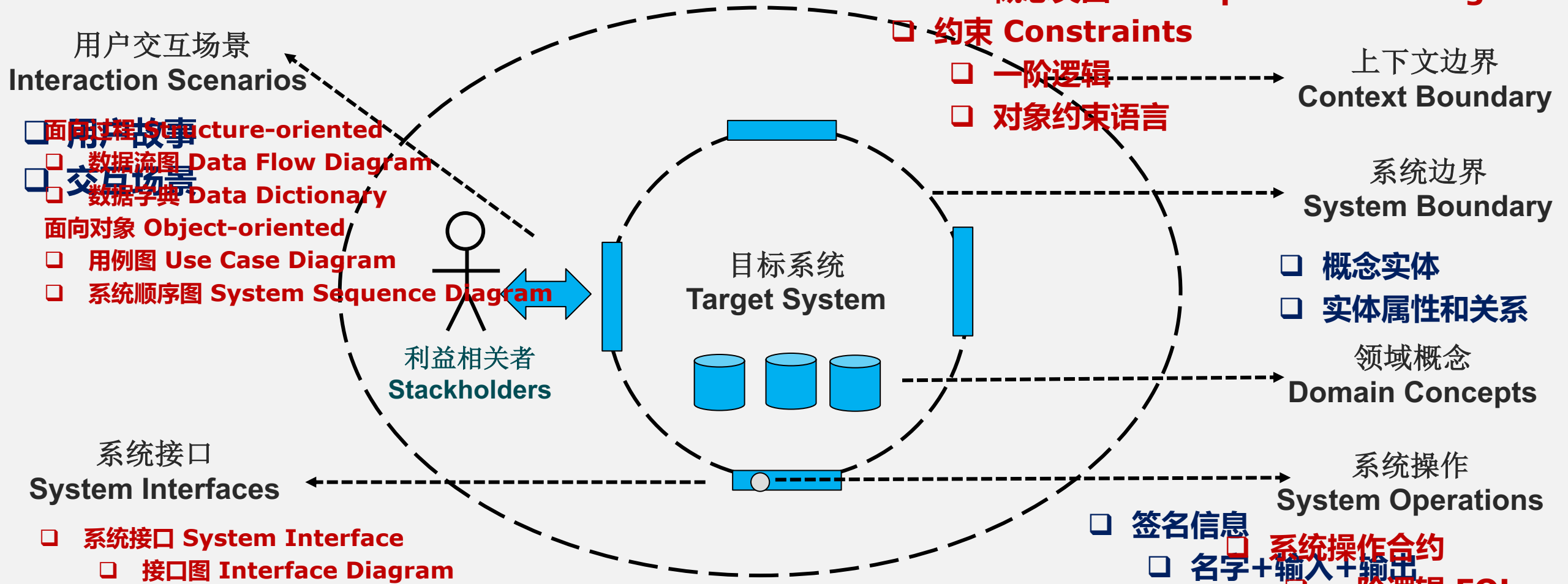


Part 5

Requirements Validation

需求确认与验证

回顾：基于模型的系统需求的建模



- 用户故事
- 交互场景

面向过程 Structure-oriented
- 数据流图 Data Flow Diagram
- 数据字典 Data Dictionary

面向对象 Object-oriented
- 用例图 Use Case Diagram
- 系统顺序图 System Sequence Diagram

- 面向过程
  - 实体关系图 E-R Diagram
- 面向对象
  - 概念类图 Conceptual Class Diagram

约束 Constraints
- 一阶逻辑
- 对象约束语言

- 概念实体
- 实体属性和关系

- 系统接口 System Interface
  - 接口图 Interface Diagram
  - 接口定义语言 Interface Definition Language (IDL)

- 签名信息
  - 名字+输入+输出
- 状态约束
  - 前置条件 + 后置条件
  - 不变条件

- 系统操作合约
- 一阶逻辑 FOL
- 对象约束语言 OCL

回顾：需求工程过程模型

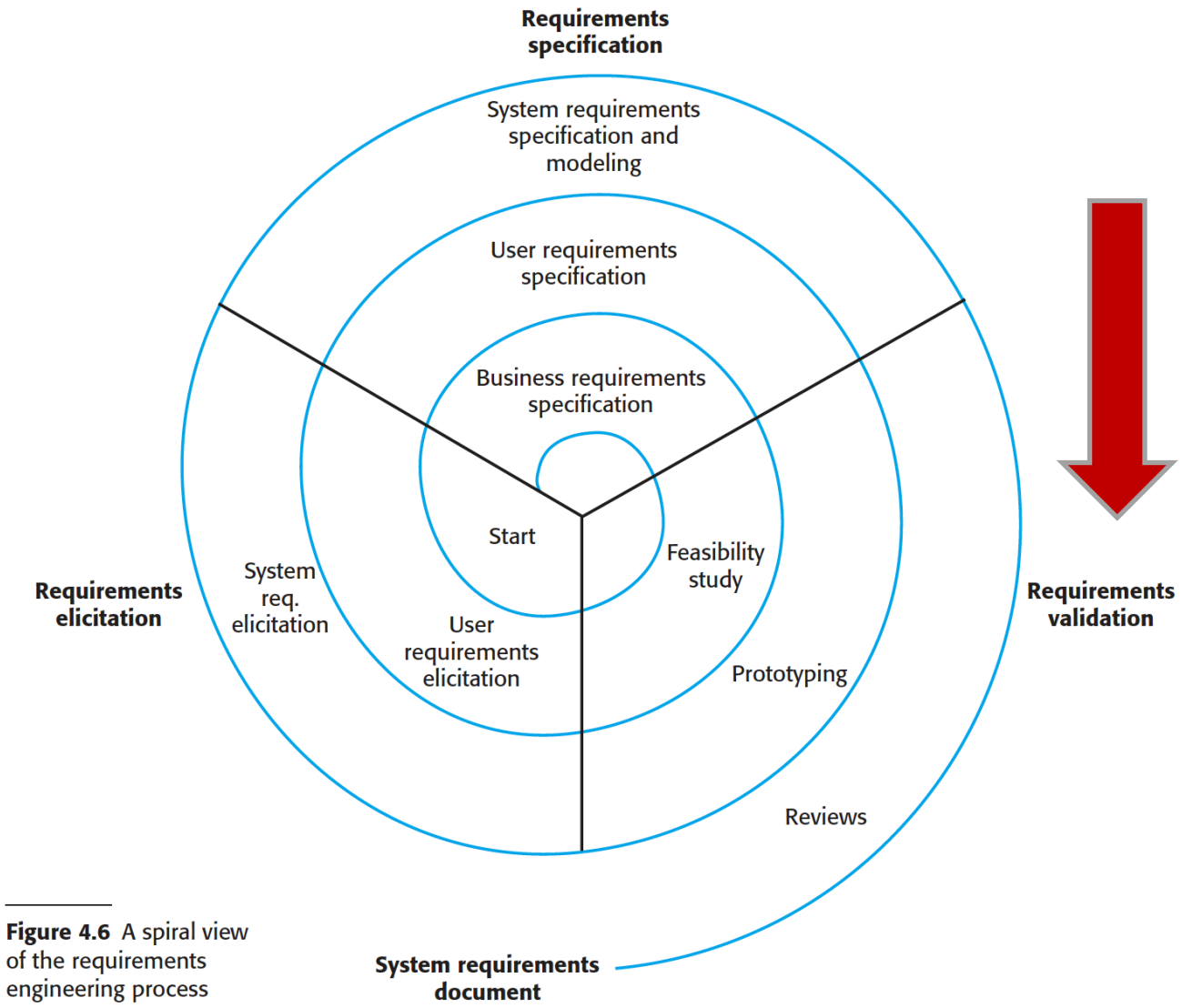


Figure 4.6 A spiral view of the requirements engineering process

需求过程：螺旋模型

- 三层：
  - 业务、用户、系统需求
- 三阶段：
  - 需求获取 (Elicitation)
  - 需求精化与规约 (Refinement&Specification)
  - 需求确认与验证 (Validation&Verfication)

主观题 10分

设置

- 为什么要进行需求的确认和验证？这两个概念有什么区别？



Answer

解决软件需求问题（1/2）

- **遗漏的软件需求**
  - ✓**再次征求**用户、客户、领域专家等意见，以补充遗漏的软件需求
- **无源头的软件需求**
  - ✓剔除或暂时**不用考虑**该部分的软件需求，或者将这些软件需求**置于低优先级**
- **不一致、相冲突的软件需求**
  - ✓**寻找到具有更高级别**的用户或客户，由他们来**最终确定软件需求**
- **不正确和和不准确的软件需求**
  - ✓与用户、客户、领域专家等进行**深入的沟通**，以正确地理解软件**需求的内涵**

解决软件需求问题（2/2）

- **不规范的软件需求文档**
  - ✓对照**软件需求规范标准和模板**，按照其要求来撰写并产生软件需求规格说明书
- **不规范和不正确的软件需求模型**
  - ✓学习UML图符和模型的用法，在此基础上**绘制出正确和规范的UML模型**
- **费解的软件需求文档**
  - ✓消除冗余的文字表述，提高语言表达的简洁性，系统梳理和组织软件需求文档的格式，以提高软件需求文档的可读性

Requirements Validation vs Verification

- **Requirements validation （需求确认） is to confirm that requirements (individually and as a set) define the right system as intended by the stakeholders**
- **Requirements verification （需求验证） is to confirm by examination that requirements (individually and as a set) are well-formed.**
- **Well-form means that a requirement or a set of requirements has been reviewed to help ensure the characteristics of good requirements are achieved and the requirements set is well - organized**

需求验证目标

- **无二义性**就是要**去掉模棱两可**的内容。
- **可验证性**就是要具有**可测试性**， 即需求是具体的、可测试的。
- **可量化性**就是需求的功能与性能要数性能指标、 并发用户数目、平均响应时间。
- **唯一性**就是每项需求给一个**唯一标识**，以便于需求跟踪与配置管理。

需求确认与验证的手段

- **需求评审 - Requirements reviews**
  - ✓Systematic manual analysis of the requirements.
- **原型生成 – Prototyping**
  - ✓Using an executable model of the system to check requirements. Covered in Chapter 2.
- **测试用例生成 Test-case generation**
  - ✓Developing tests for requirements to check testability.

软件需求评审的步骤

- 阅读和汇报软件需求制品
- 收集和整理问题
- 讨论和达成一致
- 纳入配置

为什么要评审软件需求?



评审软件需求（1/2）

- **内容完整性**，是否包含了用户和客户的所有软件需求
- **内容正确性**，软件需求是否客观、正确地反映了用户和客户的实际期望和要求
- **内容准确性**，是否准确地反映了用户和客户的期望和要求
- **内容一致性**，所描述的软件需求是否存在不一致问题
- **内容多余性**，所描述的软件需求是否都是用户所期望的
- **内容可追踪性**，每一项软件需求是否可追踪的

评审软件需求（2/2）

- **文档规范性**，软件需求规格说明书书写是否遵循文档规范
- **图符规范性**，软件需求模型是否正确地使用了UML的图符
- **表述可读性**，软件需求文档文字表述是否简洁、可读性好
- **图表一致性**，软件需求制品中的图表引用是否正确

软件原型

- **需求工程师根据需求描述，快速构造出一个可运行的软件原型**
  - ✓**展示业务操作流程**以及**每一个步骤**用户与软件之间的交互
  - ✓用户可以通过操作和使用该软件，分析需求工程师**是否正确地理解了他们所提出的软件需求**，发现软件原型所展示的软件需求中存在的问题，**导出尚未发现、新的软件需求**
- **软件原型作为需求工程师和用户之间的交流媒介，有助于直观地展示软件需求，激发用户投入到需求讨论和导出之中，因而是一项极为有效的需求获取和分析方法**

回顾：需求工程过程模型

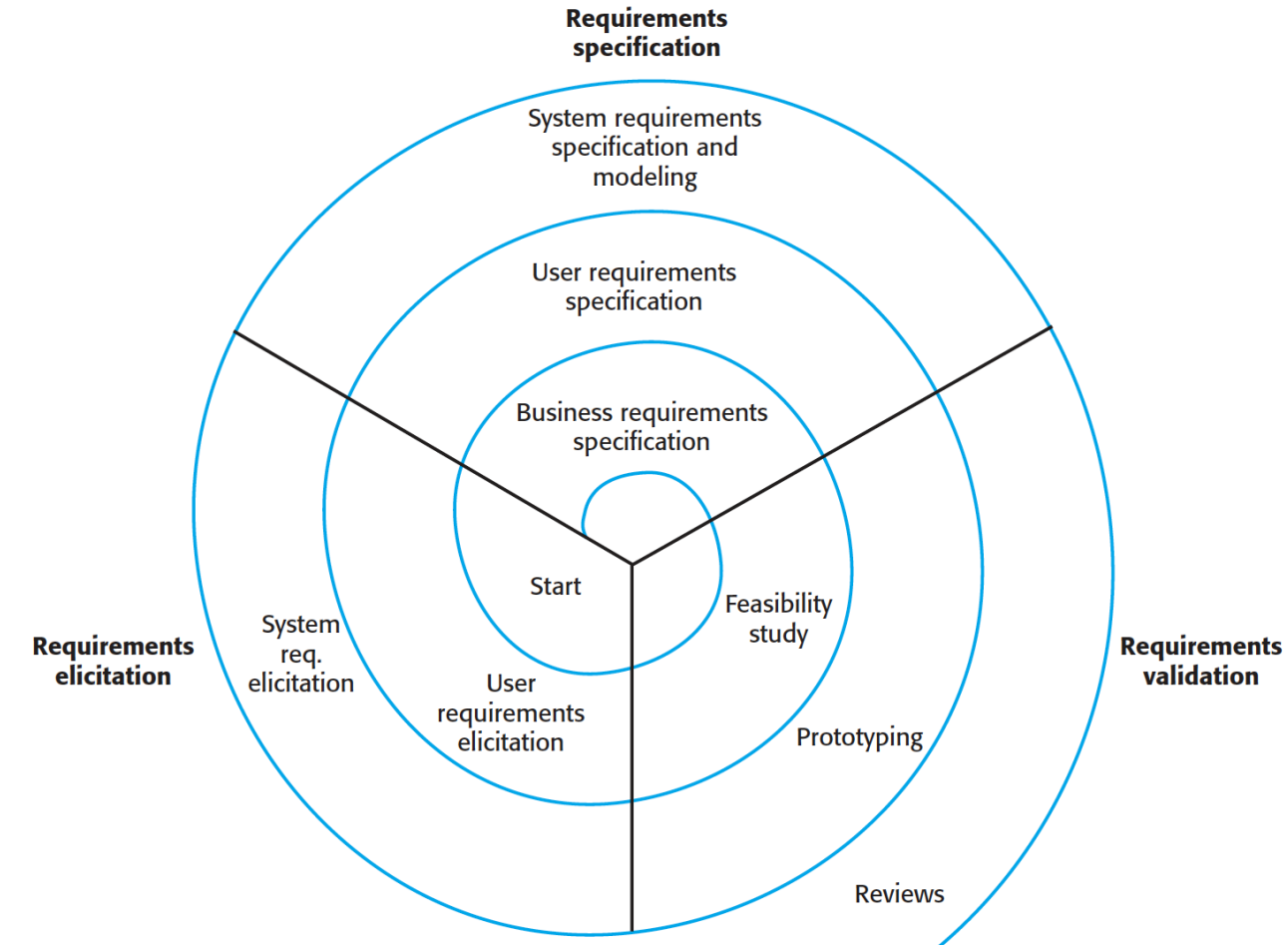


Figure 4.6 A spiral view of the requirements engineering process

需求过程：螺旋模型

- 三层：
  - 业务、用户、系统需求
- 三阶段：
  - 需求获取 (Elicitation)
  - 需求精化与规约 (Refinement&Specification)
  - 需求确认与验证 (Validation&Verfication)

软件需求分析的输出

❑软件需求模型

- ✓以可视化的图形方式，从多个不同的视角，直观地描述了软件的功能性需求，包括用例模型、用例的交互模型、分析类模型、状态模型等

❑软件原型

- ✓以可运行软件的形式，直观地展示了软件的业务工作流程、操作界面、用户的输入和输出等方面的功能性需求信息

❑软件需求文档

- ✓以图文并茂的方式，结合需求模型以及需求的自然语言描述，详尽刻画了软件需求，包括功能性和非功能性软件需求，软件需求的优先级列表等

用户需求报告

- **《用户需求报告》**是站在用户的角度、使用他们可以看懂的语言写的，内容是有关系统的**运行环境、业务流程 、业务功能、业务性能和业务接口**等。

- 需求规格说明的特点：**完整性、一致性、可修改性、可跟踪性**

需求规格说明

- **需求规格说明**是它是**需求分析阶段**产生的一份重要的文档，表达了用户**全面的、系统的、准确的、 并且用户确认的需求**，它是用户、项目开发者、项目测试者和项目管理者四方共同工作的基础，**是用户测试和验收目标的依据**，是作为软件开发机构和用户之间一份事实上的技术合同书，是软件生命周期中的第一根基线。

- **《需求分析规格说明书》**是**站在开发者**的角度、使用开发者的语言写的，目的是**作为概要设计和详细设计的依据**，内容是对系统的业务模型、功能模型、数据模型和接口模型的进一步定量描述。它是需求分析阶段产生的第二份重要的文档。

撰写软件需求文档的注意事项

- **遵循规范**，按照规范来撰写软件需求文档
- **图文并茂**，将软件需求模型以及自然语言描述二者结合在一起，给出软件需求的清晰、准确和详实的表述
- **完整表述**，要给出软件功能性需求和非功能性需求的描述
- **共同参与**，需求工程师要与用户、客户等一起参与软件需求规格说明书的撰写
- **语言简练**，软件需求规格说明书的语言表述要简练，便于阅读和理解
- **前后一致**，在软件需求文档中，对同一个软件需求的表述前后要一致，不要产生相互矛盾或不一致的需求表达

软件需求文档模板

1. 引言
    - 1.1 编写目标
    - 1.2 读者对象
    - 1.3 文档概述
    - 1.4 术语定义
    - 1.5 参考文献
2. 软件系统概述
    - 2.1 软件产品概述
    - 2.2 用户特征
    - 2.3 设计和实现约束
    - 2.4 假设与依赖

3. 功能性需求描述
    - 3.1 软件功能概述
    - 3.2 软件需求的用例模型
    - 3.3 软件需求的分析模型

4. 非功能性需求
5. 界面需求
6. 接口定义
7. 进度要求
8. 交付要求
9. 何种形式来交付
10. 验收要求

软件需求模型是文档中的重要组成成分

主观题 10分

设置

❑如果软件需求模型和文档存在诸多的问题，没有得到有效的解决，会产生什么样的后果?



Answer

需求管理活动

- 需求管理活动包括：（Evolution）
  1）定义**需求基线**
  2）**评审**提出的**需求变更**、评估每项变更的**可能影响**，从而决定是否实施它。
  3）以一种可控制的方式将需求**变更融入**到项目中。
  4）使当前的项目计划与需求一致。
  5）估计变更需求所产生影响并在此基础上协商新的承诺（约定）。

需求工程管理活动

- 需求管理活动还包括：（Traceability）

  1）让每项需求都能与其对应的设计、源代码和测试用例联系起来以实现跟踪。

  2）在整个项目过程中跟踪需求状态及其变更情况。

北京航空航天大學
BEIHANG UNIVERSITY

Thank you

北京航空航天大学